# Lab 11: Linux Essentials

Brittany Rose

CITC 1332 Linux Operating System

Instructor: Dr. Noman Saied

04/02/2026

# Table of Contents

# 11.1 Introduction

This is **Lab 11: Basic Scripting**. By performing this lab, students will learn how to use the **vi** editor to create basic shell scripts using basic shell commands, variables and control statements.

In this lab, you will perform the following tasks:

- Use the **vi** editor to create and edit text files.
- Create simple shell scripts.
- Create shell scripts with conditional execution.
- Use loops in the script for repetition.

# 11.2 Basic Text Editing

Most distributions of Linux have more than one text editor. These may include simple text-only editors, such as `nano`, or graphical editors, such as `gedit`.

In this task, you will explore some of the basic text editing features of the `vi` editor. All distributions have some version of `vi`. The `vi` editor is a powerful text editor with a bit of a learning curve, but capable of performing a wide variety of text editing tasks.

The `vi` editor has two modes: insert and command. In insert mode, you add text to a document. In command mode, operations such as navigation, searching, saving, and exiting the editor can be performed.

# 11.2.1 Step 1

To create a new file, execute the following command:

```
vi myfile
```

```
sysadmin@localhost:~$ vi myfile
```

Type an **i** to enter the "insert" mode of `vi` (more about this later). Then enter the following text:

```
Welcome to the vi editor.
It is a very powerful text editor.
Especially for those who master it.
```

Then press the **Esc** key to leave insert mode. Type `:wq` to write the file to disk and quit.

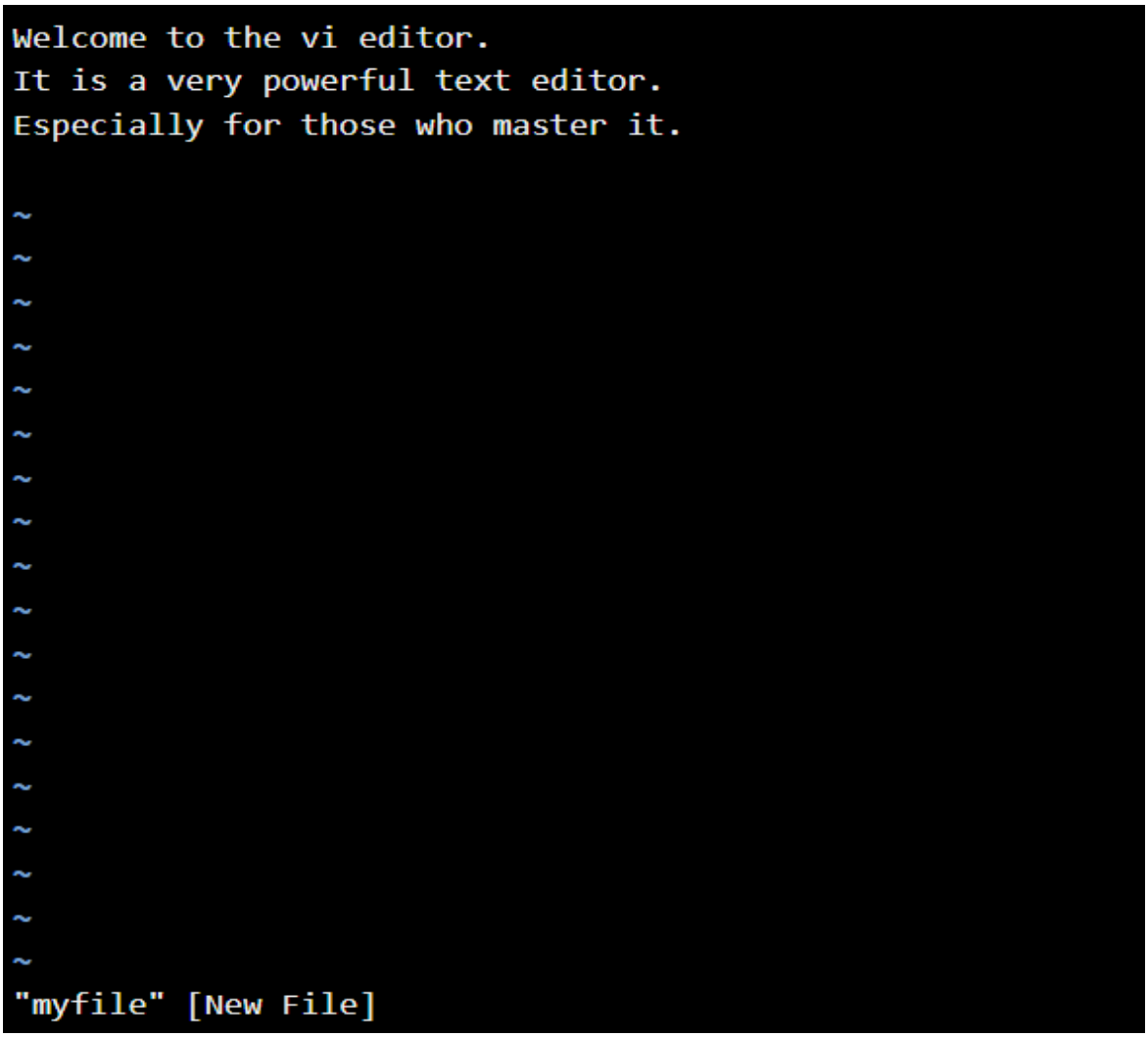
> **Note**
>
> Each of the previous commands will be covered in greater detail later in this lab. The purpose of the previous step was to create a file to work with during this lab.

# 11.2.2 Step 2

Invoke the `vi` editor to modify the file you created. When `vi` is invoked, you are placed in *command* mode by default:

```
vi myfile
```

Your output should be similar to the following:



# 11.2.3 Step 3

Press each of the following keys two times and observe how the cursor moves. Remember that you are in command mode:

| Key | Function |
|---|---|
| `j` | Moves cursor down one line (same as down arrow) |
| `k` | Moves cursor up line (same as up arrow) |
| `l` | Moves cursor to the right one character (same as right arrow) |
| `h` | Moves cursor to the left one character (same as left arrow) |
| `w` | Moves cursor to beginning of next word |
| `e` | Moves cursor to end of word |
| `b` | Moves cursor to beginning of previous word |

**Warning:** If you type any other keys then those listed above, you may end up in insert mode. Don't panic! Press the **Esc** key, then `:q!` + the **Enter** key. This should exit `vi` without saving any changes. Then execute `vi myfile` and you are back in the `vi` editor!

# 11.2.4 Step 4

More vi cursor navigation: press the following keys and observe how the cursor moves:

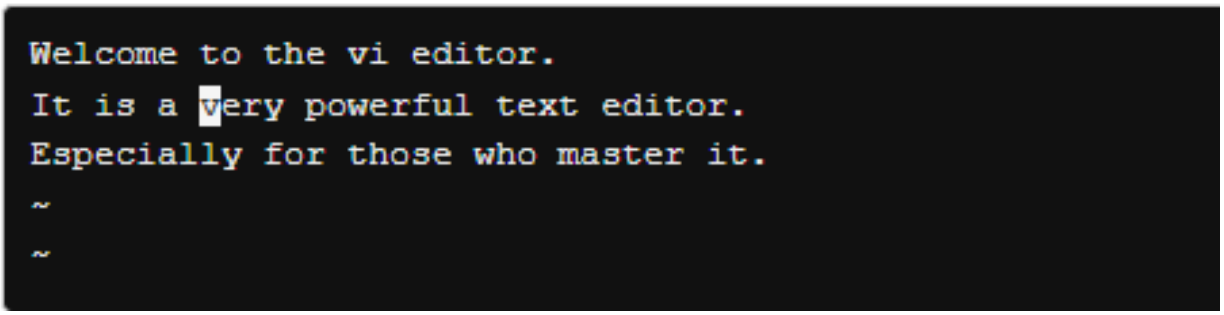
| Keys | Function |
| --- | --- |
| `$` | Moves cursor to end of current line (same as **End** key) |
| `0` (zero) | Moves cursor beginning of current line (same as **Home** key) |
| `3G` | Jumps to third line ( `nG` jumps to the nth line) |
| `1G` | Jumps to first line |
| **Shift+G** | Jumps to the last line |

# 11.2.5 Step 5

Move the cursor to the beginning of the word "very" by pressing the following keys:

```
3G (hold down the number "3" key and press the letter "g")
k
81 (that's the number eight followed by the letter "l")
```

The cursor should be on the letter v of the word "very" as shown below:

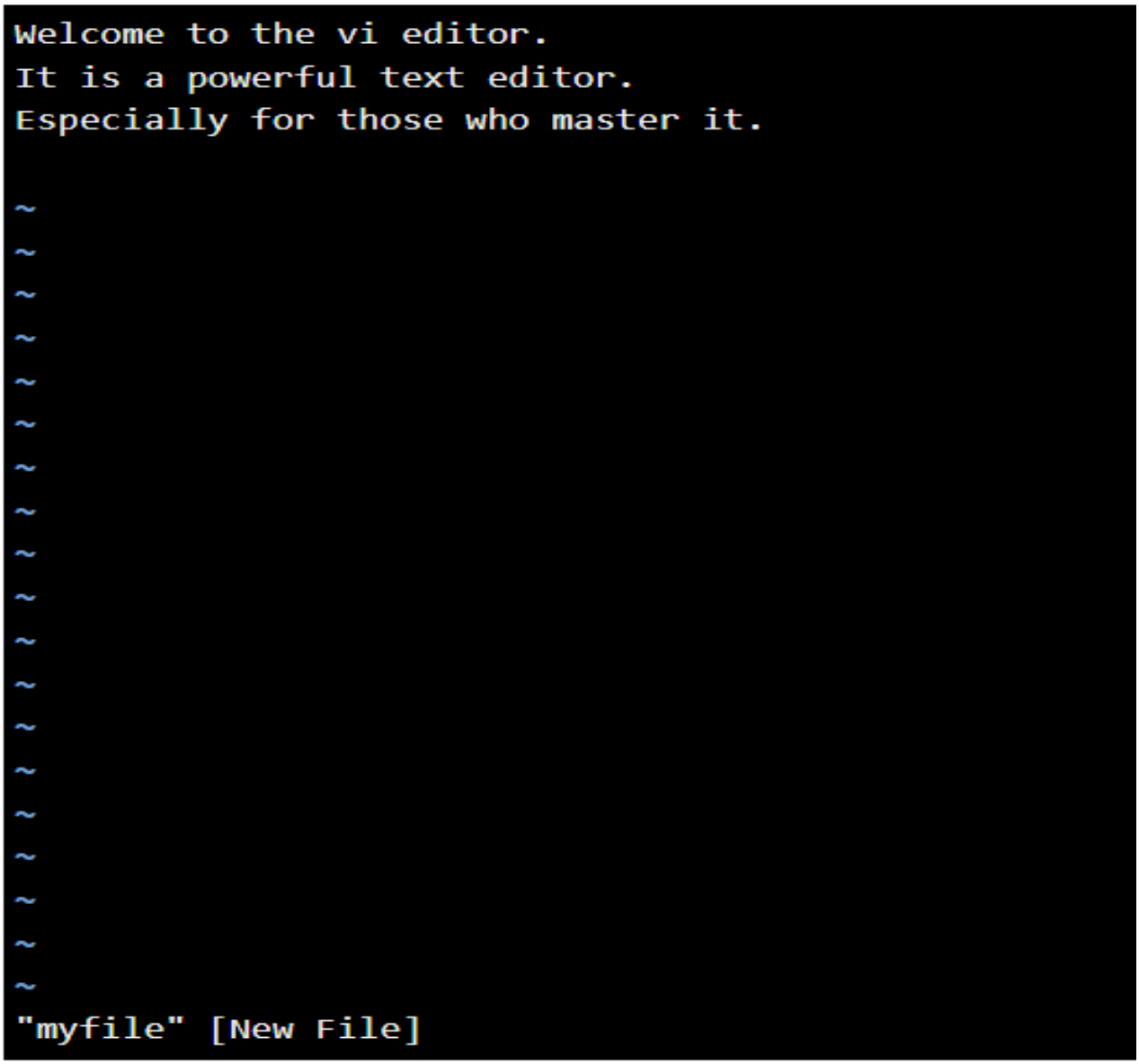


# 11.2.6 Step 6

Delete the word “very” by issuing the command dw (**d**elete **w**ord):

```
dw
```

Your screen should look similar to the following:

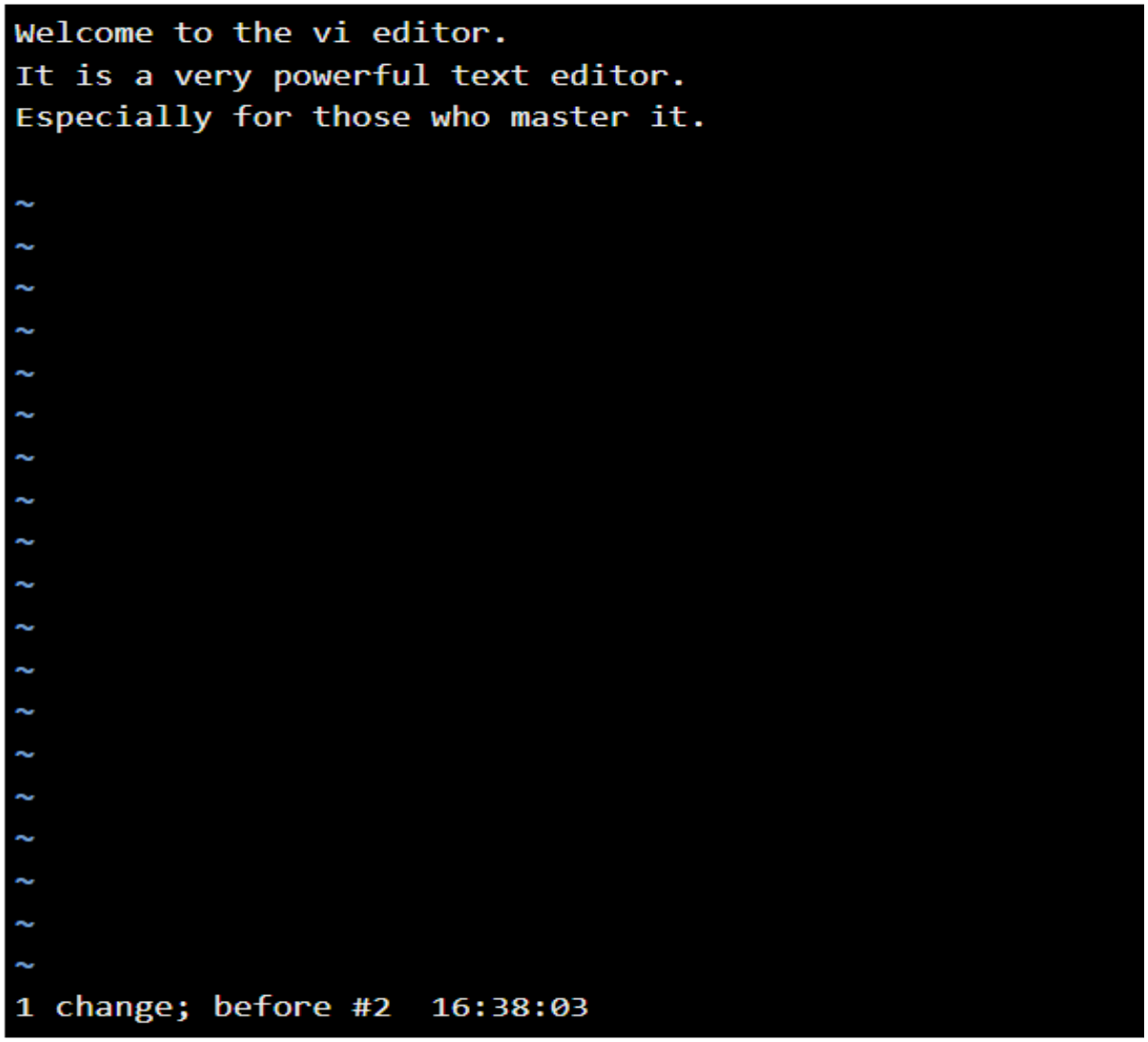


# 11.2.7 Step 7

Undo the last operation:

```
u
```

Your screen should look similar to the following:

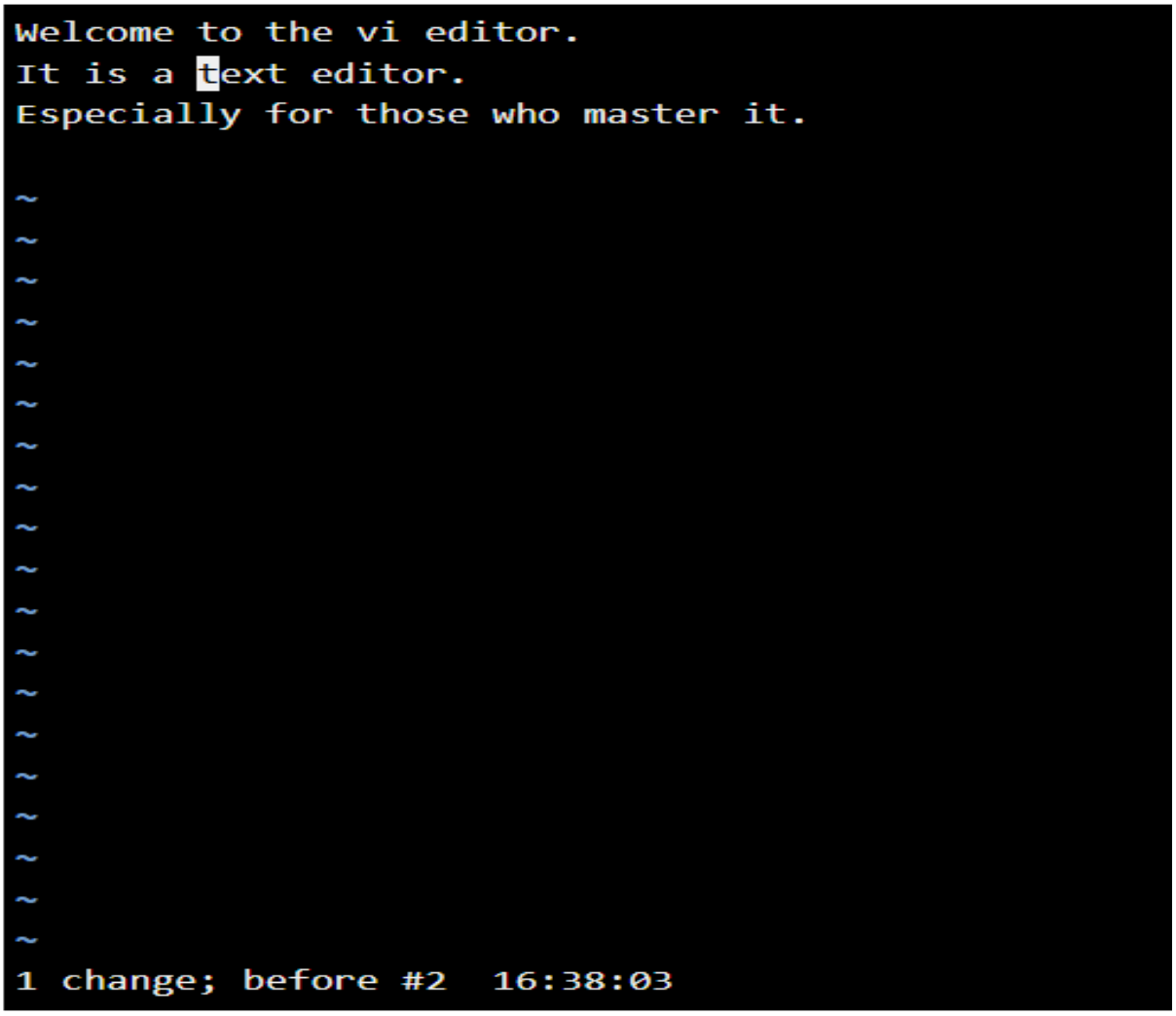


# 11.2.8 Step 8

Delete two words:

```
2dw
```

Your screen should look similar to the following:

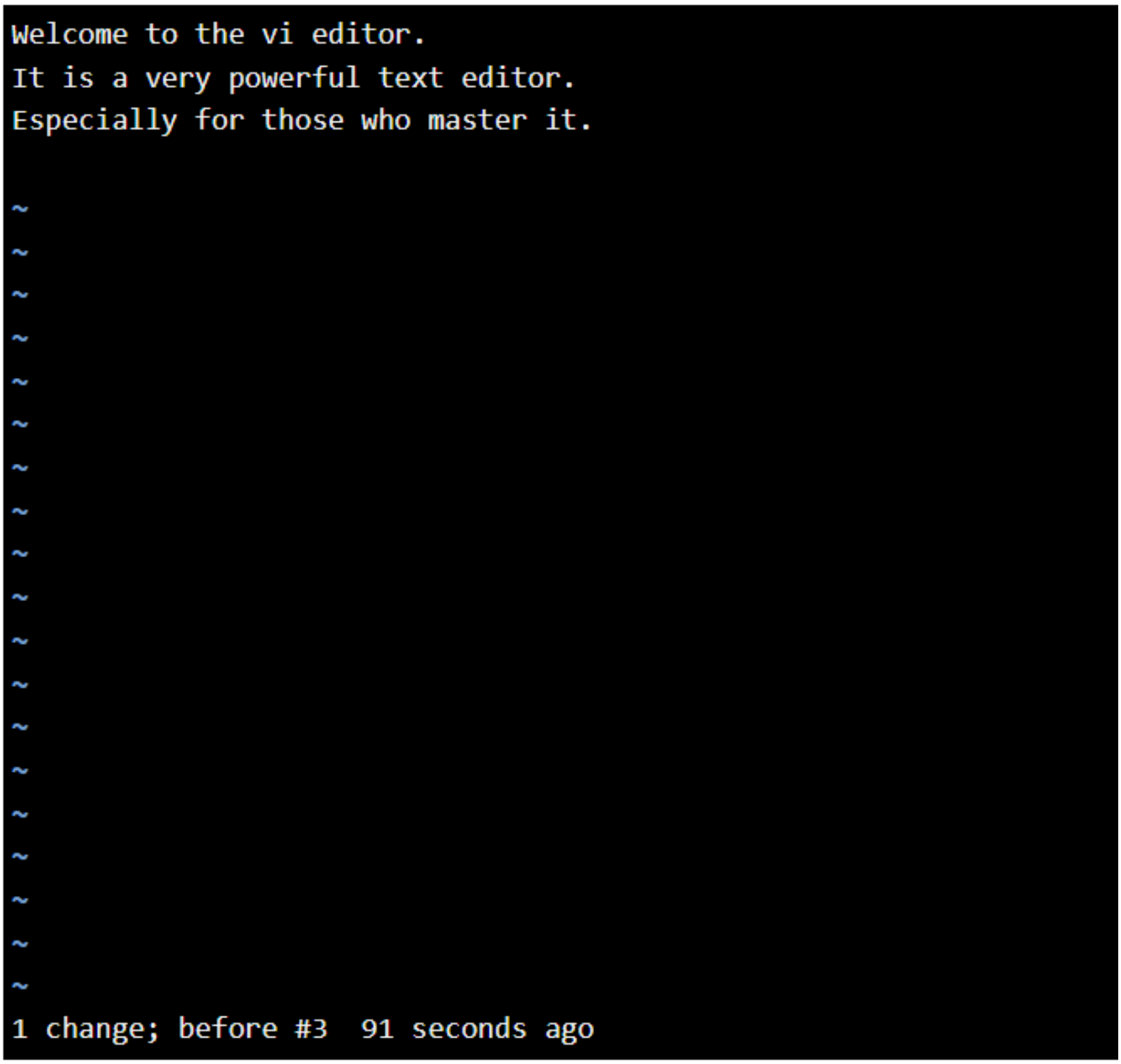


# 11.2.9 Step 9

Undo the last operation:

```
u
```

Your screen should look similar to the following:

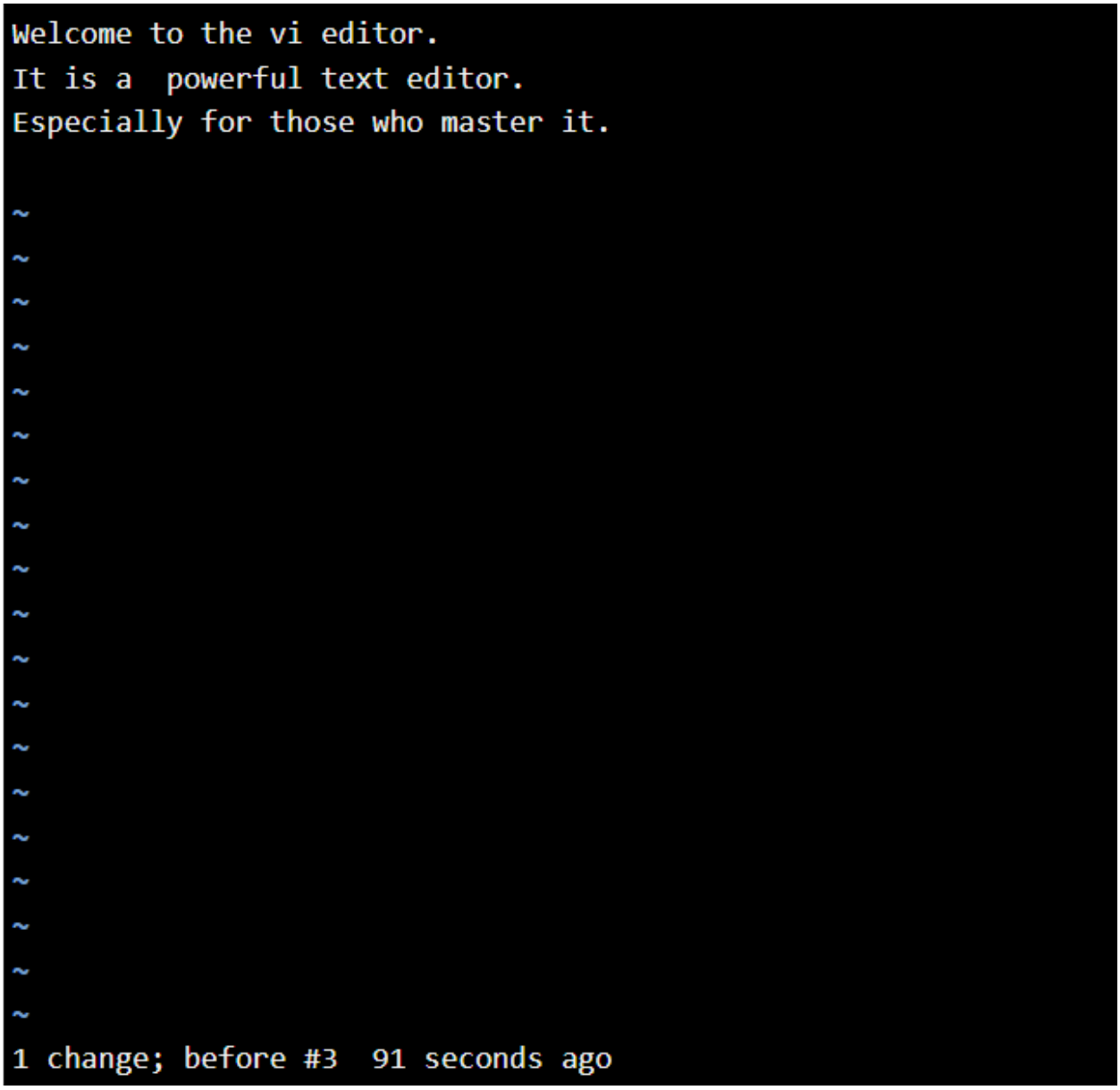


# 11.2.10 Step 10

Delete four characters, one at a time:

```
xxxx
```

Your screen should look similar to the following:

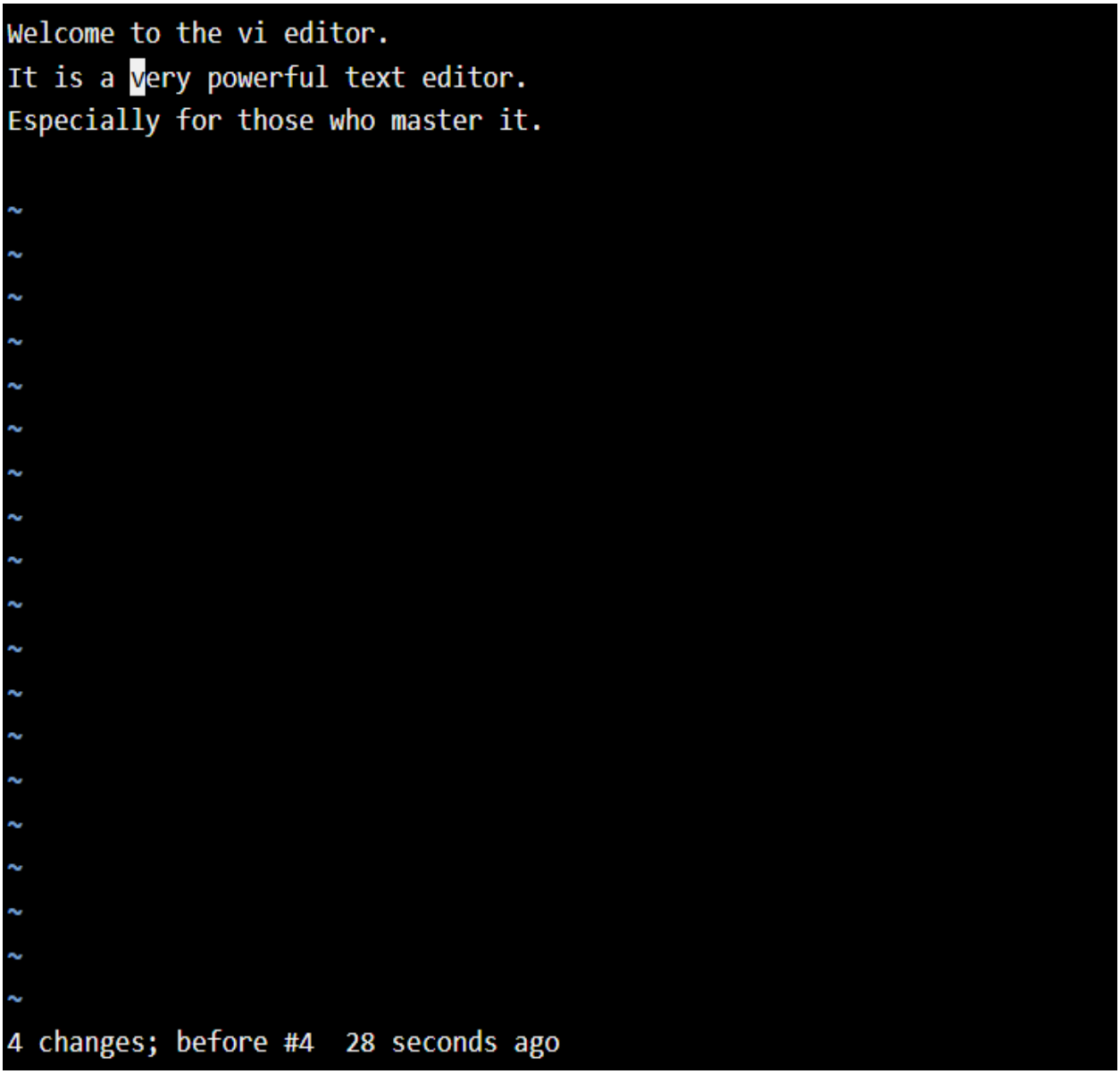


# 11.2.11 Step 11

Undo the last 4 operations and recover the deleted characters:

```
4u
```

Your screen should look similar to the following:

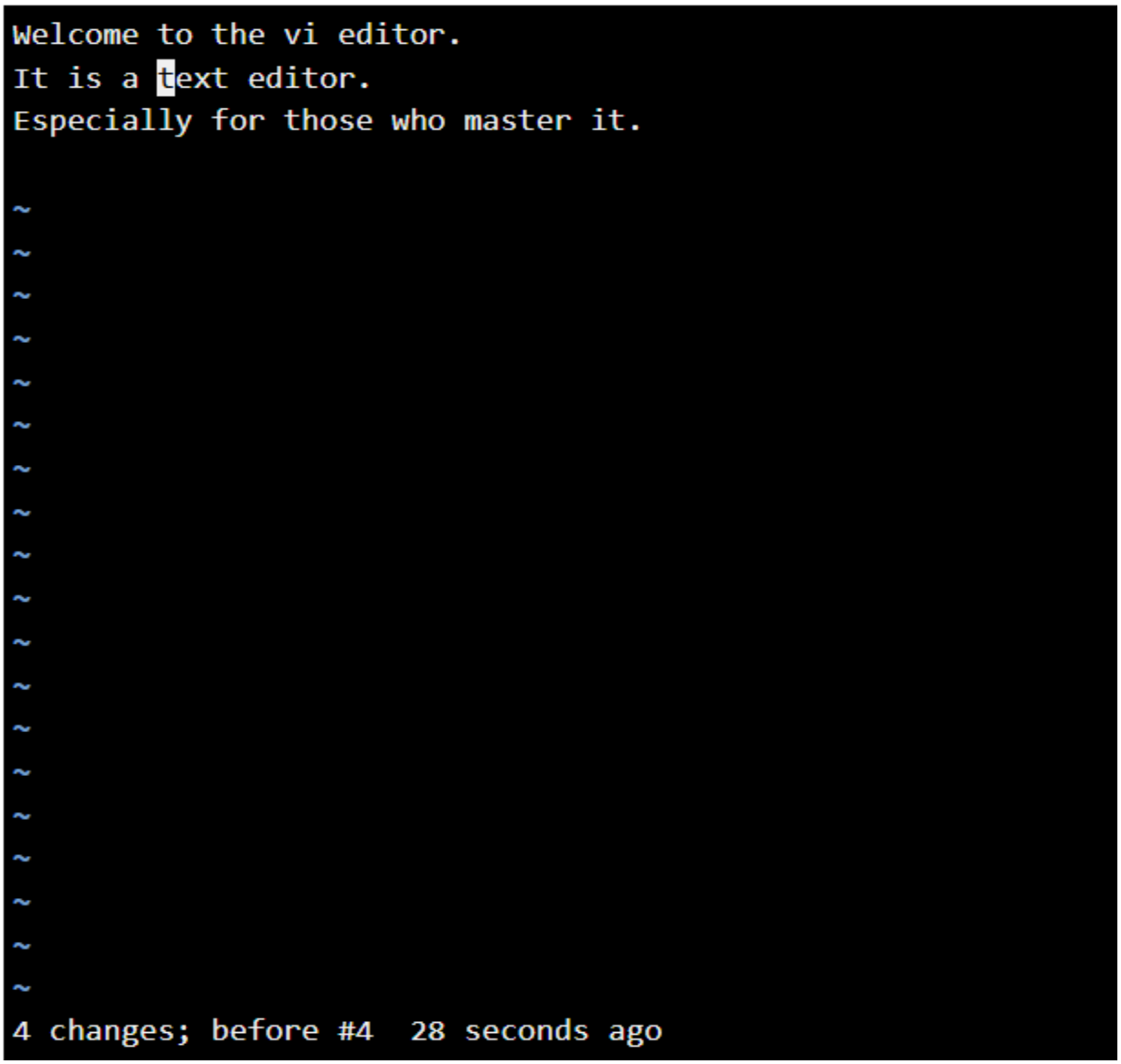


# 11.2.12 Step 12

Delete 14 characters:

```
14x
```

Your screen should look similar to the following:

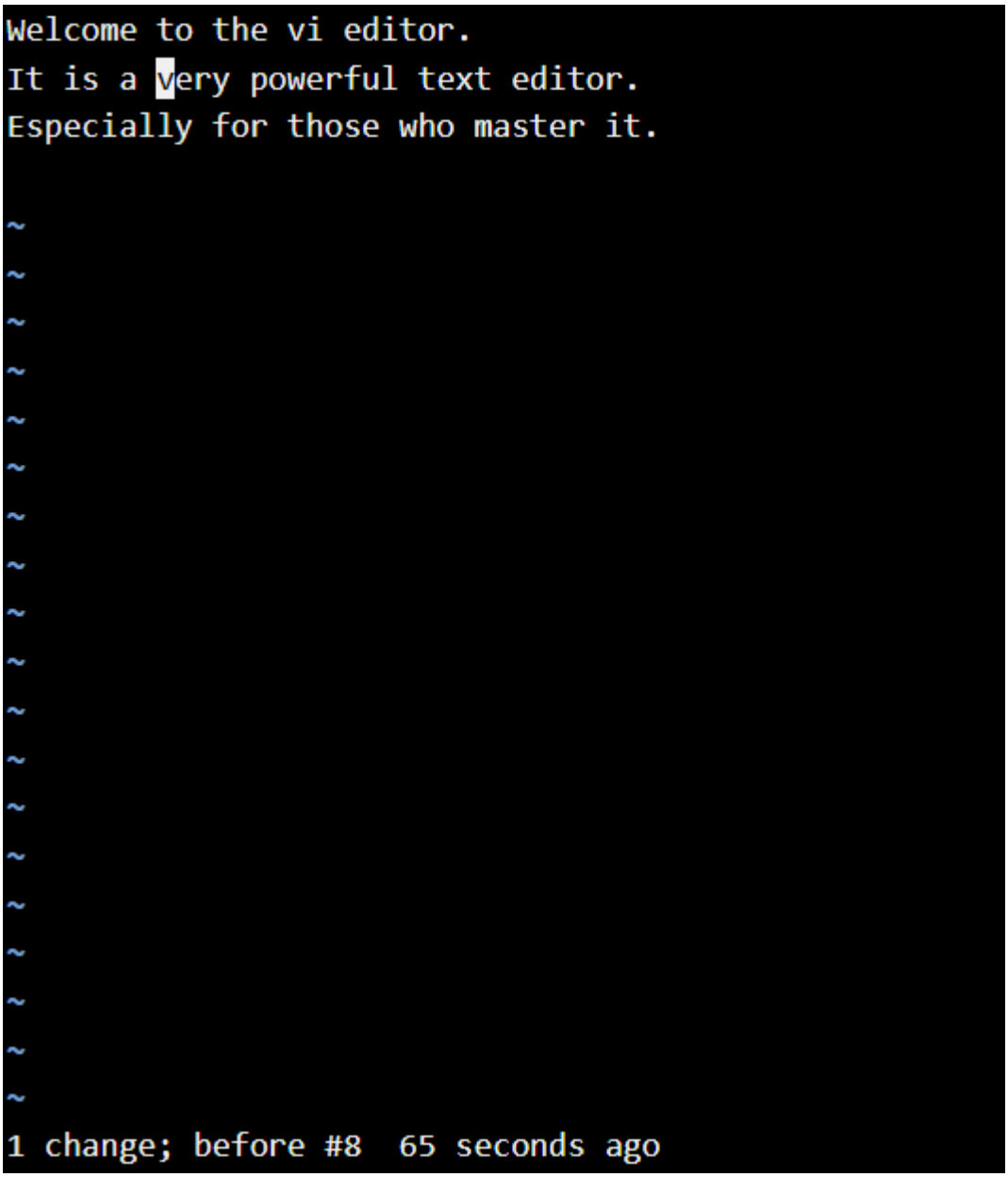


# 11.2.13 Step 13

Undo the last operation:

```
u
```

Your screen should look similar to the following:

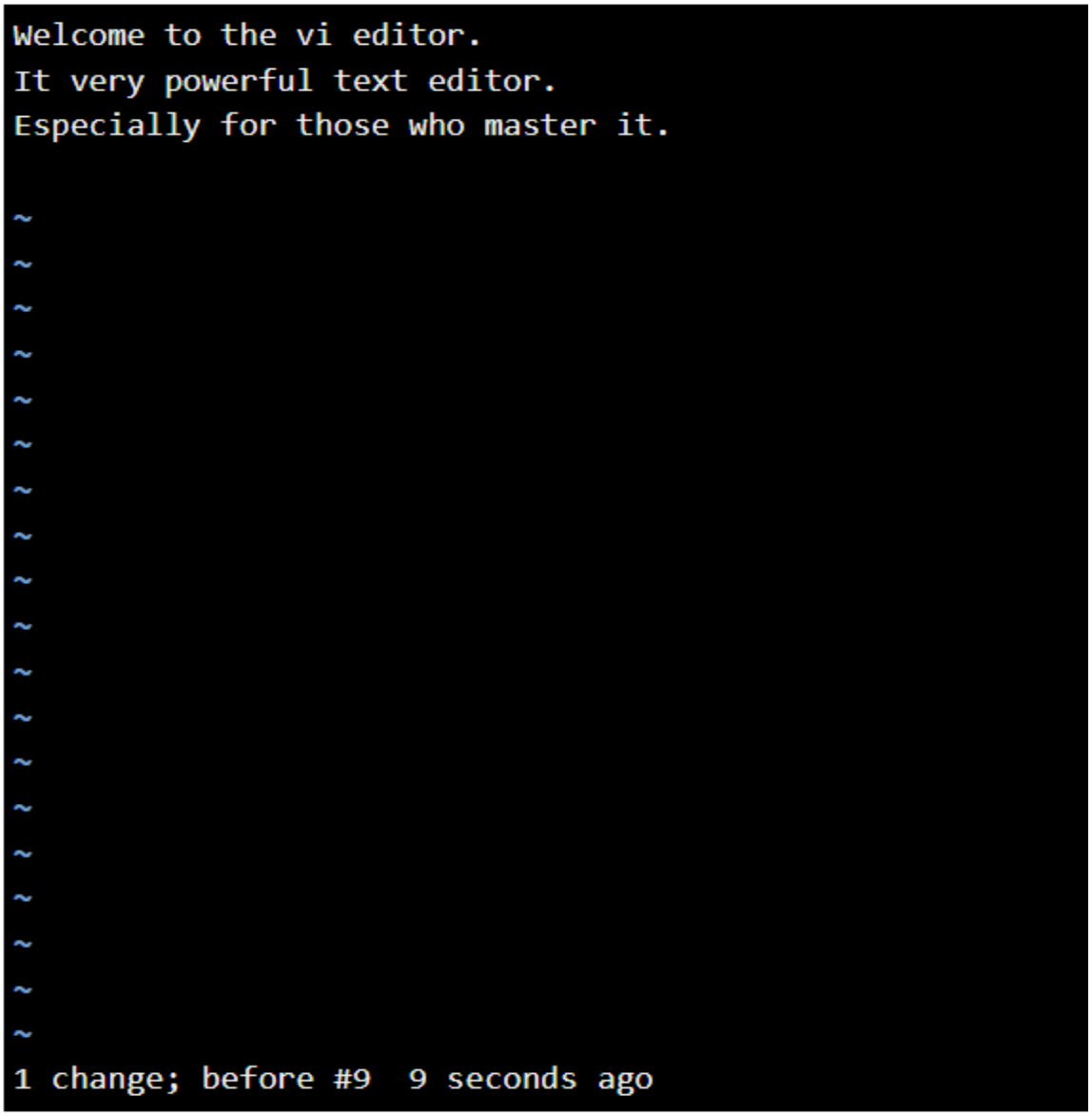


# 11.2.14 Step 14

Delete the five characters to the left of the cursor (type `5` then **Shift+x**):

```
5X
```

Your screen should look similar to the following:

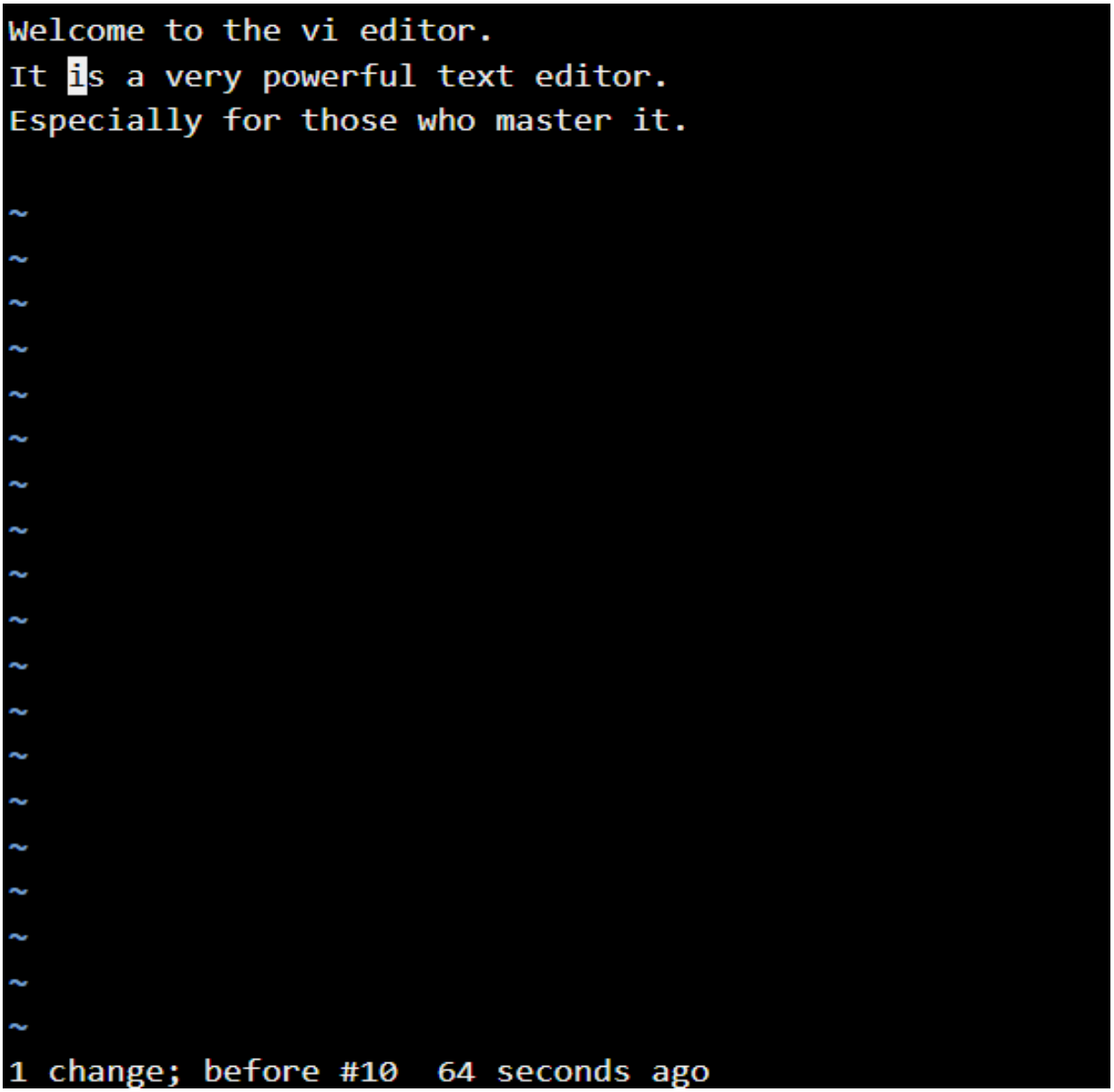


# 11.2.15 Step 15

Undo the last operation:

```
u
```

Your screen should look similar to the following:

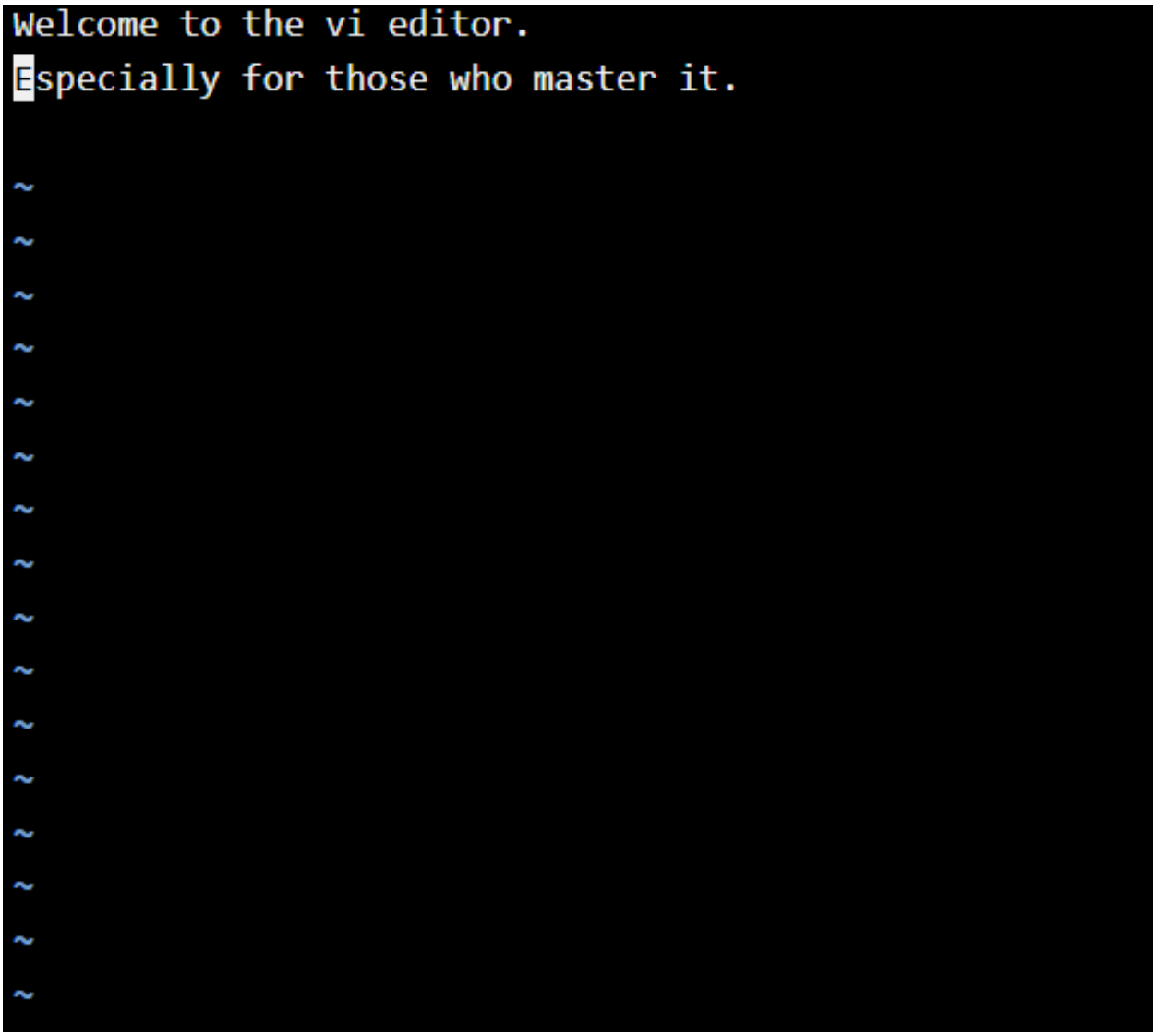


# 11.2.16 Step 16

Delete the current line:

```
dd
```

Your screen should look similar to the following:

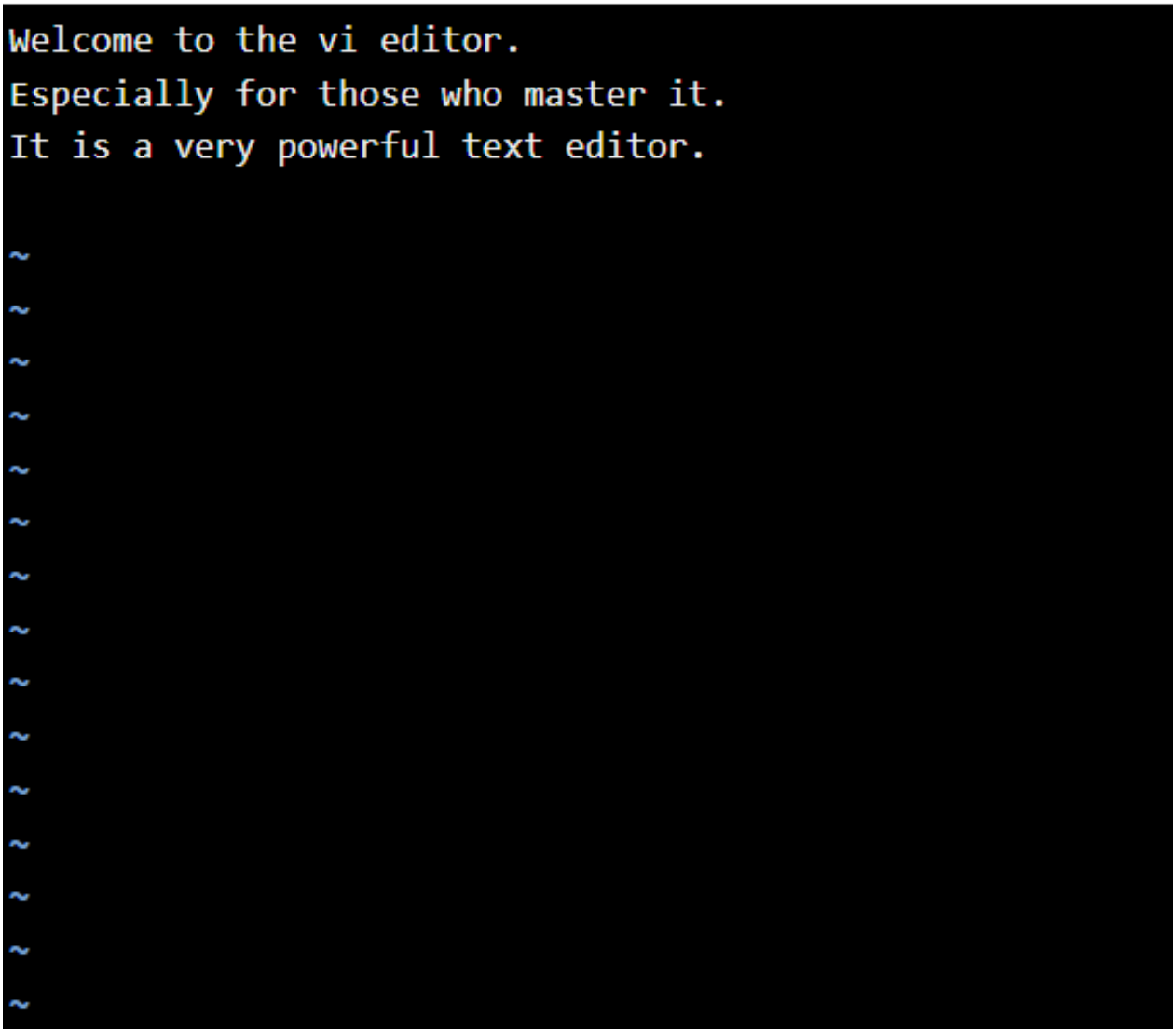


# 11.2.17 Step 17

Whatever was lasted deleted or yanked can be “pasted”. Paste the deleted lines below the current line:

```
p
```

Your screen should look similar to the following:

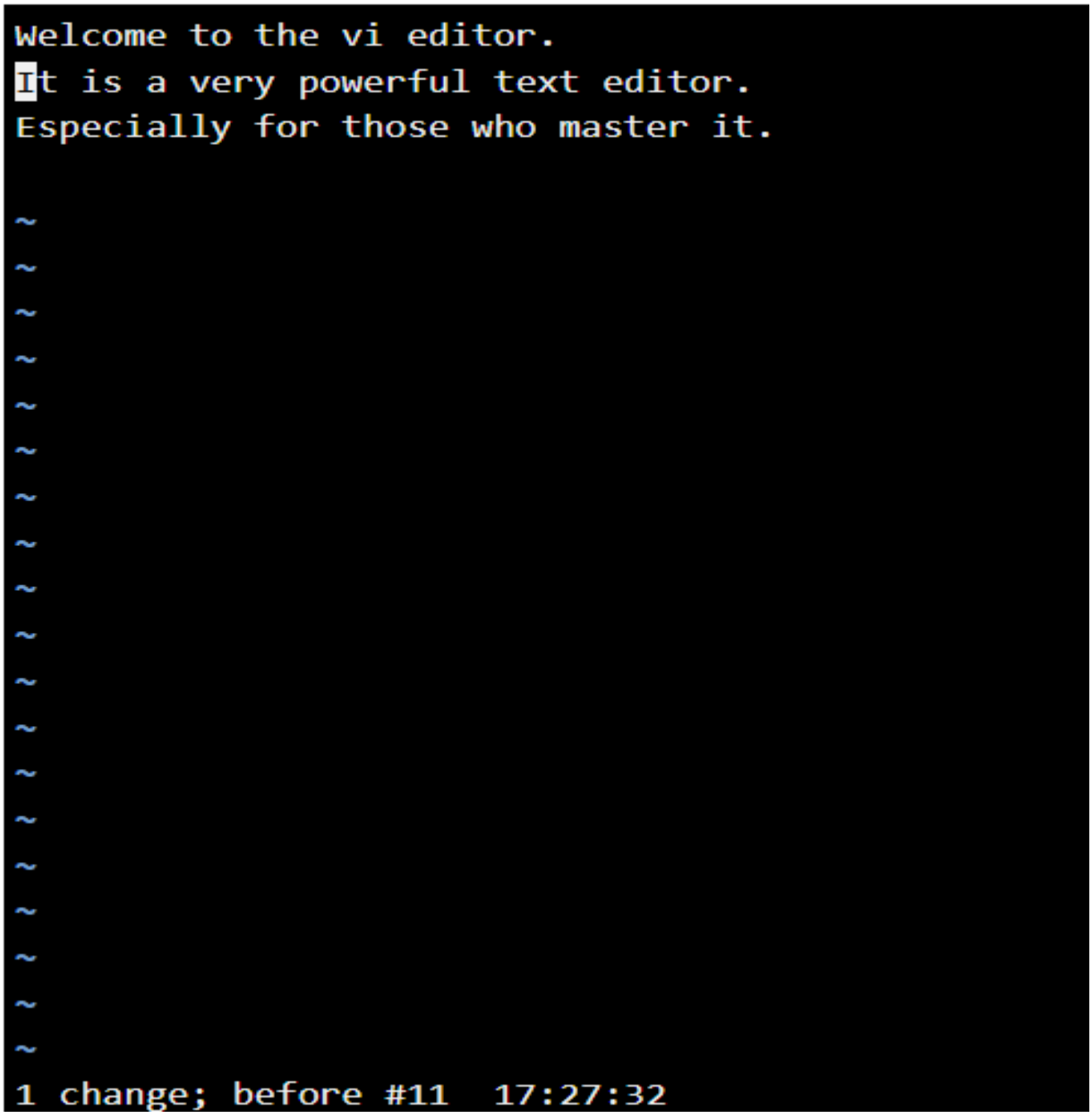


# 11.2.18 Step 18

Undo the last two operations:

```
2u
```

Your screen should look similar to the following:

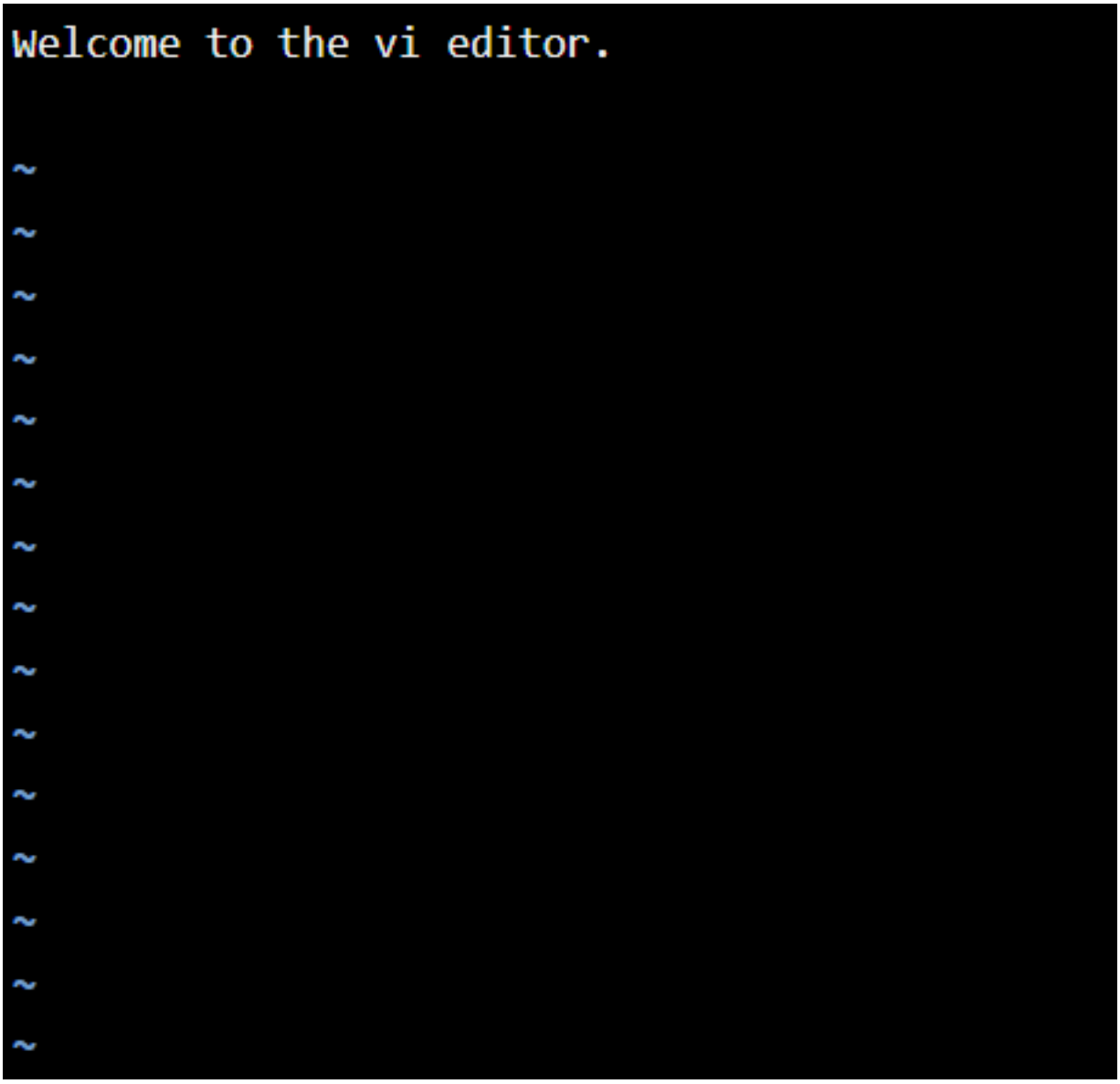


# 11.2.19 Step 19

Delete two lines, the current and the next:

```
2dd
```

Your screen should look similar to the following:

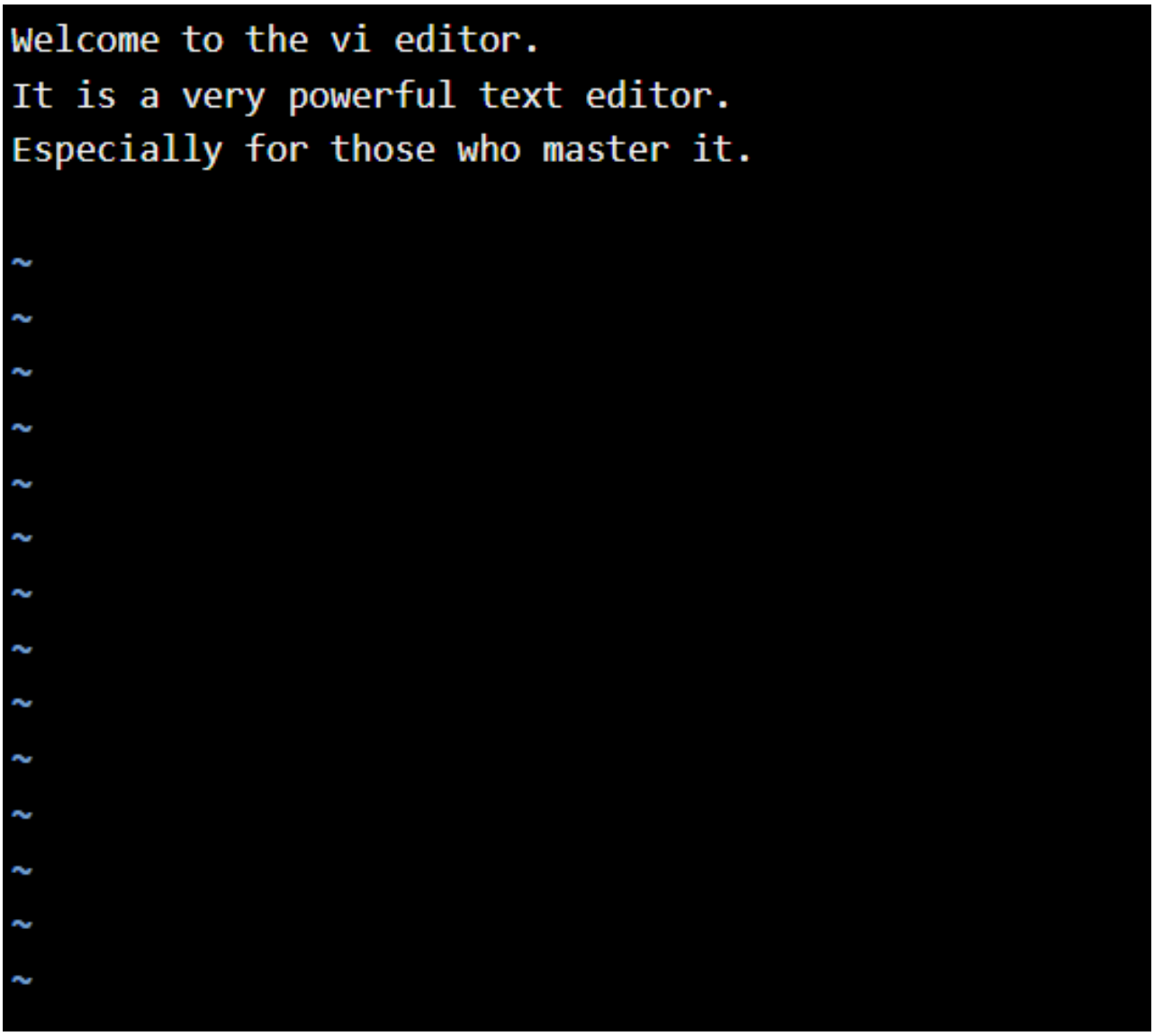


# 11.2.20 Step 20

Undo the last operation:

```
u
```

Your screen should look similar to the following:

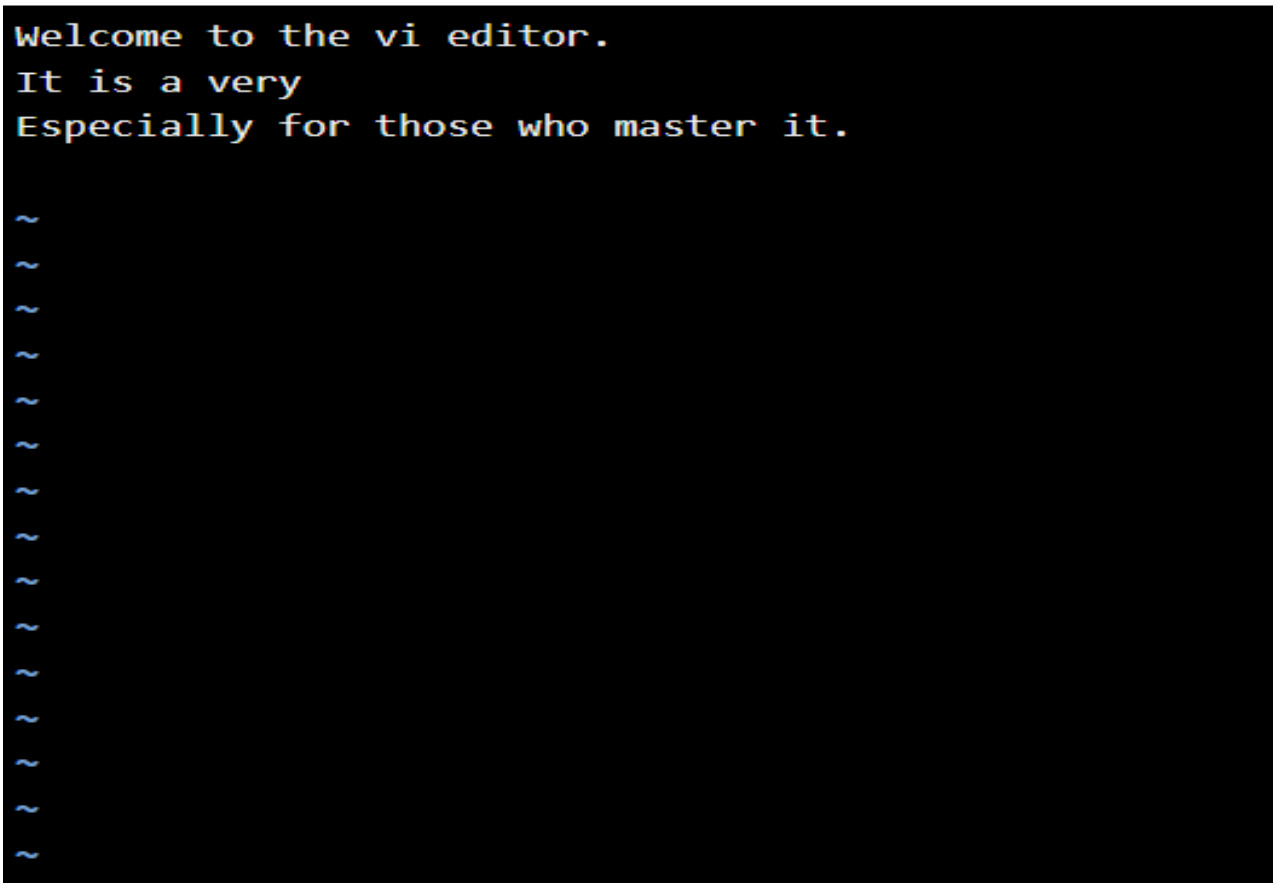


# 11.2.21 Step 21

Move to the fourth word then delete from the current position to the end of the line **Shift+D**:

```
4w
D
```

Your screen should look similar to the following:



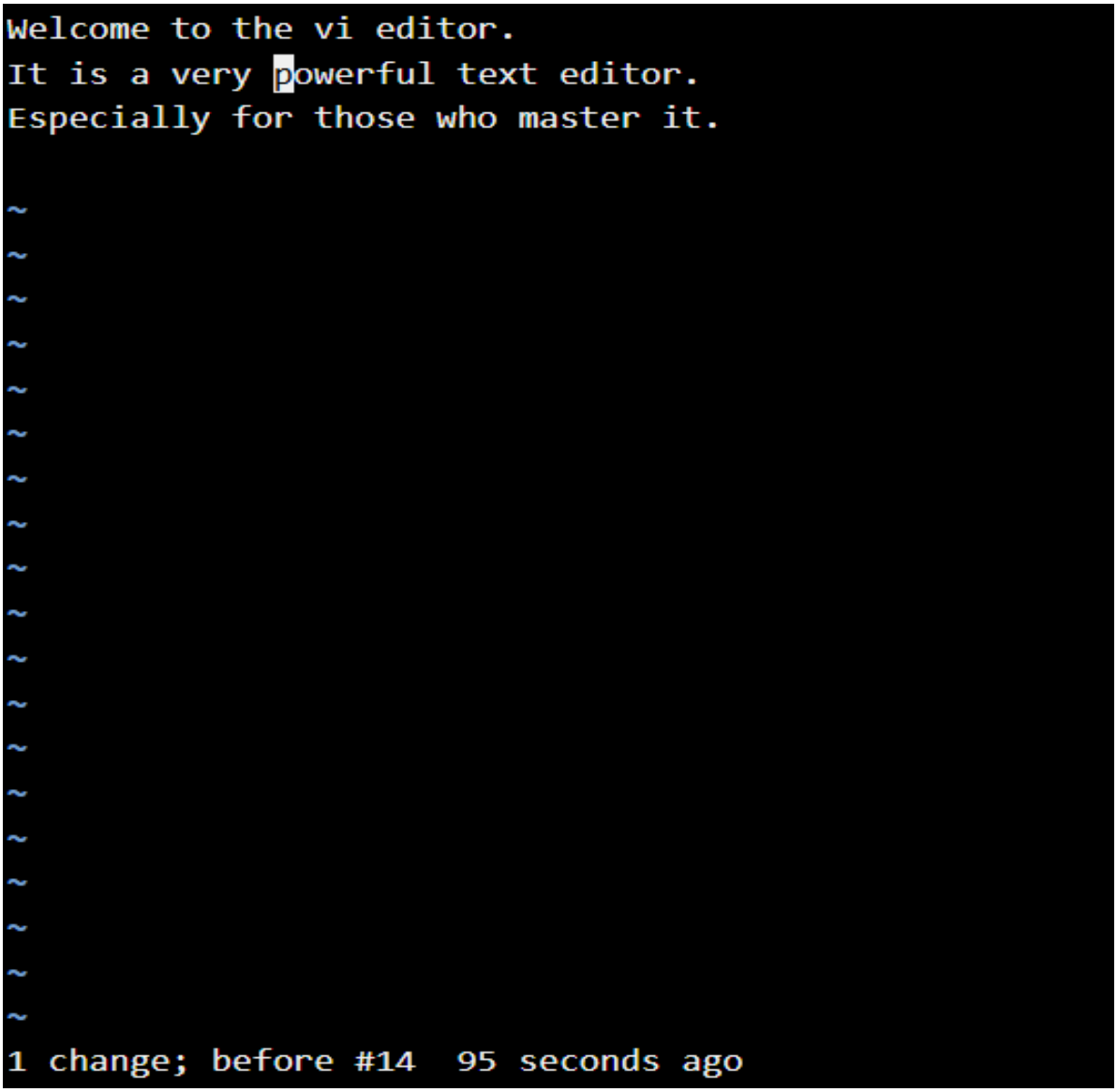

The command **d$** also deletes to end of line. The **$** character, as seen earlier, advances to end of line. Thus, **d$** deletes to end of line.

# 11.2.22 Step 22

Undo the last operation:

```
u
```

Your screen should look similar to the following:

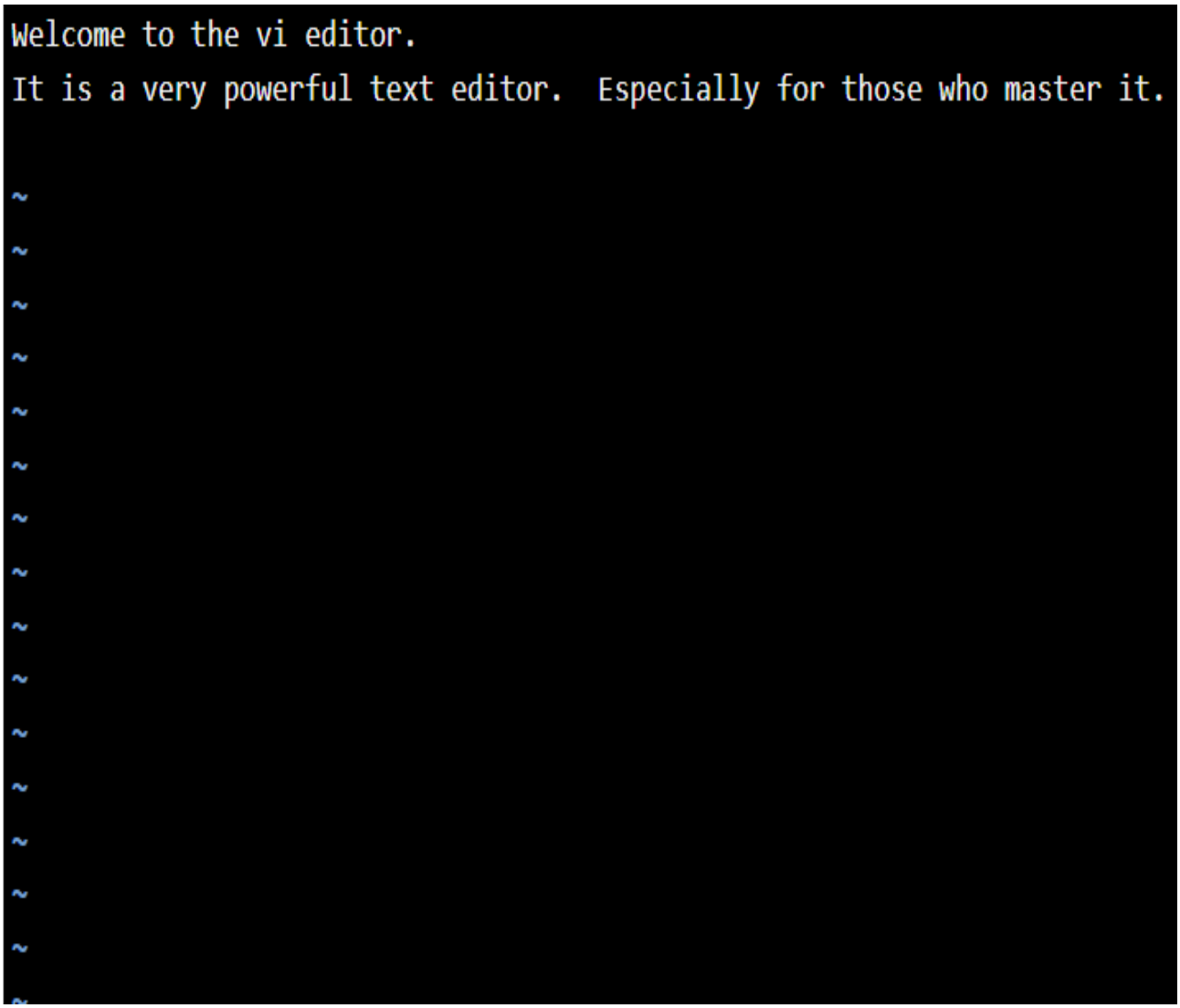


# 11.2.23 Step 23

Join two lines, the current and the next by typing a capital `J` (**Shift+J**):

```
J
```

Your screen should look similar to the following:

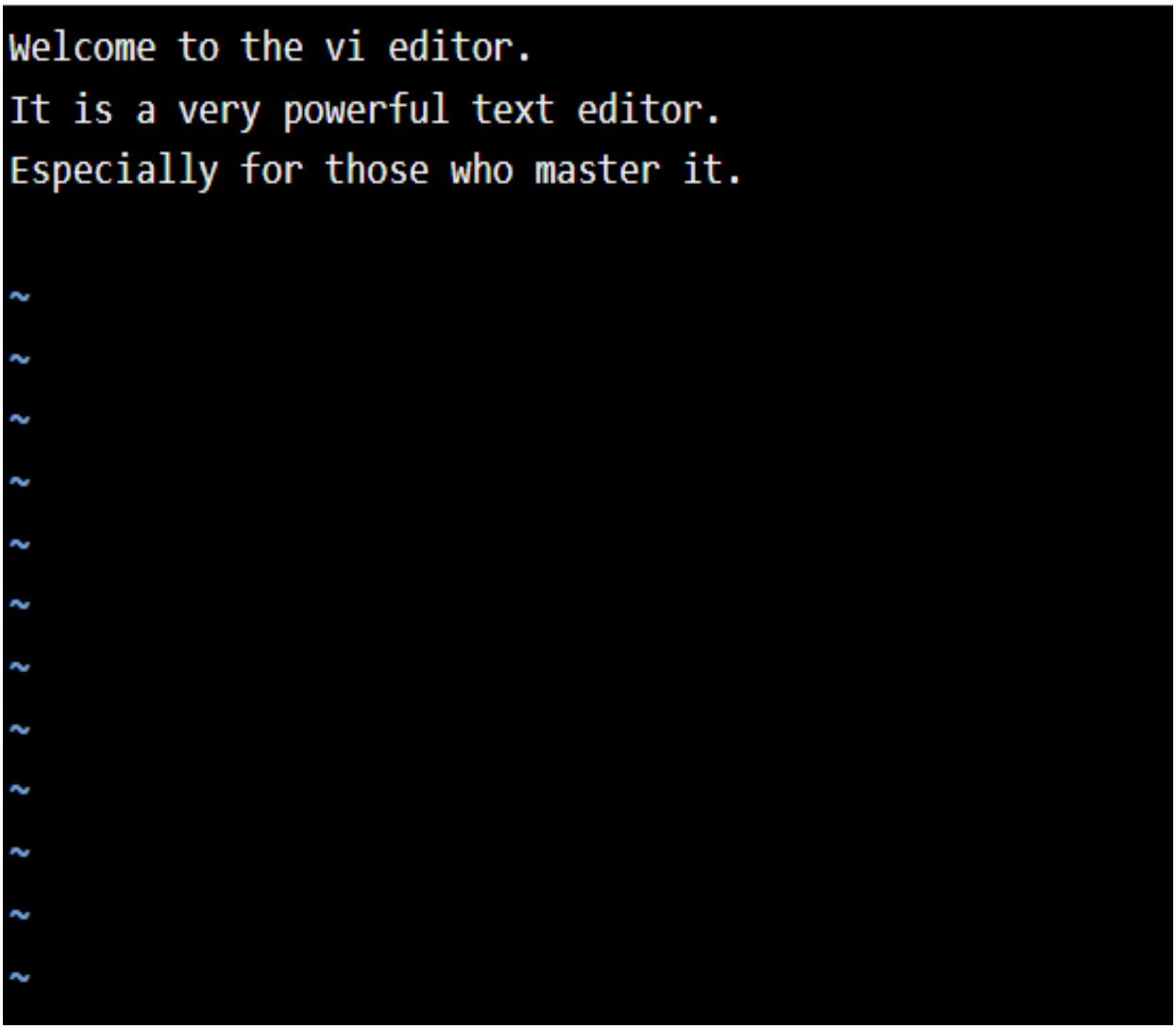


# 11.2.24 Step 24

Undo the last operation:

```
u
```

Your screen should look similar to the following:

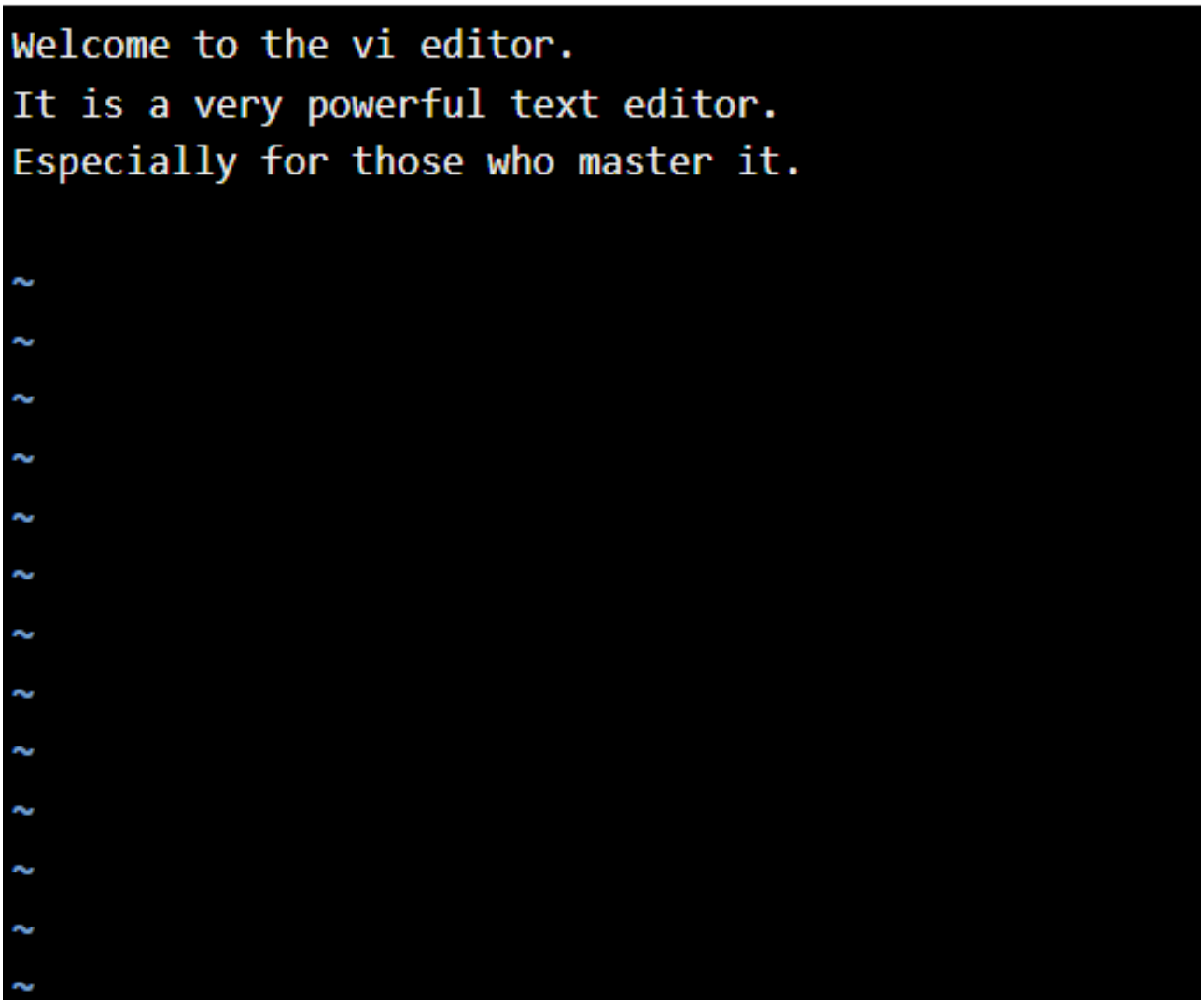


# 11.2.25 Step 25

Copy (or “yank”) the current word:

```
yw
```

When you copy text, no change will take place on the screen.

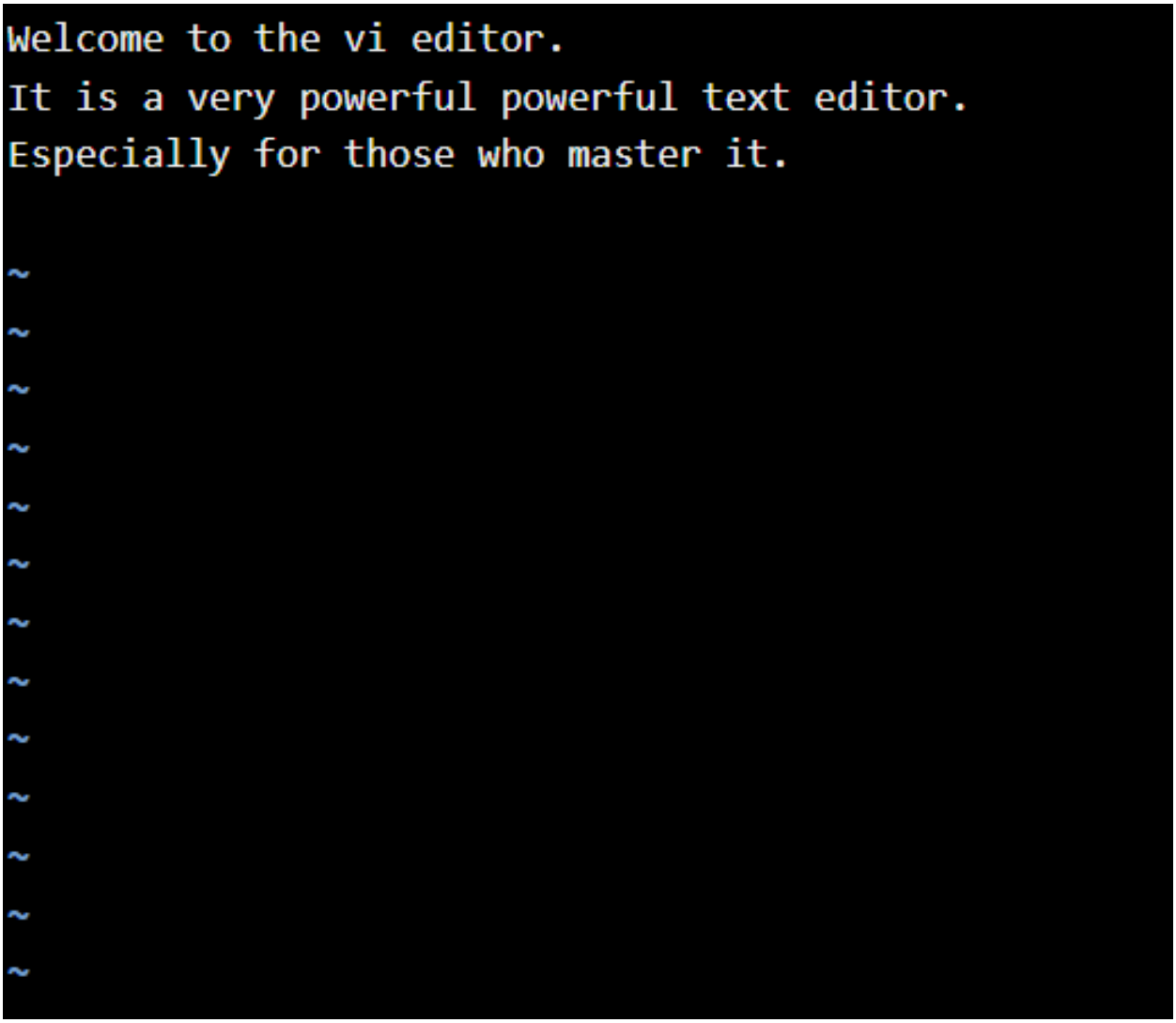


# 11.2.26 Step 26

Paste (or “put”) the copied word before the current cursor by typing **Shift+p**:

```
P
```

Your screen should look similar to the following:

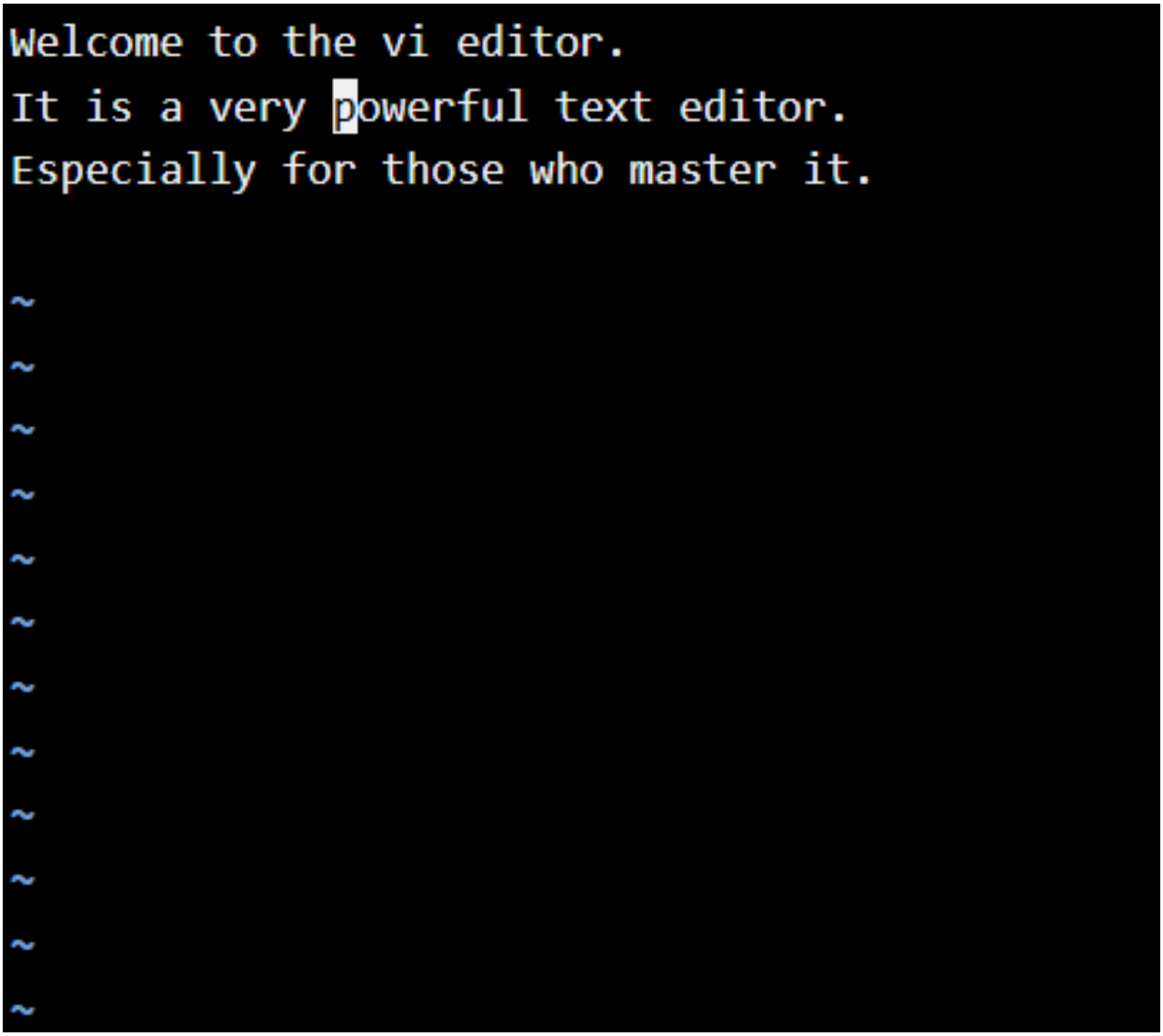


# 11.2.27 Step 27

Undo the last operation:

```
u
```

Your screen should look similar to the following:

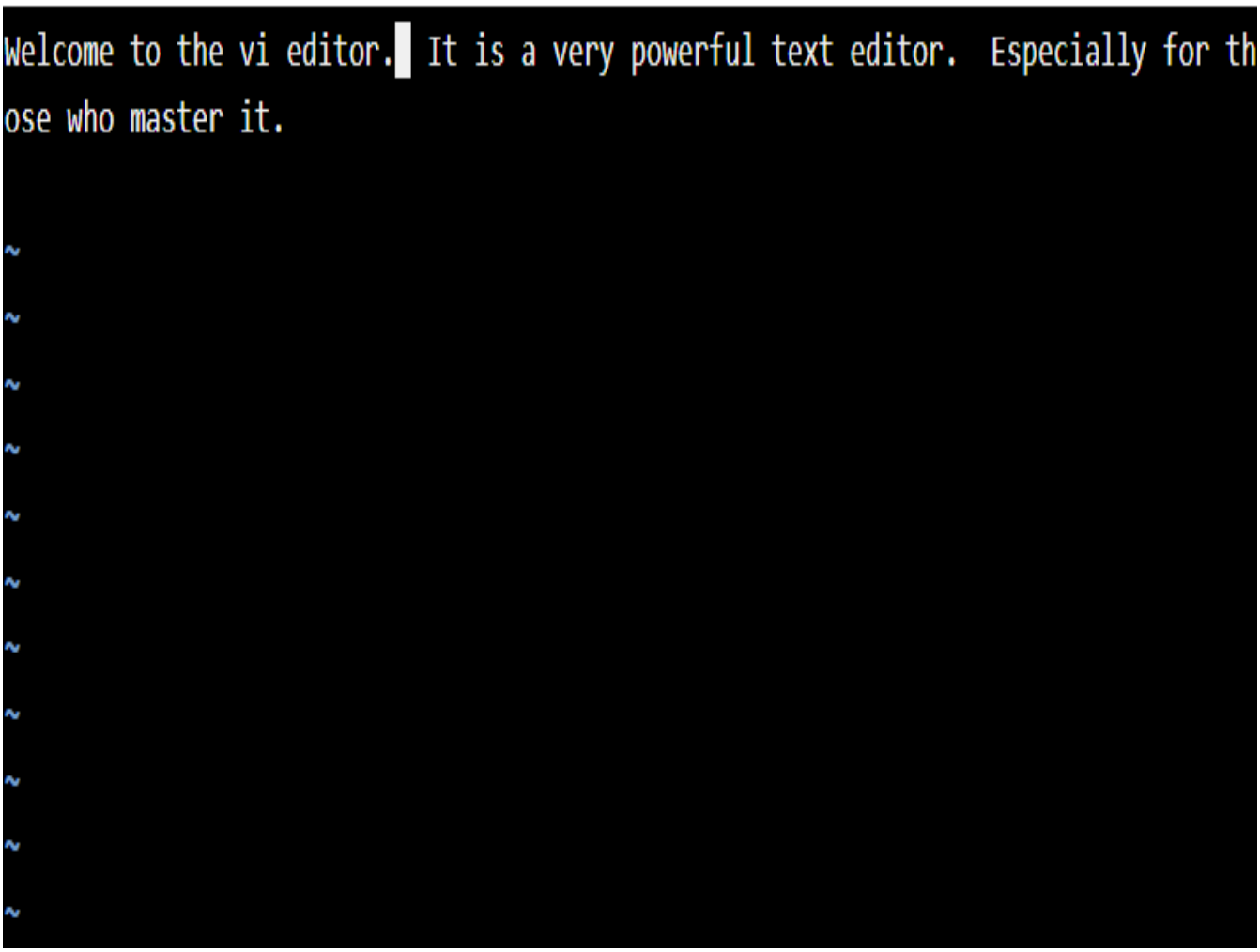


# 11.2.28 Step 28

Move to the first line, then join three lines:

```
1G
3J
```

Your screen should look similar to the following:

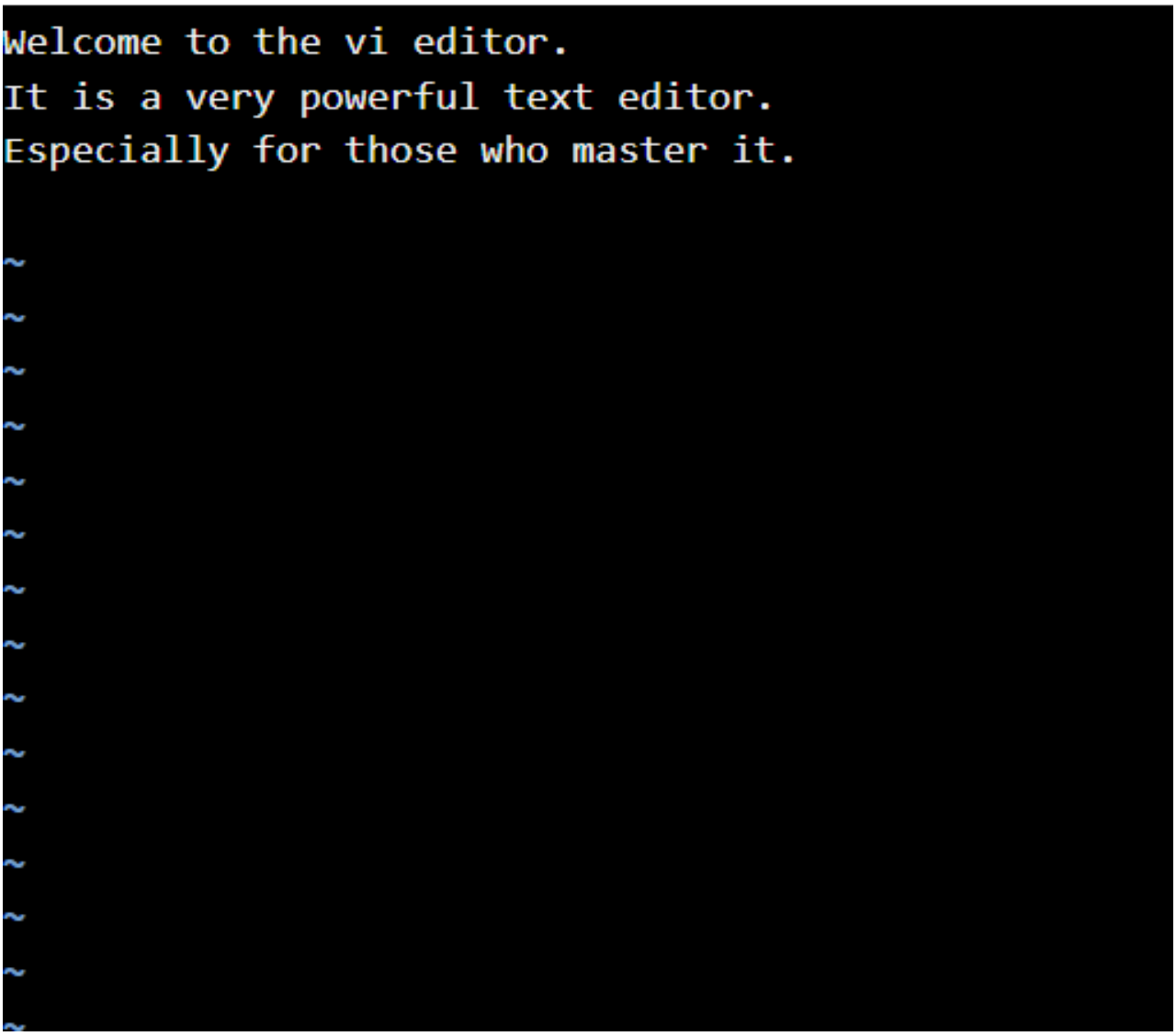


# 11.2.29 Step 29

Undo the last operation:

```
u
```

Your screen should look similar to the following:

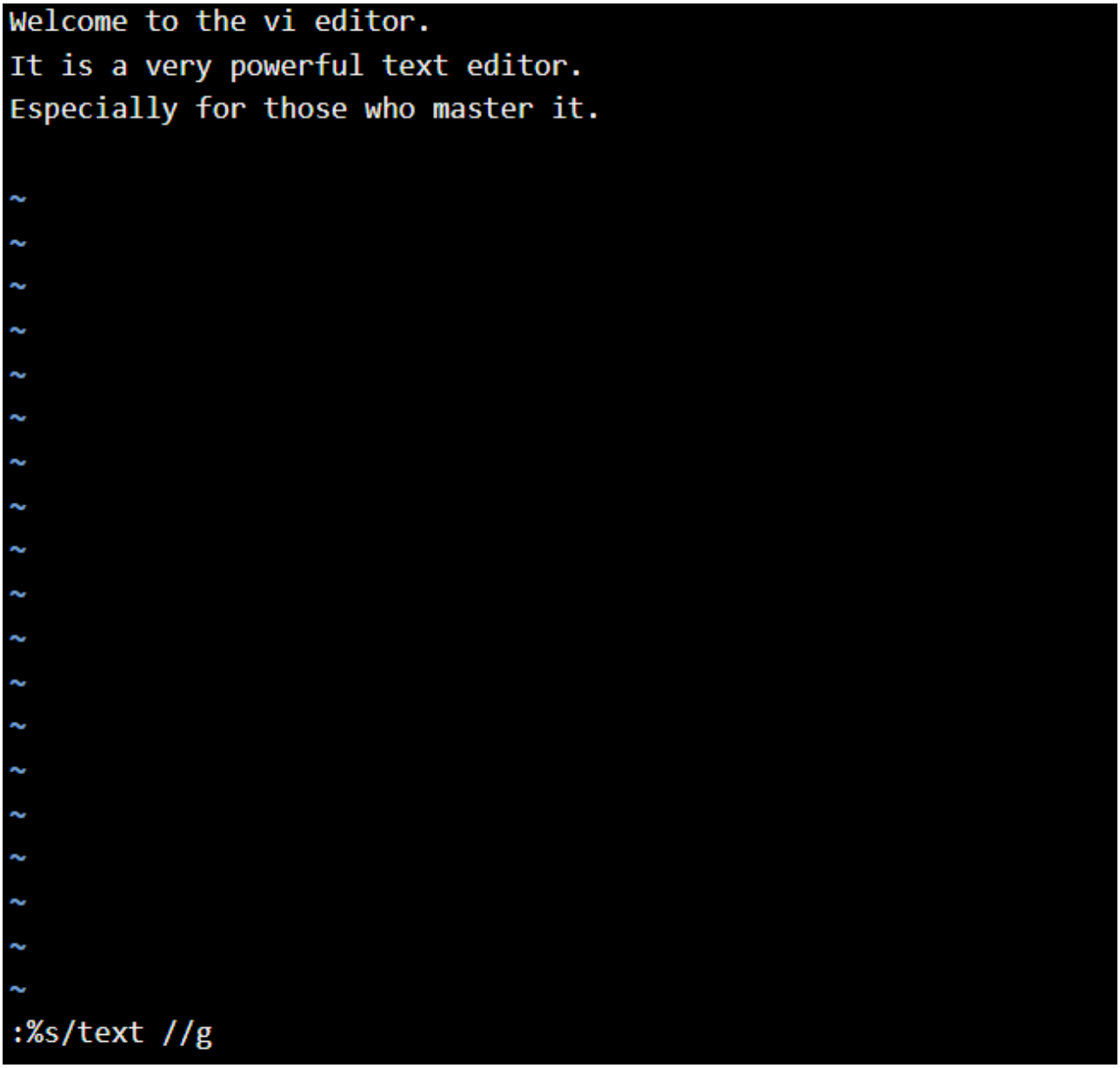


# 11.2.30 Step 30

Search for and delete the word `text` (add a space after the word `text`):

```
:%s/text //g
```

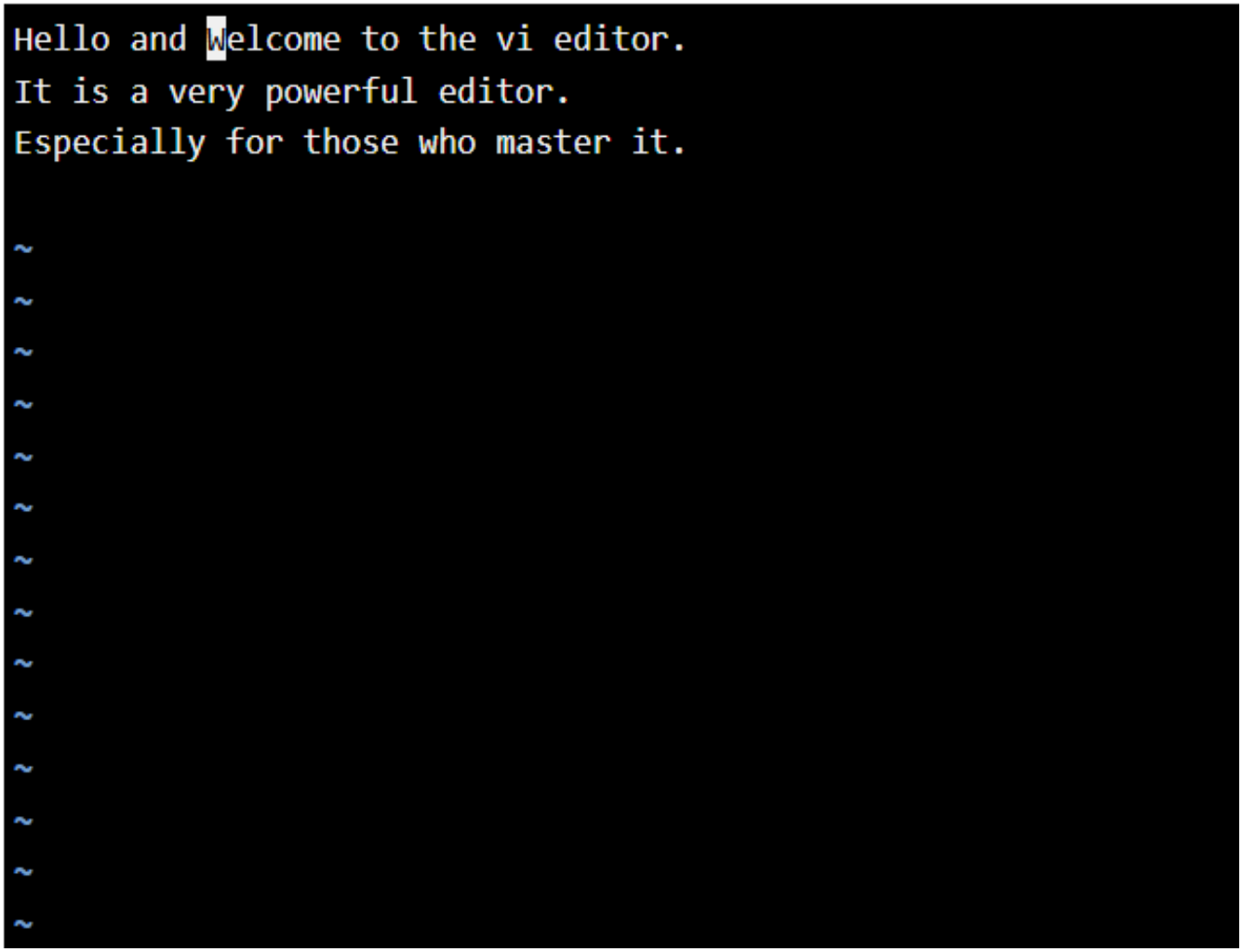
Your screen should look similar to the following:



# 11.2.31 Step 31

Navigate to beginning of file, then press `i` to enter insert mode to add text:

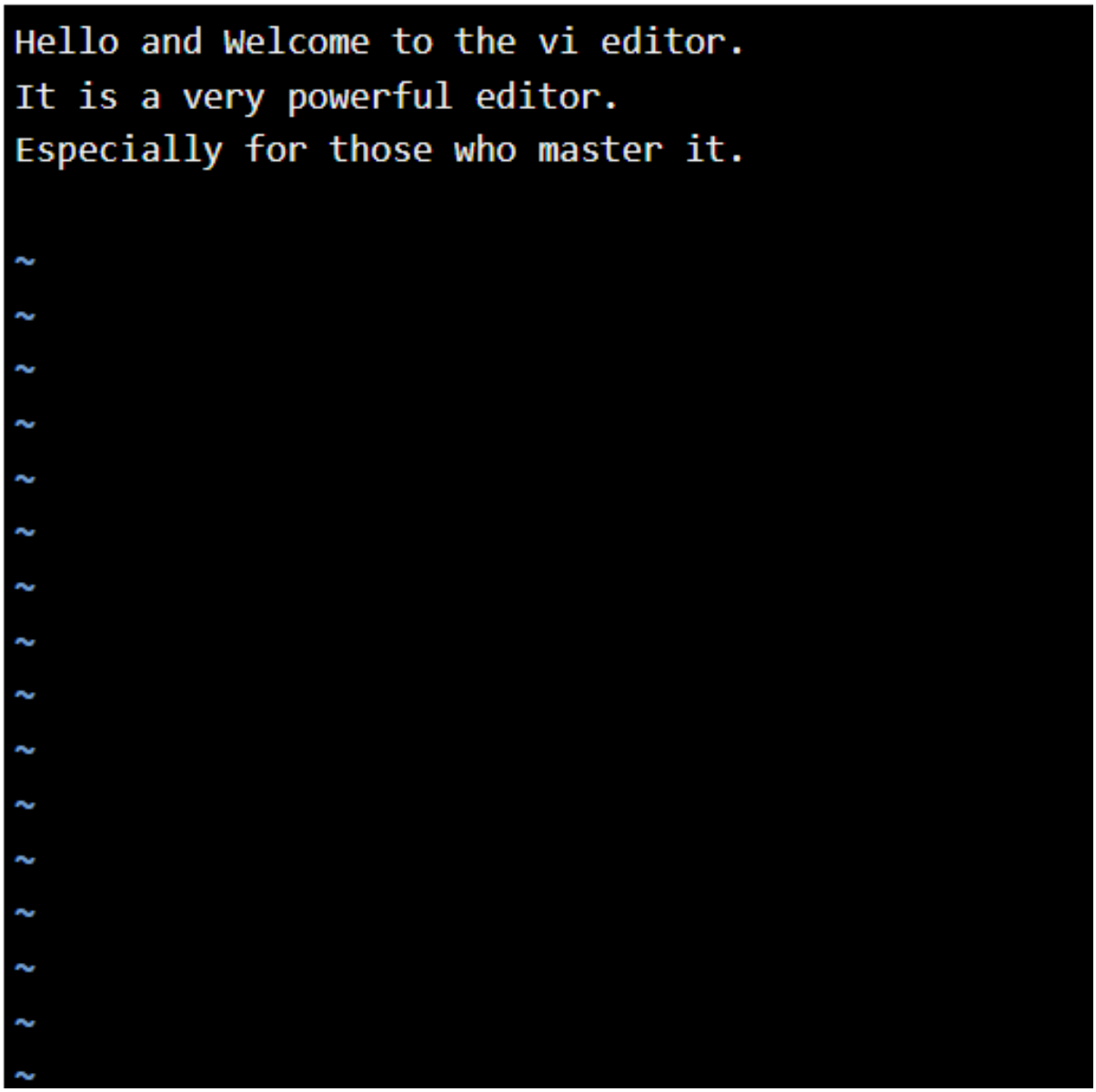
| Keys | Function |
| --- | --- |
| `1G` | Go to beginning of file (**Shift+G**) |
| `i` | Enter insert mode |
| `Hello and` | Add text to document with a space after “`and`” |



# 11.2.32 Step 32

Exit insert mode and return to command mode by pressing the **Escape** key:

```
ESC
```

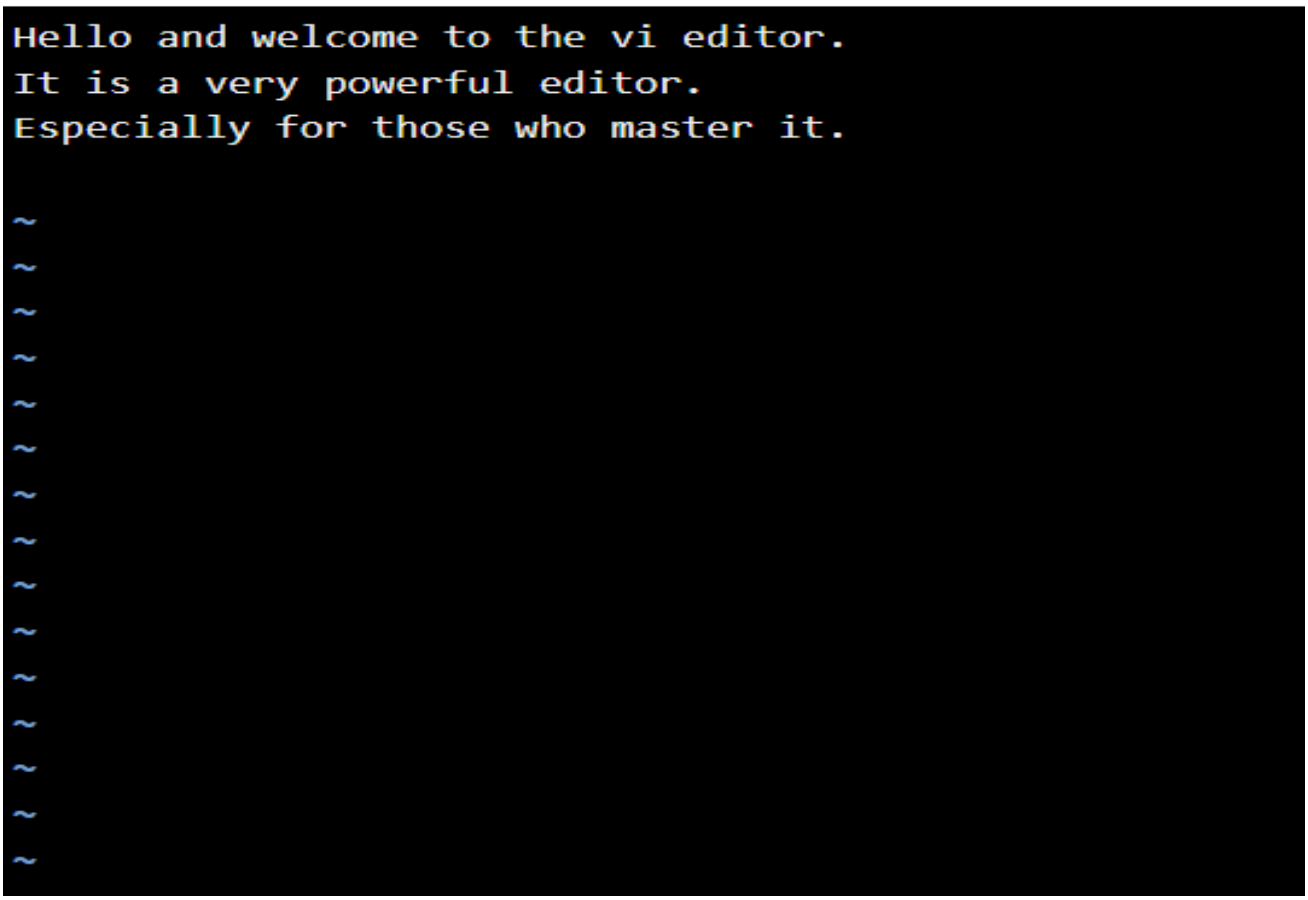


# 11.2.33 Step 33

Move forward one space by pressing the lower case `l` to place the cursor on the `W` and toggle it to lower case by pressing the tilde (`~`):

| Keys | Function |
| --- | --- |
| `l` | Lowercase ‘L’ moves forward one space |
| `~` | **Shift+`** changes letter to lower case |

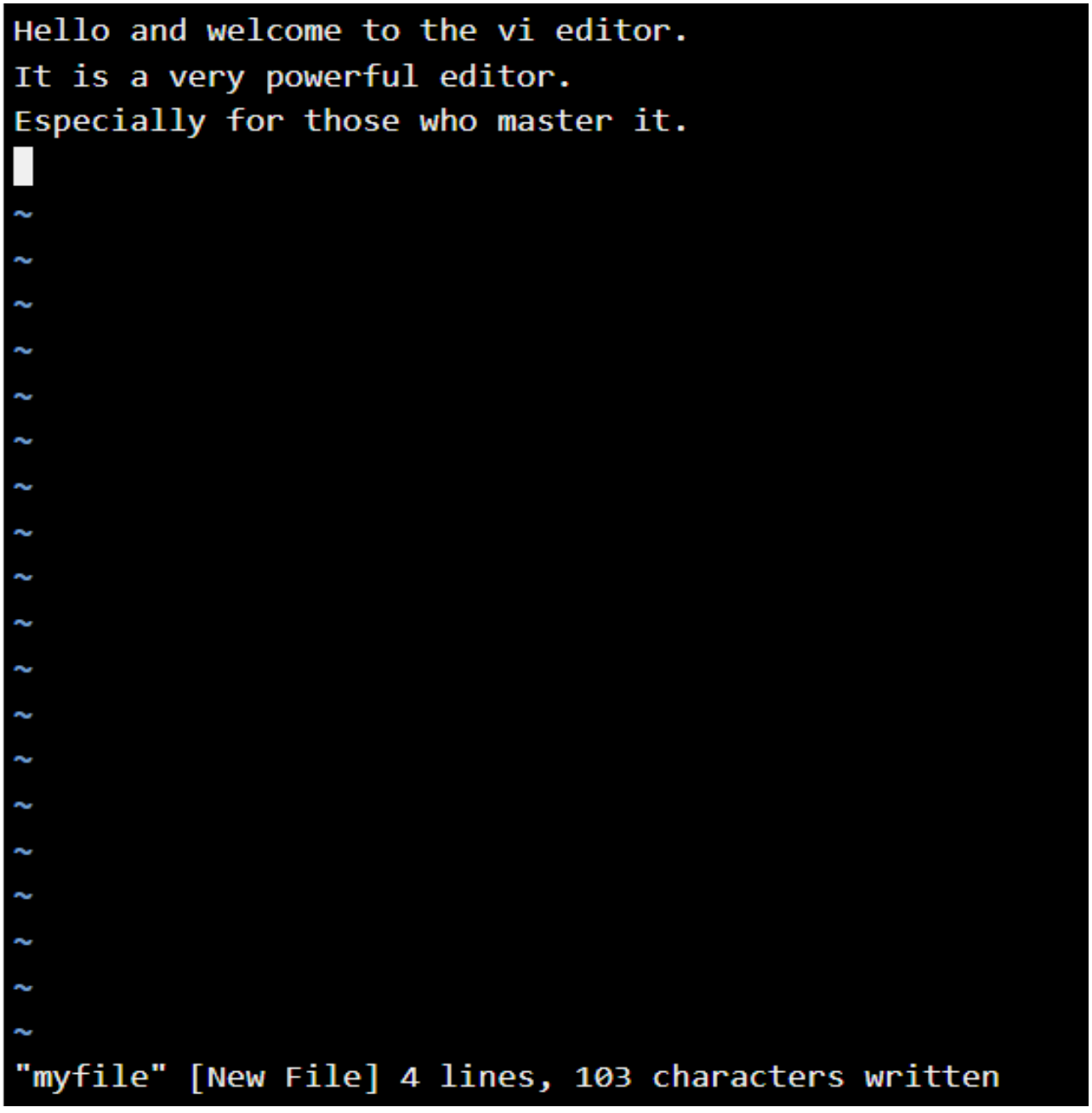
Your screen should look similar to the following:



# 11.2.34 Step 34

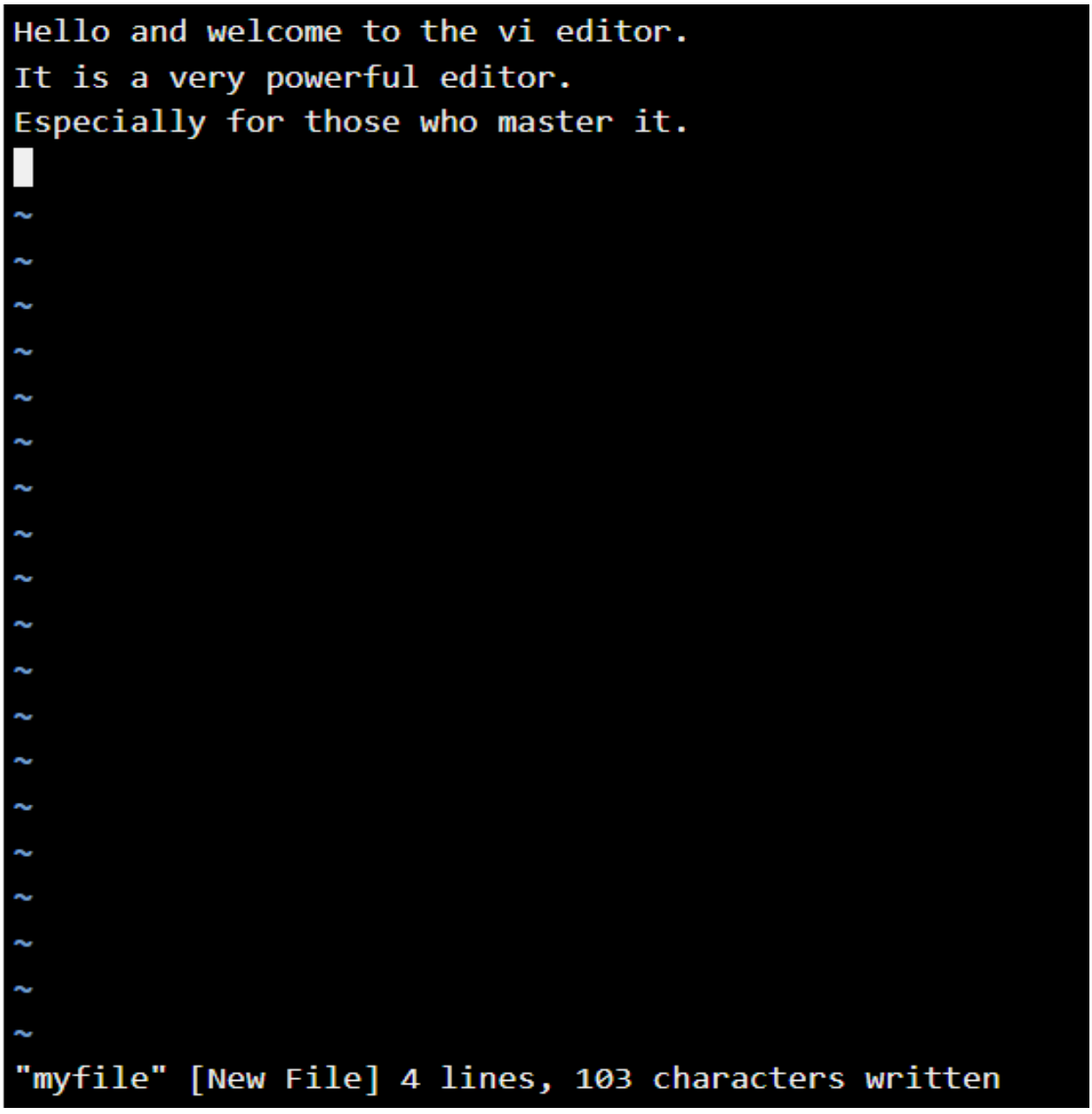
Save the file. Press the **Esc** key to ensure you are in command mode. Then type `:w` and the **Enter** key:

```
:w
```



# 11.2.35 Step 35

When you press **Enter** to commit the change, note the message in lower left indicating the file has been written:

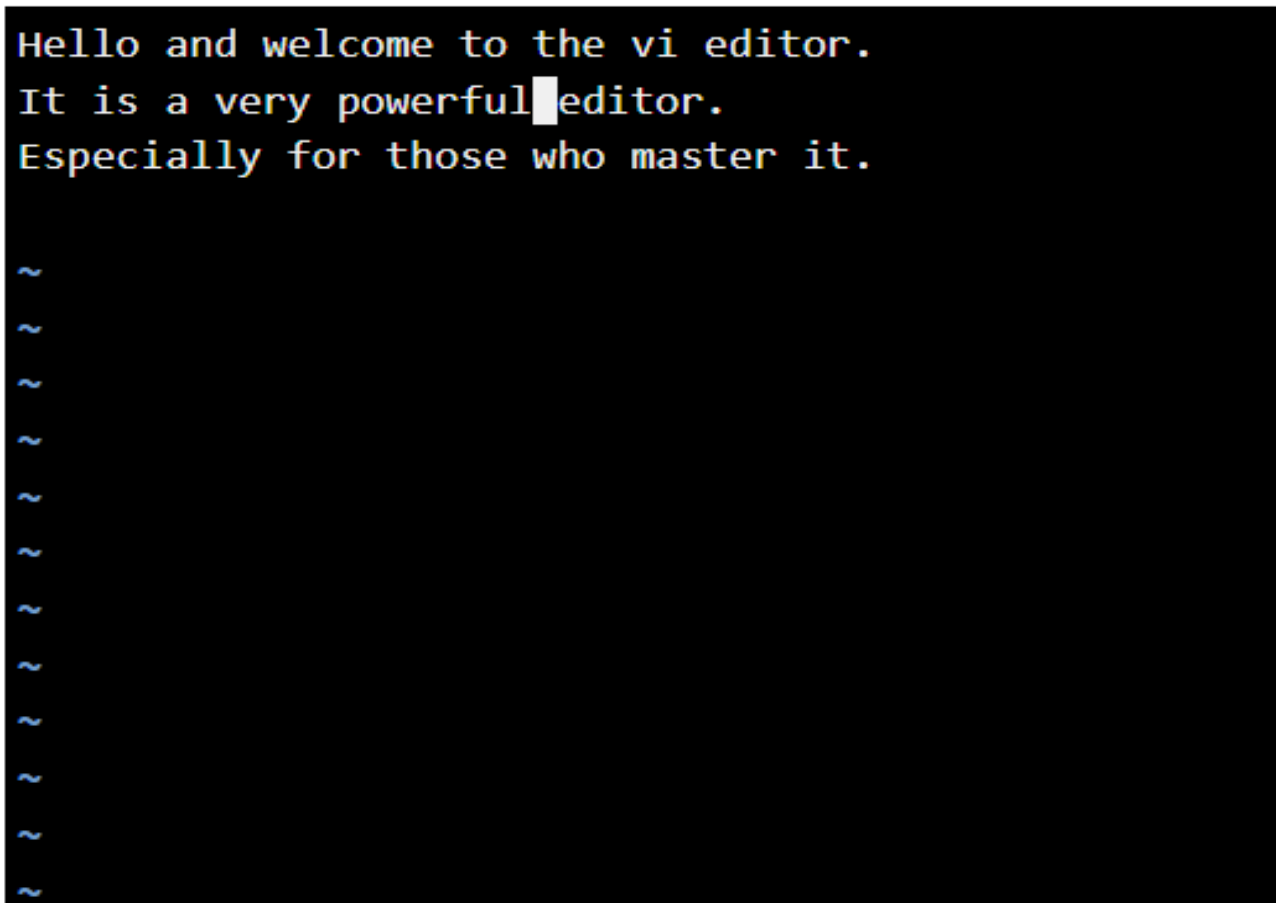


# 11.2.36 Step 36

Navigate to the space between the word "powerful" and "editor" in the second line as shown in the image below. You could press j followed by 10l or use the arrow keys:

| Command | Function/Keys |
| --- | --- |
| j | Move down to second line |
| 10l | 10 followed by the lowercase ‘L’ |

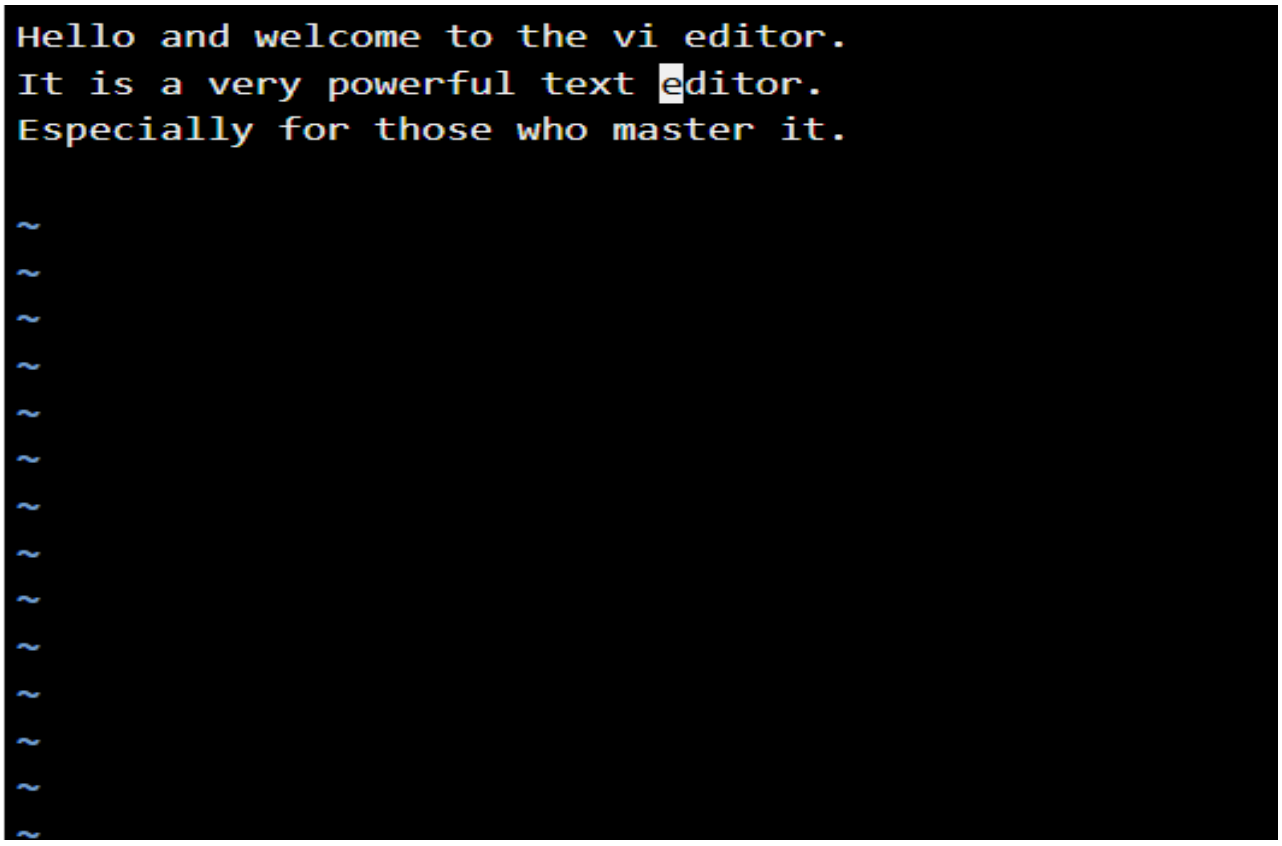
Your screen should look similar to the following:



# 11.2.37 Step 37

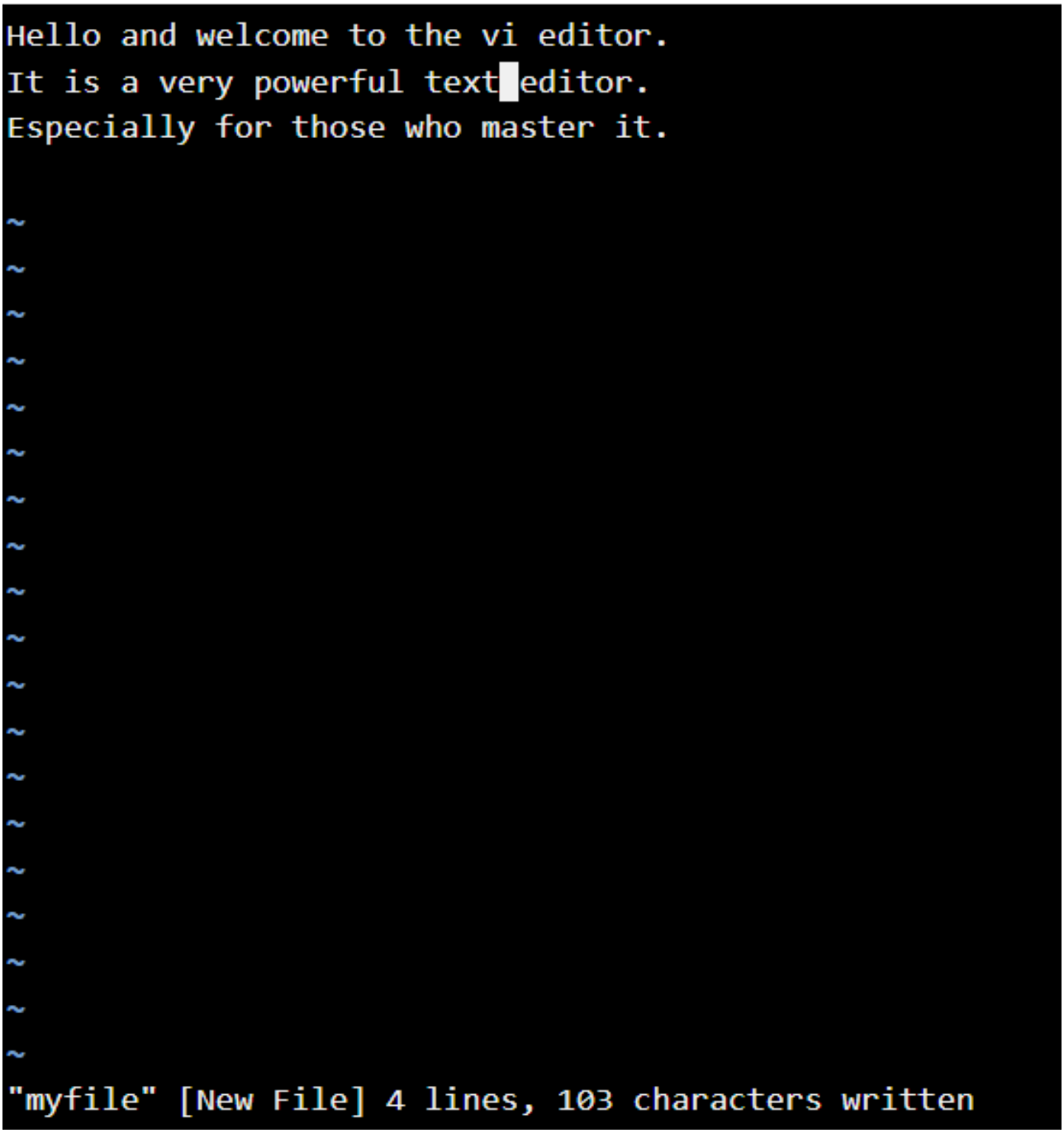
Append text to the right of the cursor by pressing the letter a. This moves the cursor to the right and enters insert mode. Type the word text followed by a space as shown in image below:

| Command | Function/Keys |
| --- | --- |
| a | Enter insert mode. |
| text | text followed by a space |



# 11.2.38 Step 38

Exit insert mode by pressing the **Esc** key.

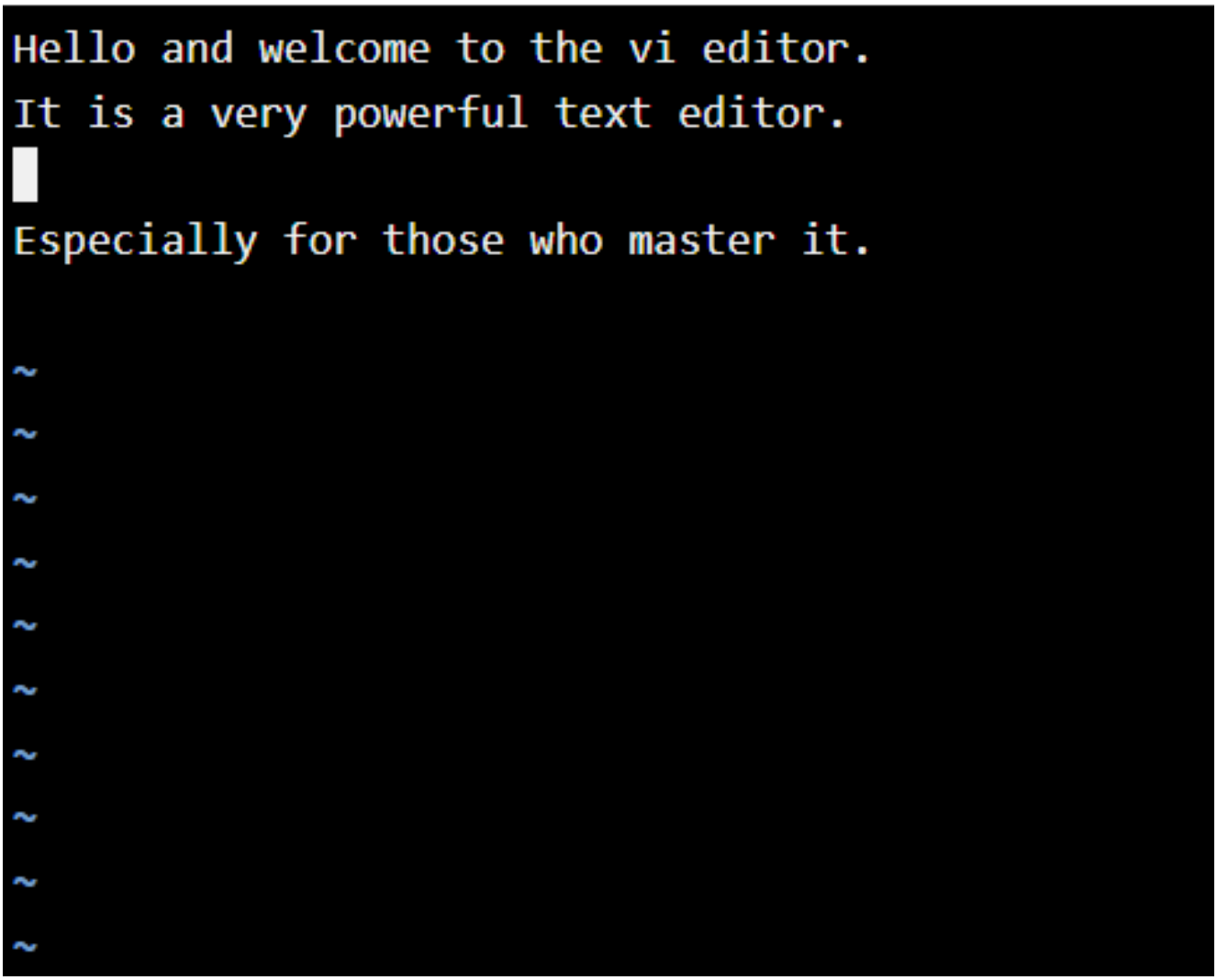


# 11.2.39 Step 39

Open a blank line below the current line by typing a lowercase letter ○:

```
○
```

Your screen should look similar to the following:

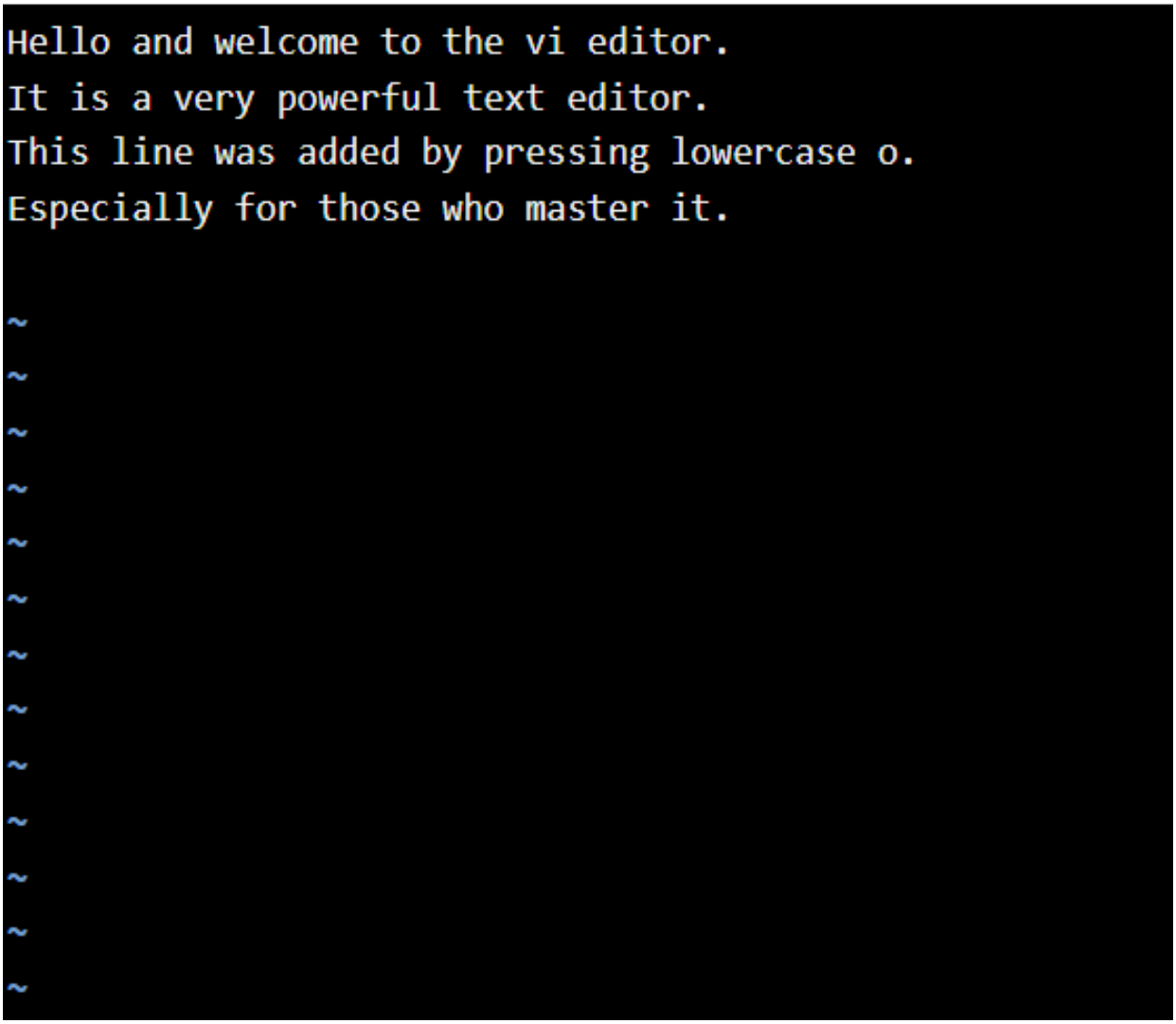


# 11.2.40 Step 40

Enter the following text:

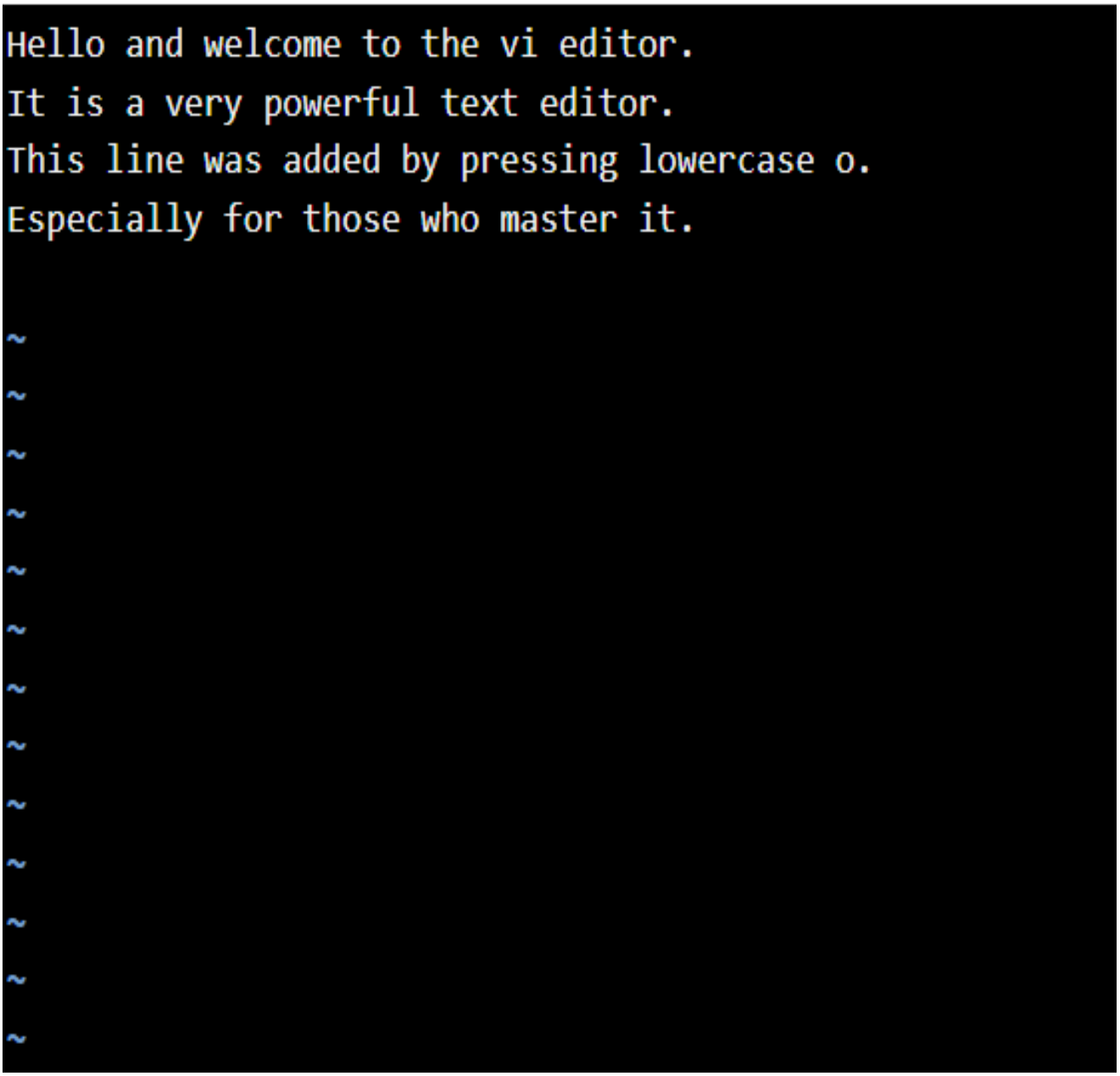
```
This line was added by pressing lowercase o.
```

Your screen should look similar to the following:



# 11.2.41 Step 41

Exit insert mode by pressing the **Esc** key.

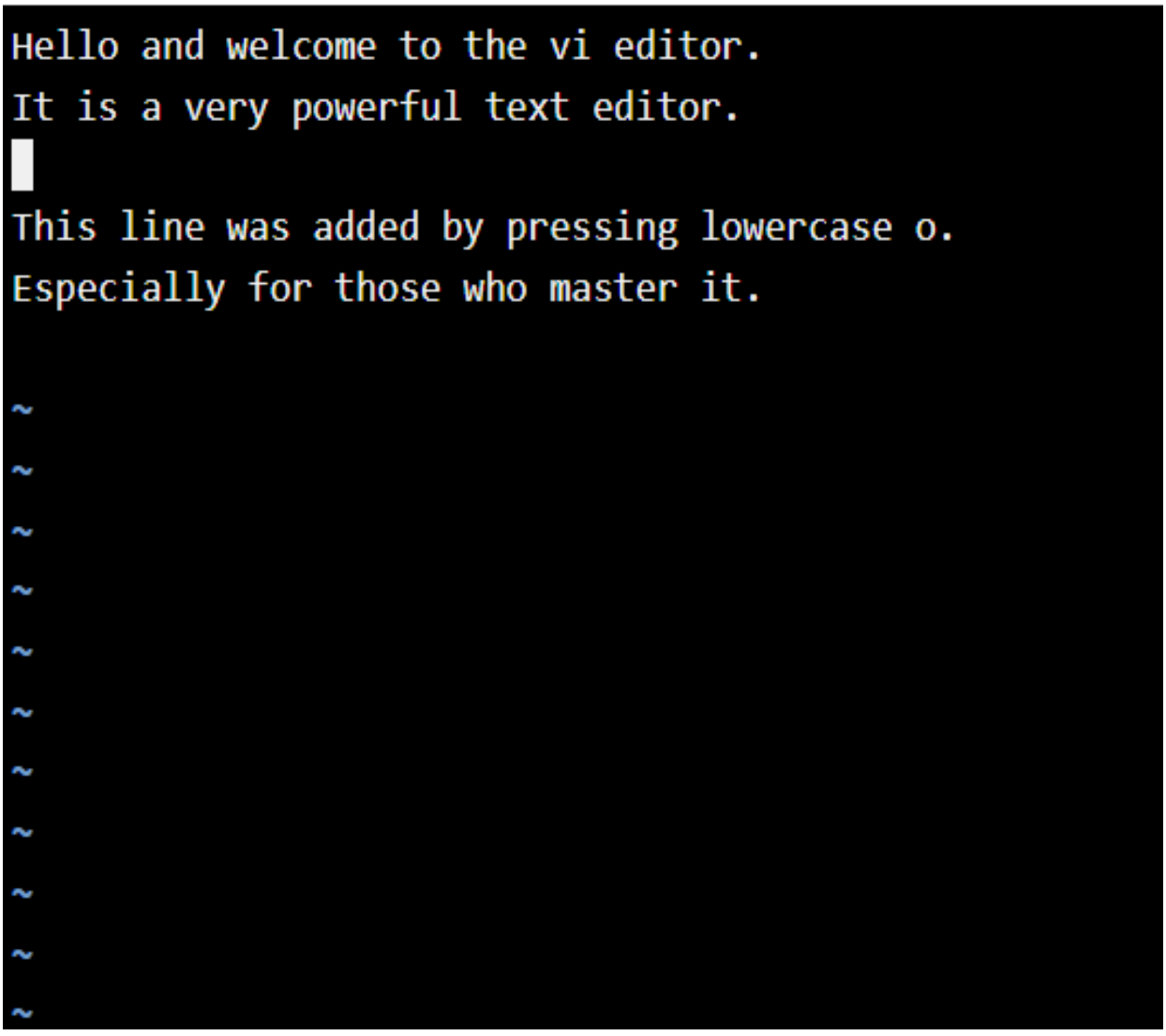


# 11.2.42 Step 42

Open a blank line above the current line by pressing uppercase `O`:

```
O
```

Your screen should look similar to the following:

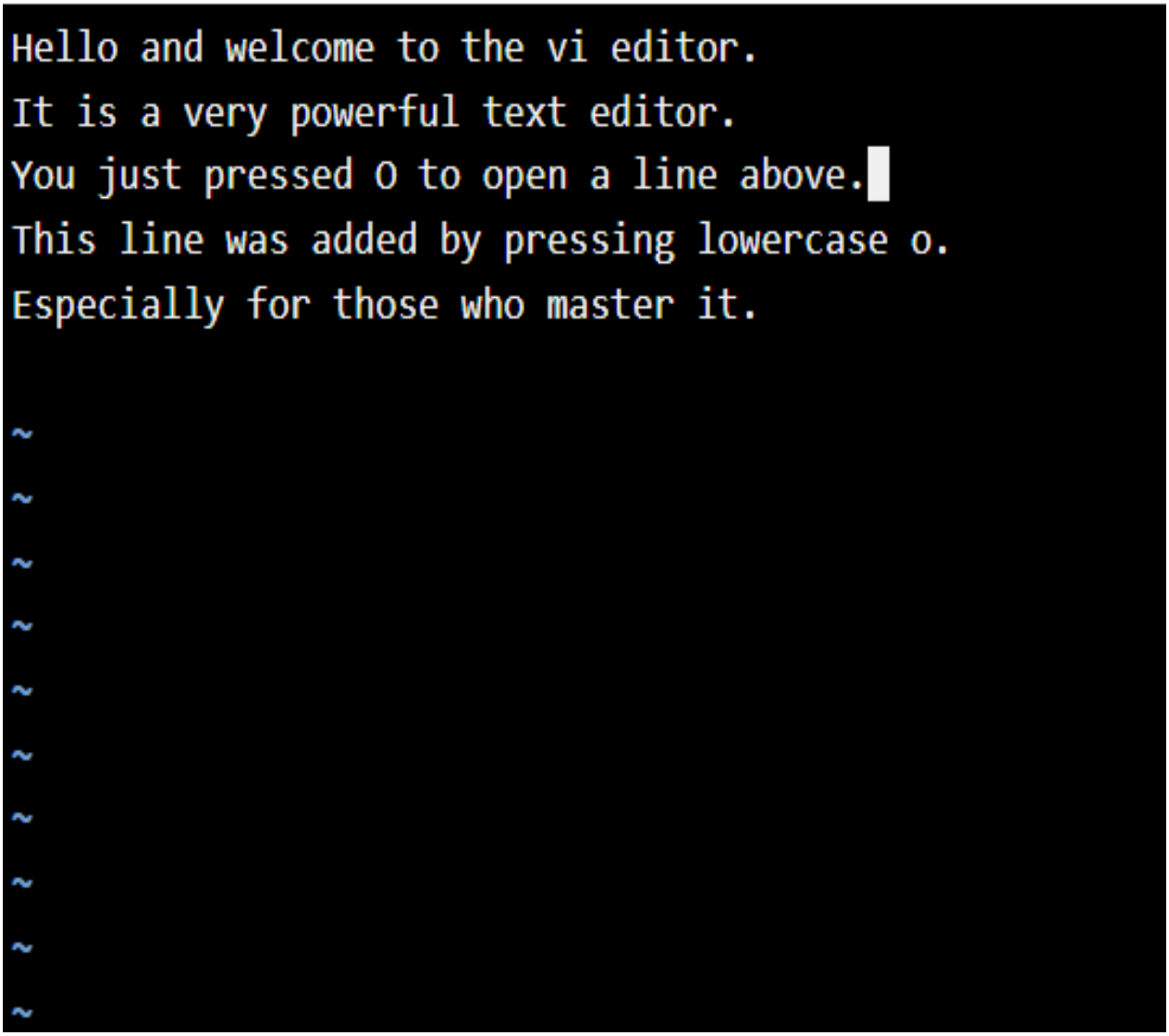


# 11.2.43 Step 43

Enter the following text:

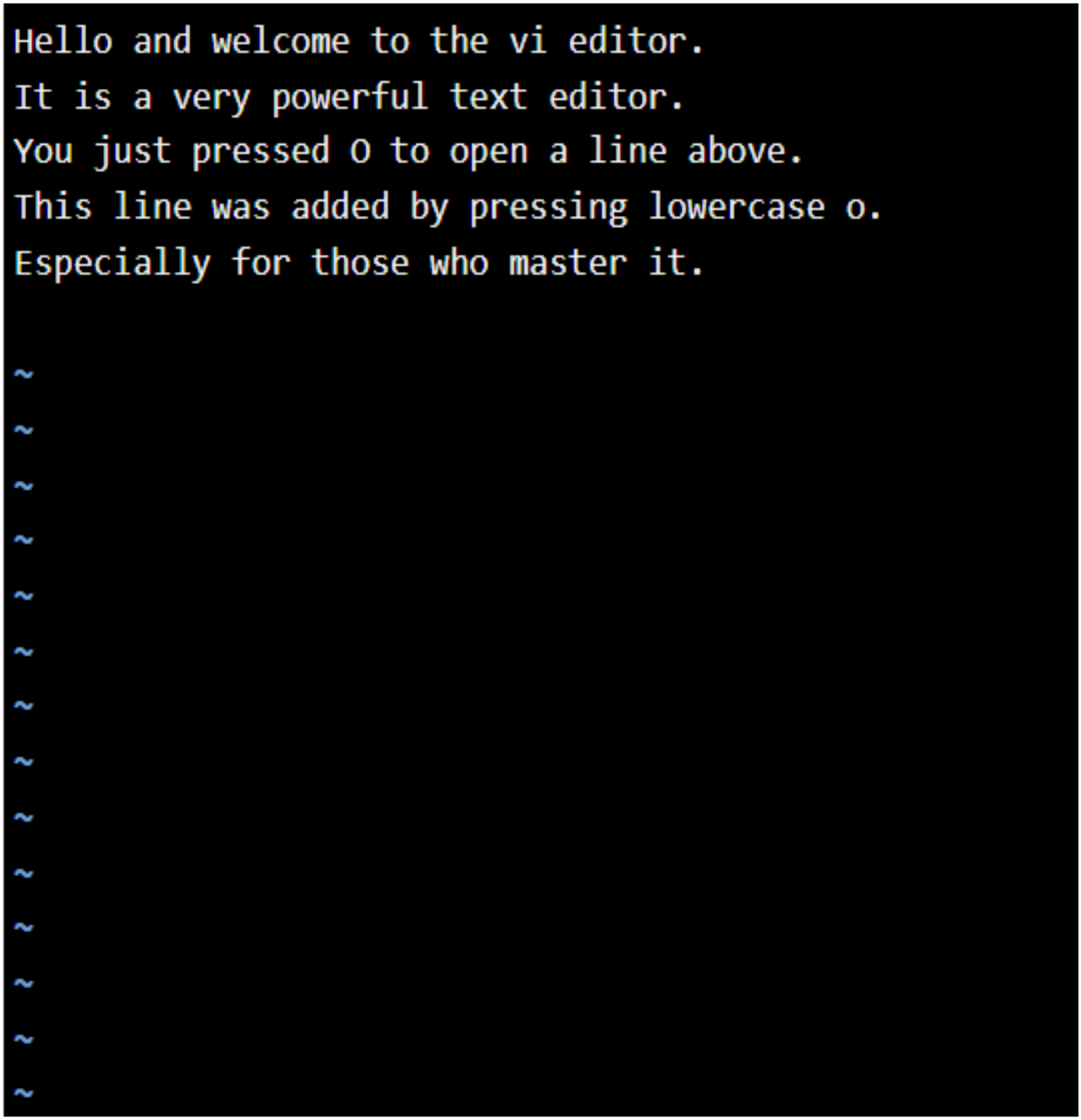
```
You just pressed O to open a line above.
```

Your screen should look similar to the following:



# 11.2.44 Step 44

Exit insert mode by pressing the **Esc** key.



# 11.2.45 Step 45

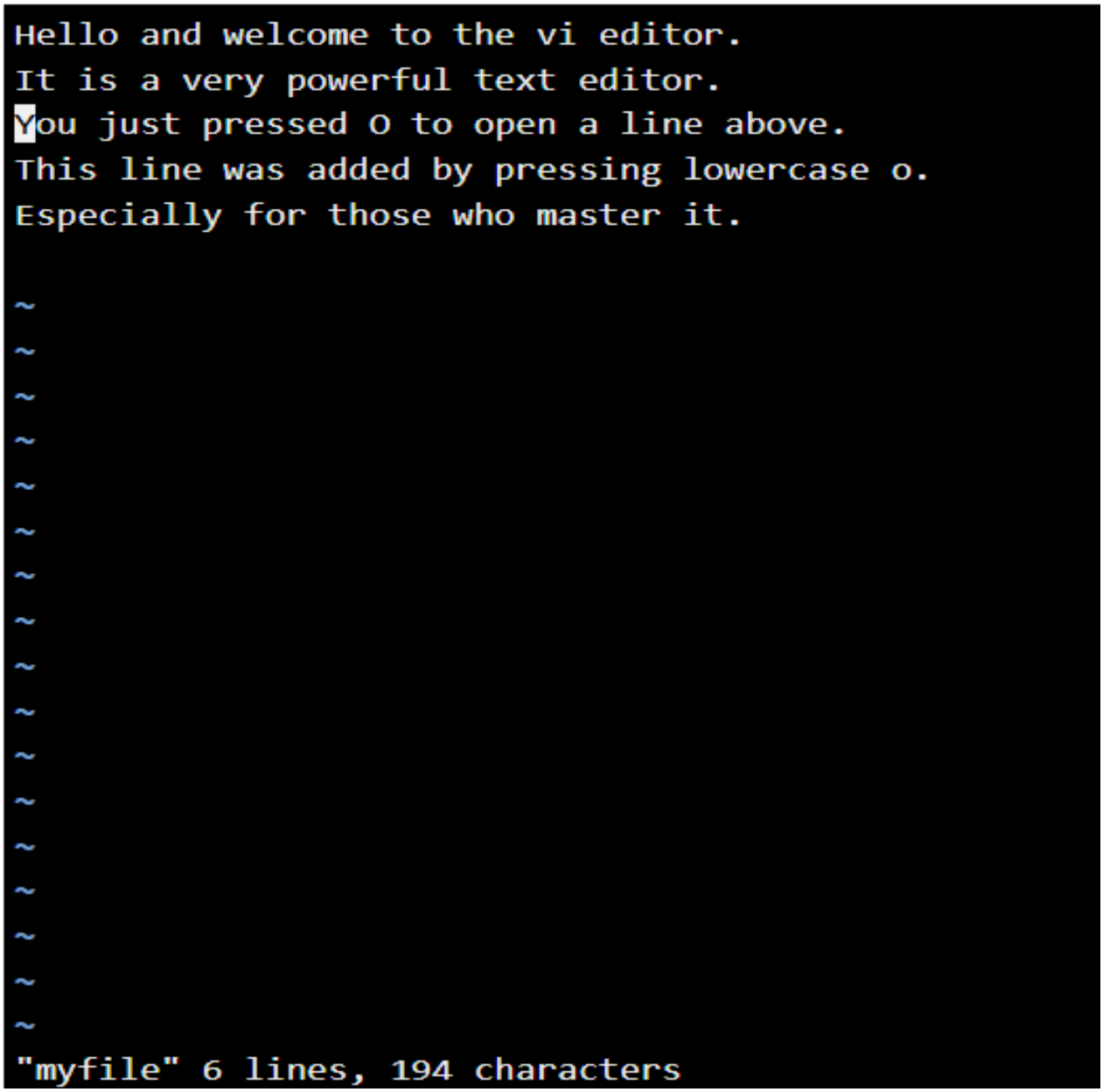
Save the file and close the `vi` editor using any one of the following methods that *saves* changes:

| Command | Function/Keys |
| --- | --- |
| `:x` | Will save and close the file. |
| `:wq` | Will write to file and quit. |
| `:wq!` | Will write to a read-only file, if possible, and quit. |
| `ZZ` | Will save and close. Notice that no colon `:` is used in this case. |
| `:q!` | Exit without saving changes |
| `:e!` | Discard changes and reload file |
| `:w!` | Write to read-only, if possible. |

# 11.2.46 Step 46

Once again open `myfile` using the `vi` editor:

```
vi myfile
```

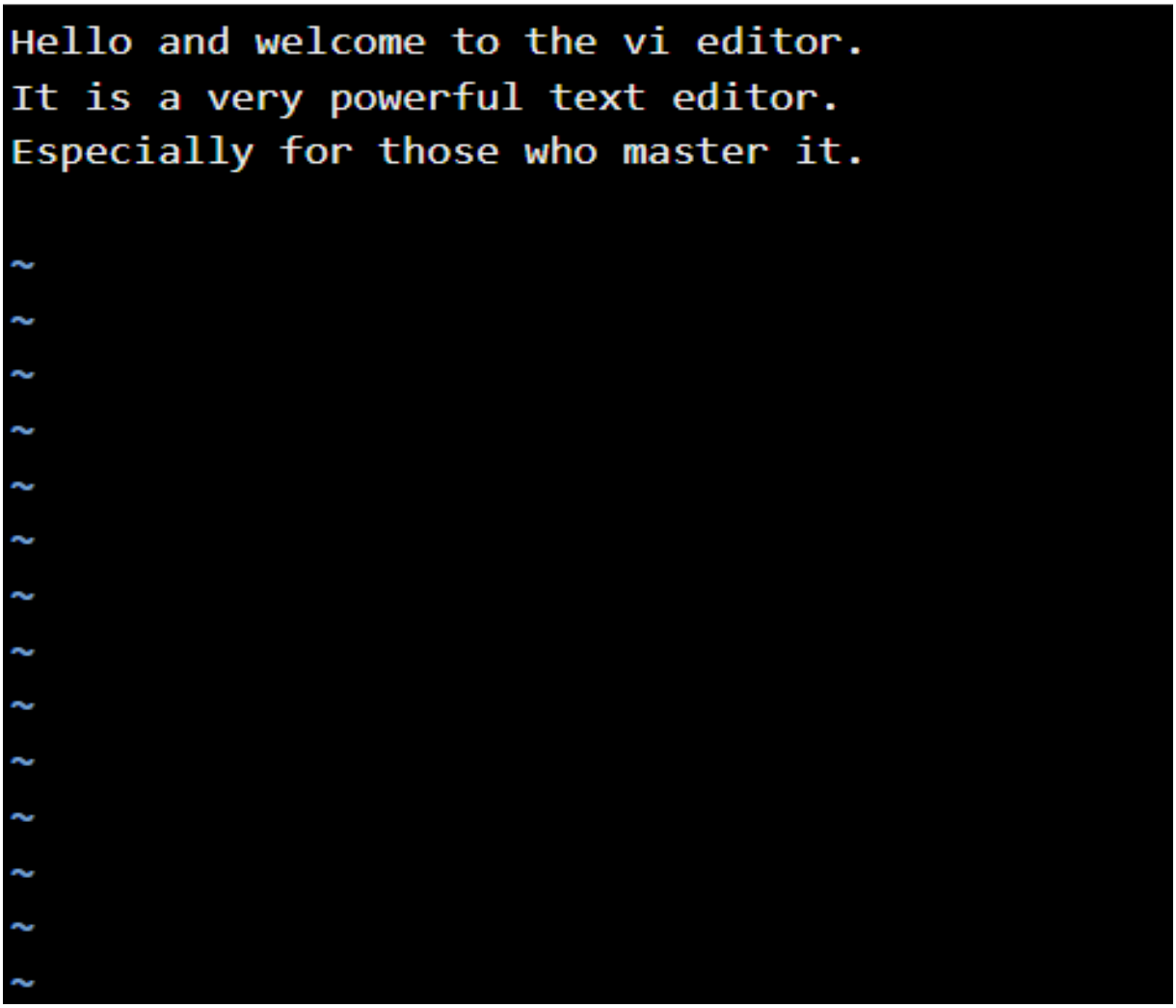


# 11.2.47 Step 47

Navigate to the third line, and delete the third and fourth lines:

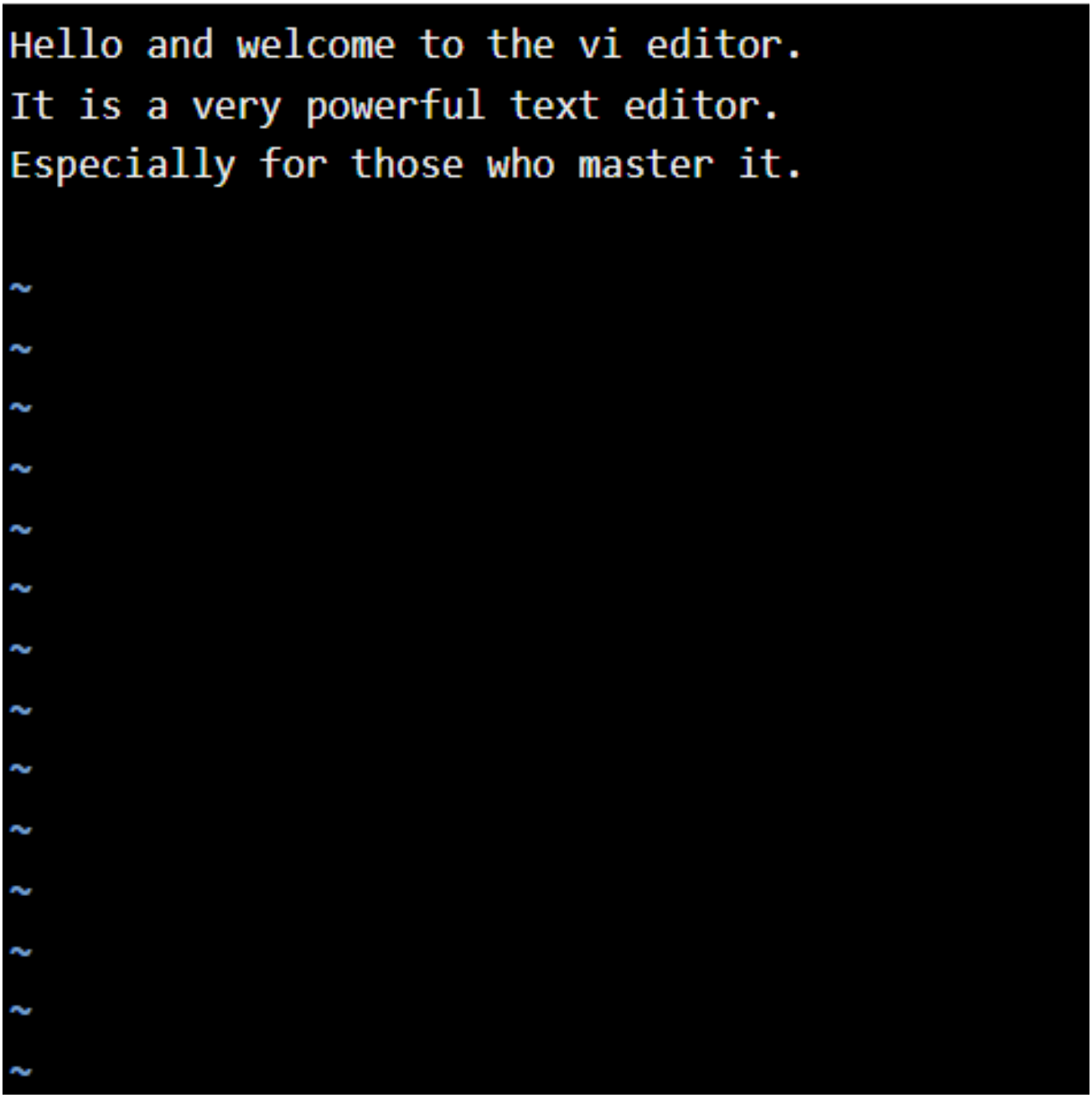
```
3G
2dd
```

Your screen should look similar to the following:



# 11.2.48 Step 48

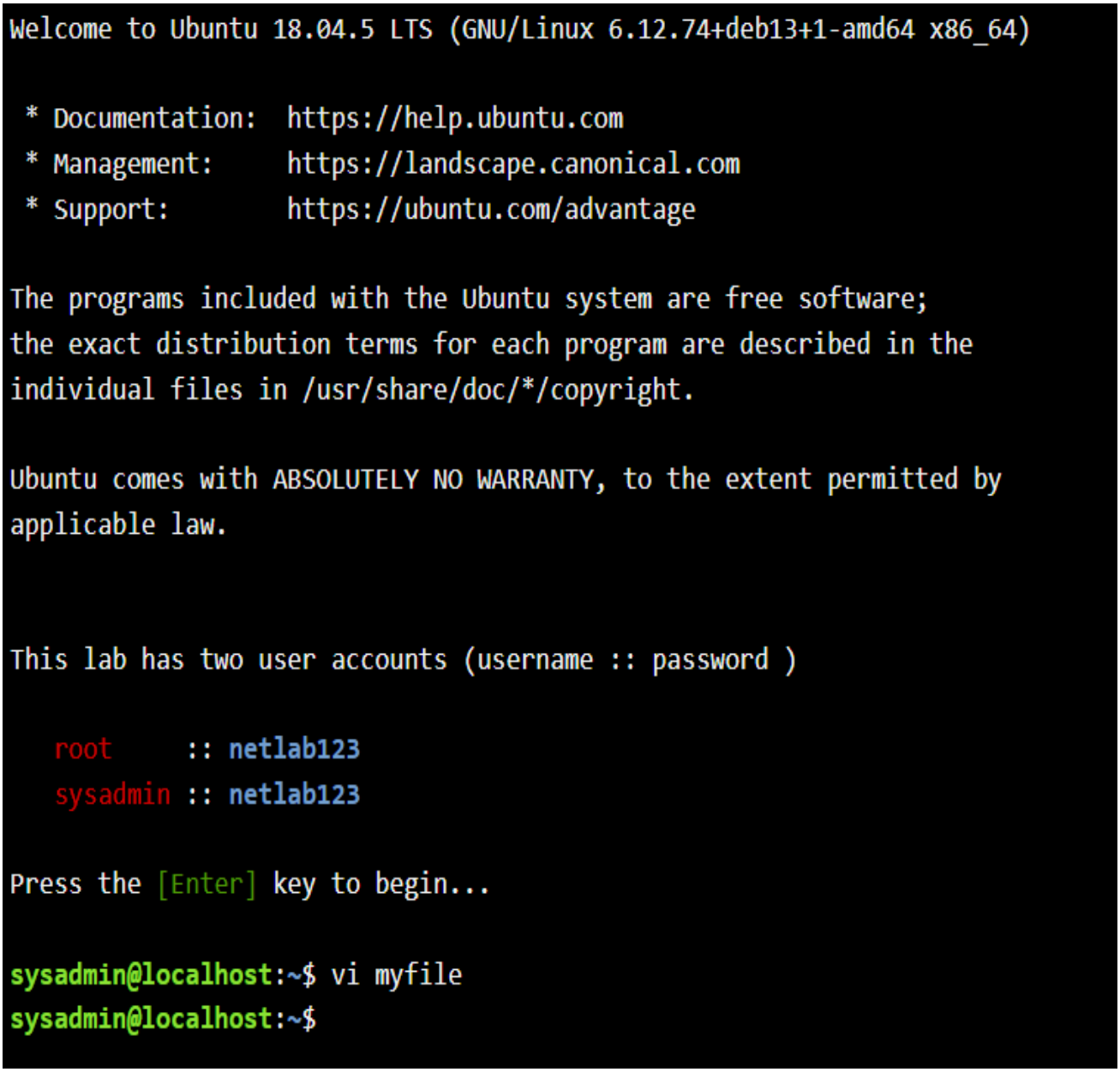
Press the **Esc** key to confirm you are in command mode.



# 11.2.49 Step 49

Quit the `vi` editor without saving your changes:

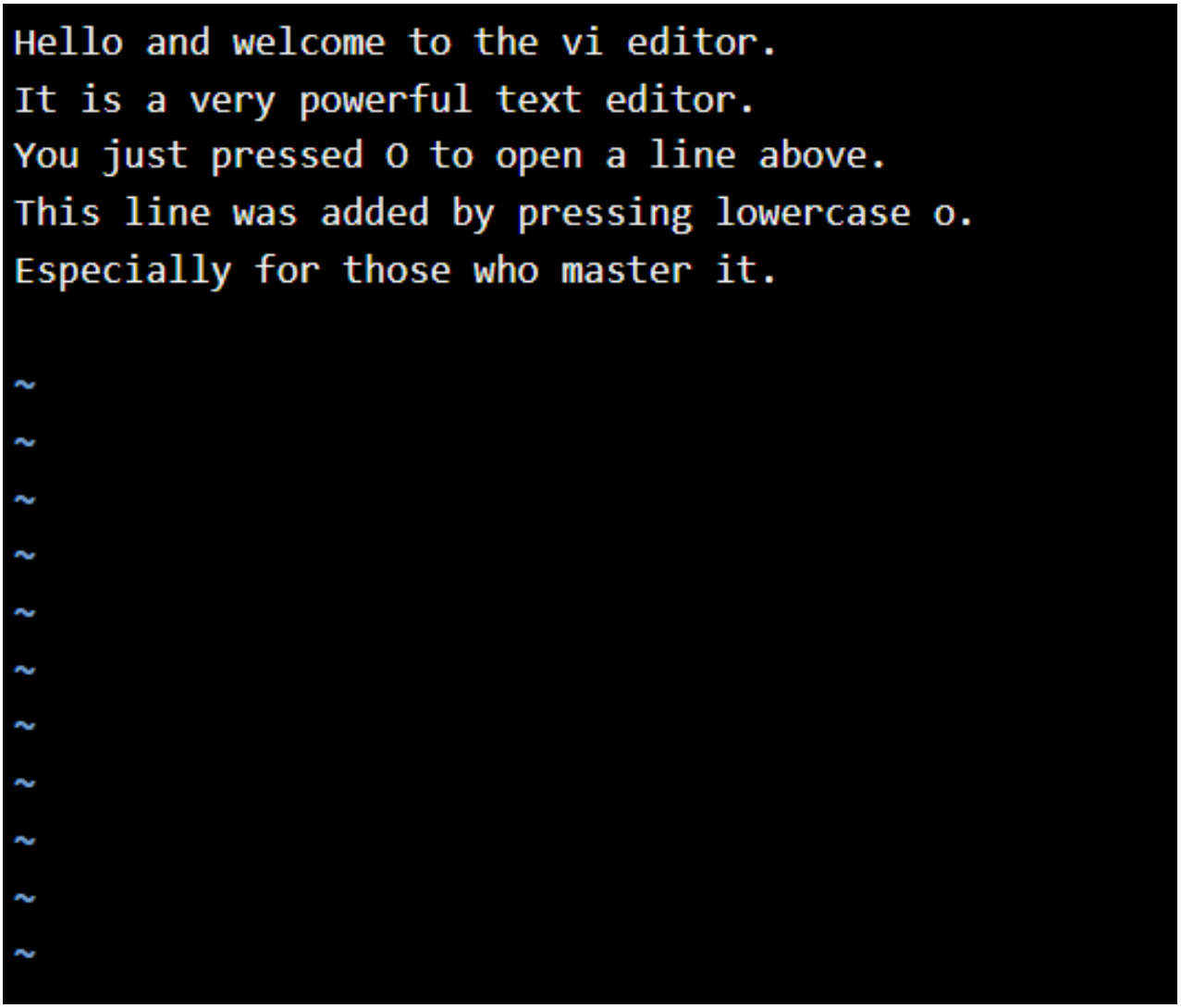
```
:q!
```



# 11.2.50 Step 50

Open `myfile` with the `vi` editor:

```
vi myfile
```



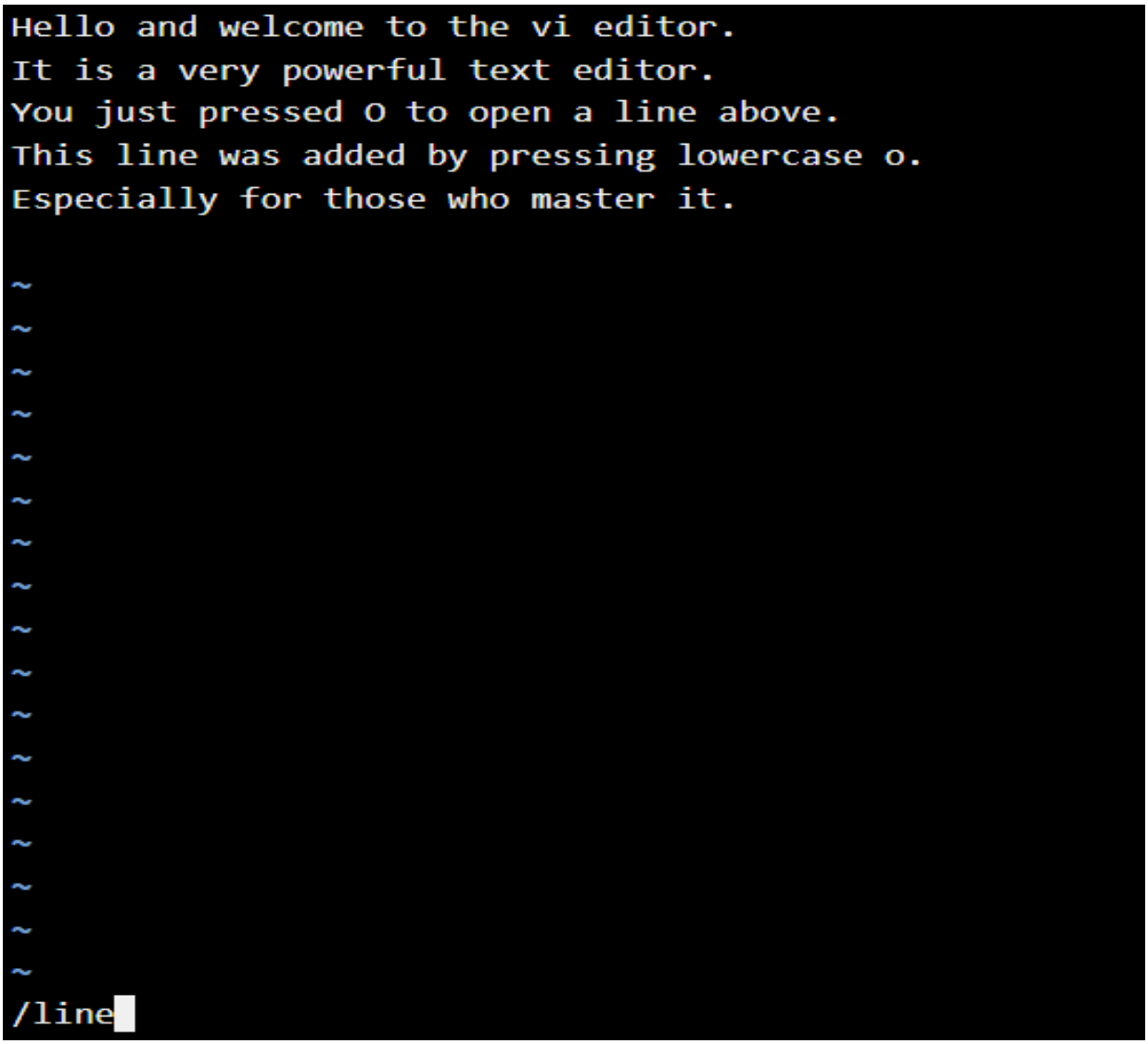

Notice that lines 3 and 4 are still present.

# 11.2.51 Step 51

Search forward for the word `line`. You'll notice the cursor moves to the beginning of the first instance of the word `line` as shown in image below:

```
/line
```

Your screen should look similar to the following:

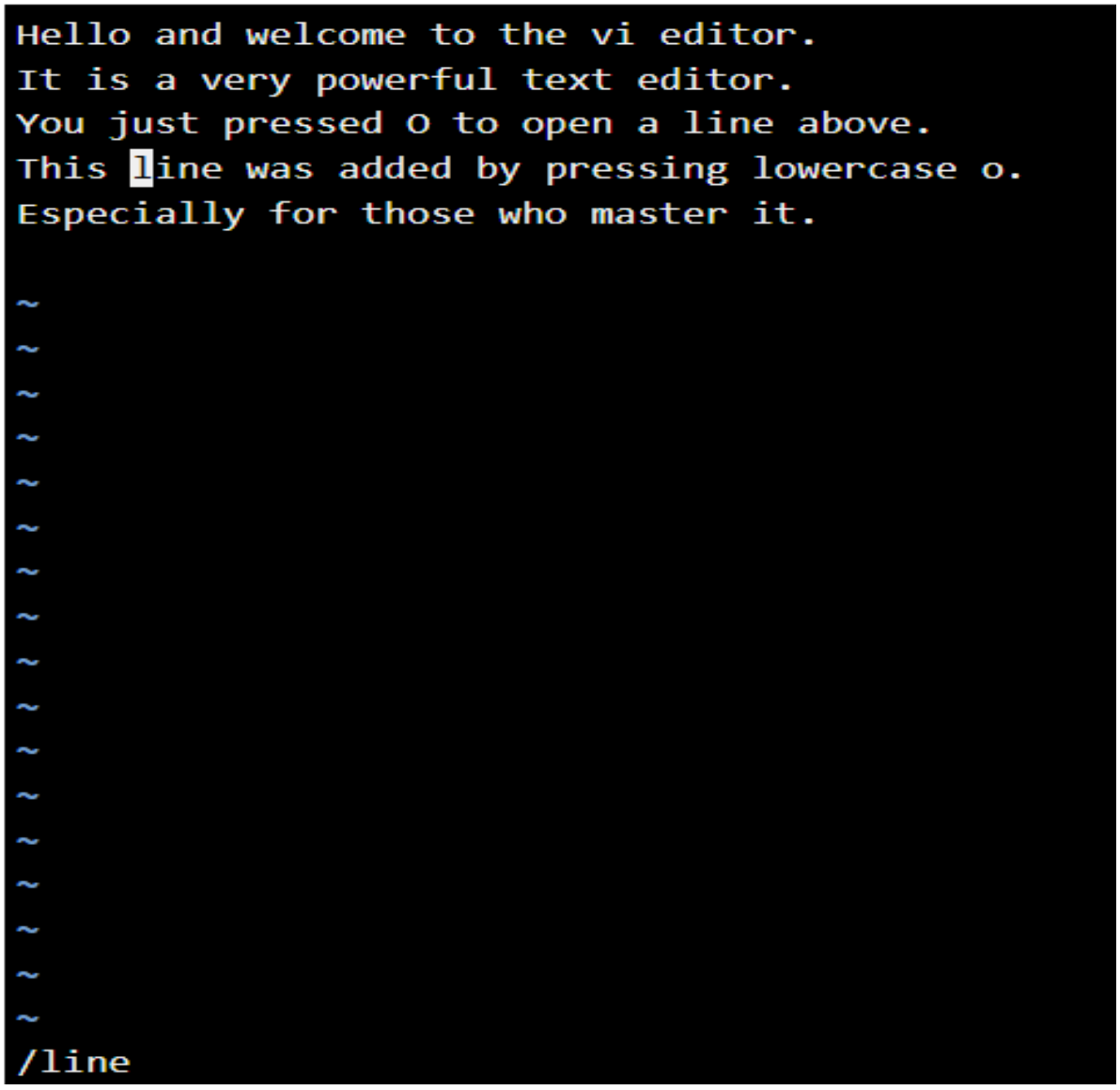


# 11.2.52 Step 52

Search for the next instance of the word `line` by pressing the letter `n`:

```
n
```

Your screen should look similar to the following:

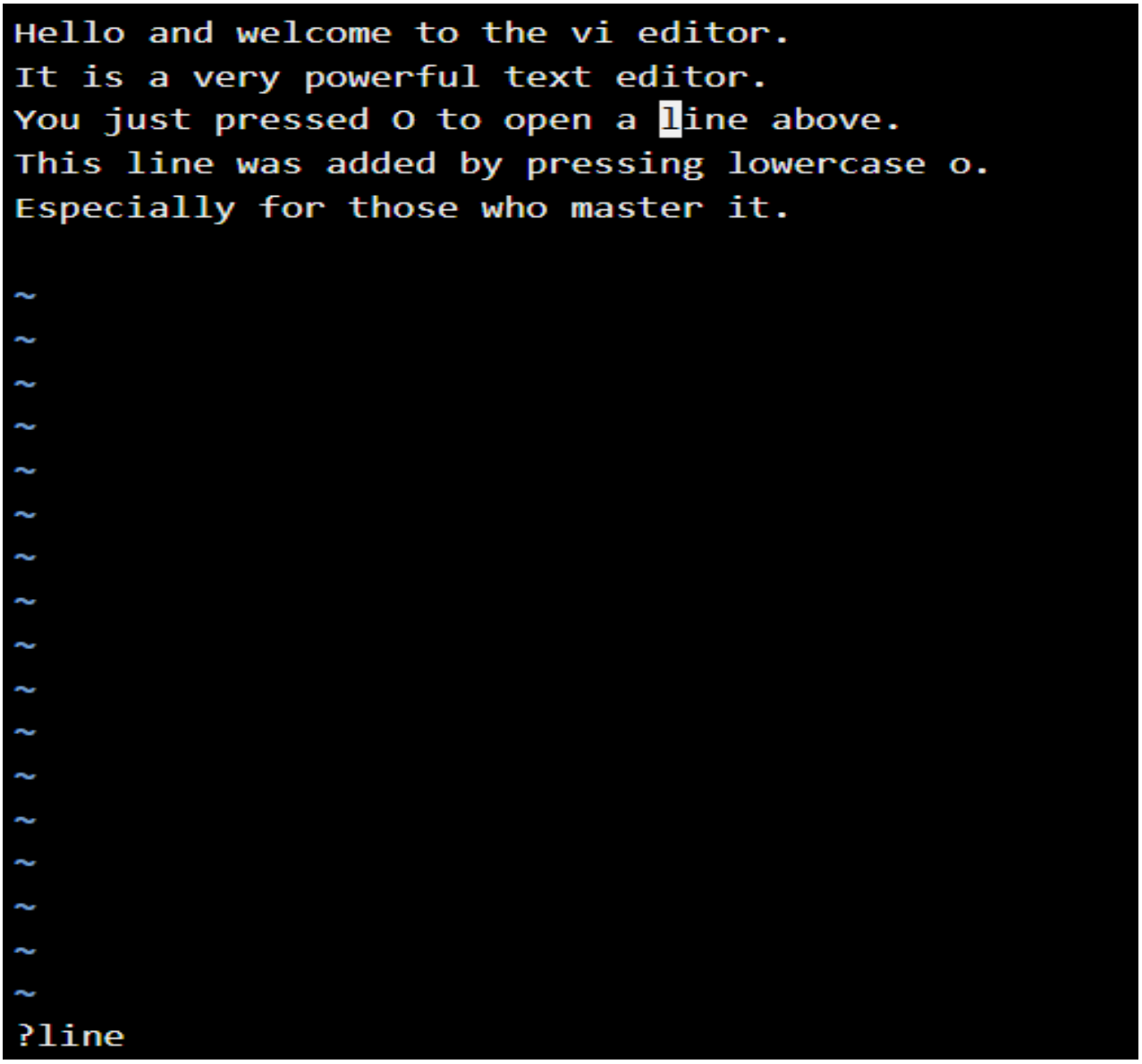


# 11.2.53 Step 53

Search backward for the word `line`. You'll notice the cursor moves to the beginning of the previous instance of the word `line` as shown in image below:

```
?line
```

Your screen should look similar to the following:

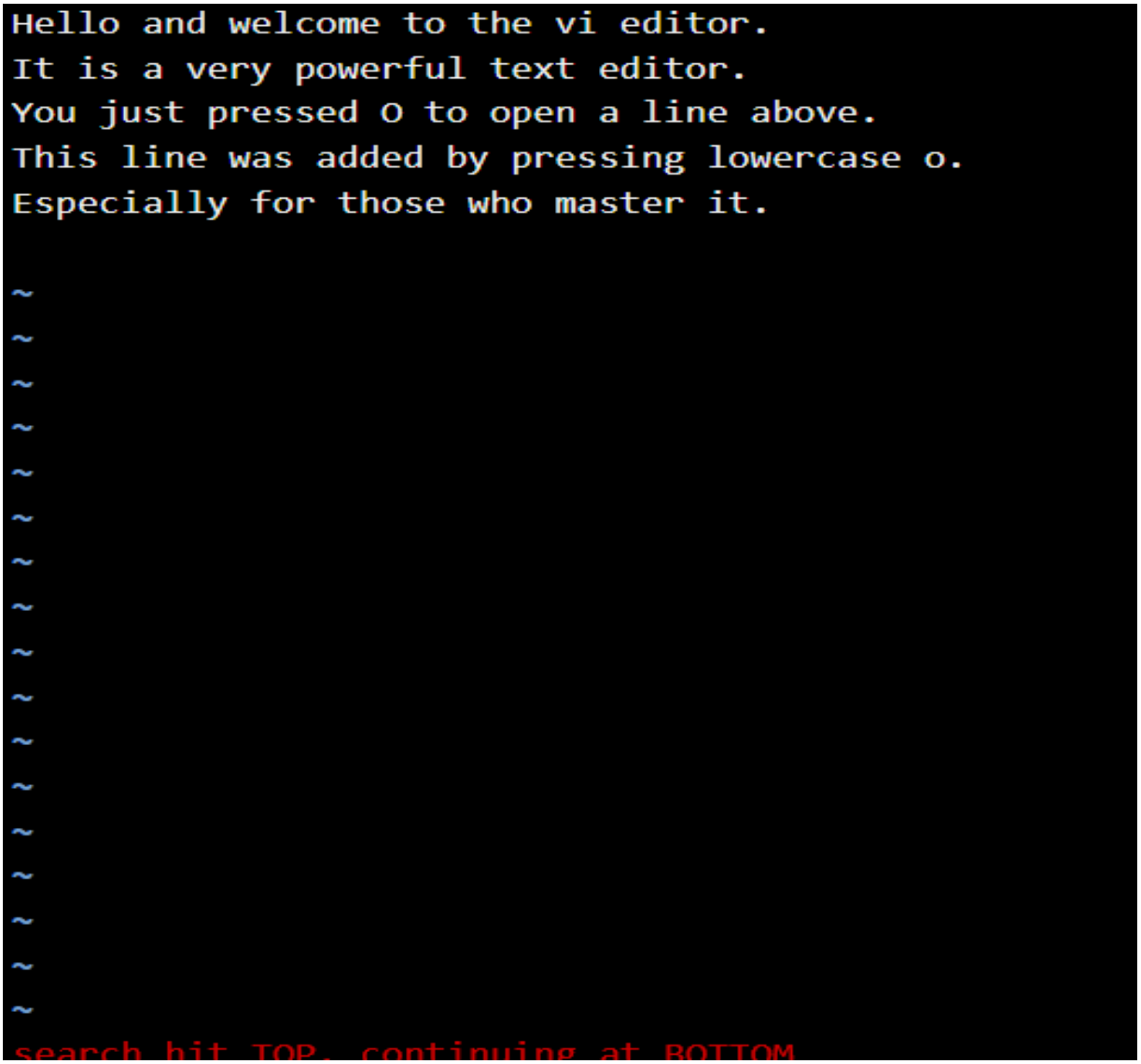


# 11.2.54 Step 54

Search for the previous instance of the word `line` by pressing the letter `n`. Since there are none in this direction, `vi` will wrap around the document:

```
n
```

Your screen should look similar to the following:

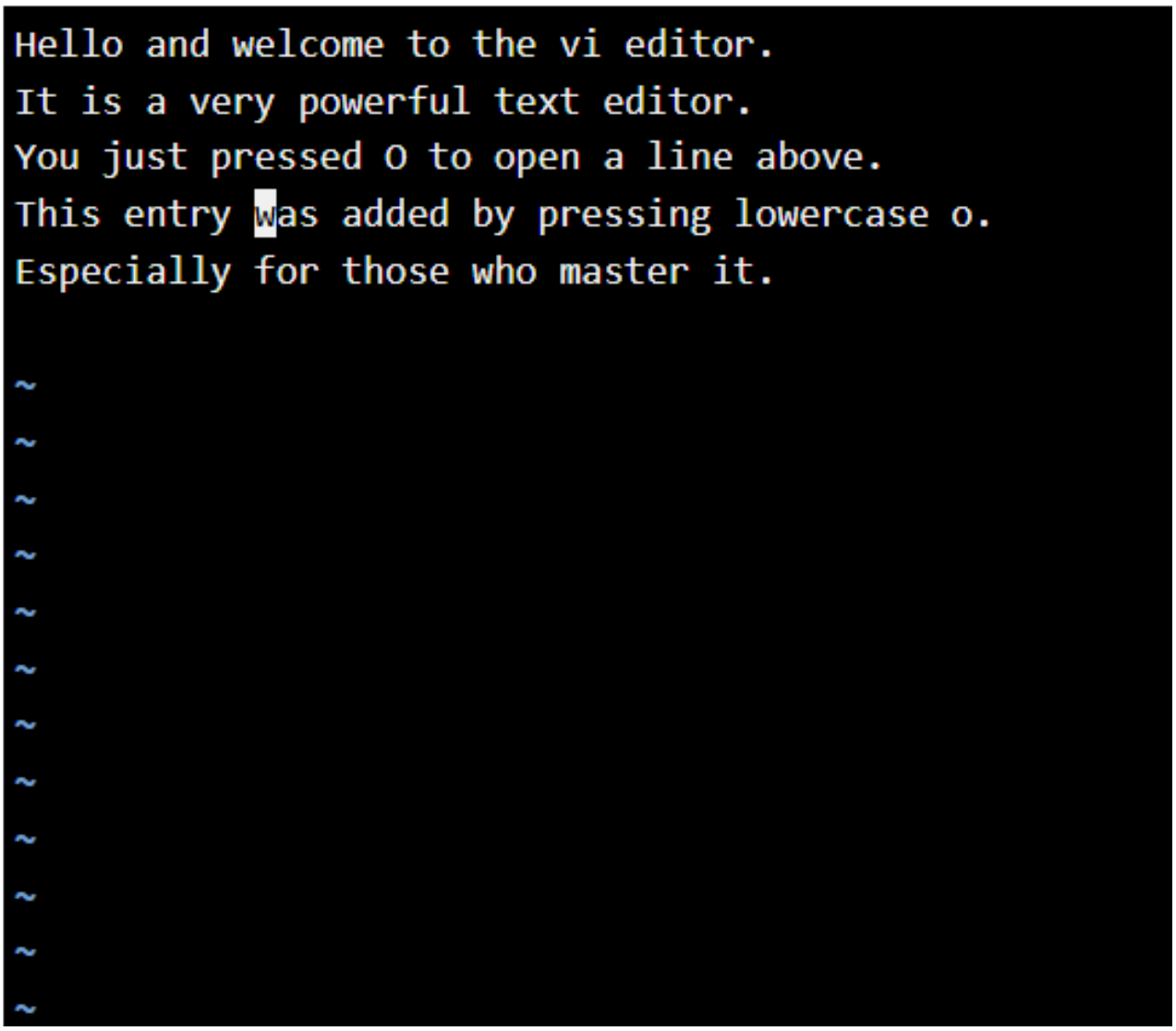


# 11.2.55 Step 55

You will replace the word `line` with the word `entry`. When you press `cw` you will be in insert mode and you will be able to type over the word `line`:

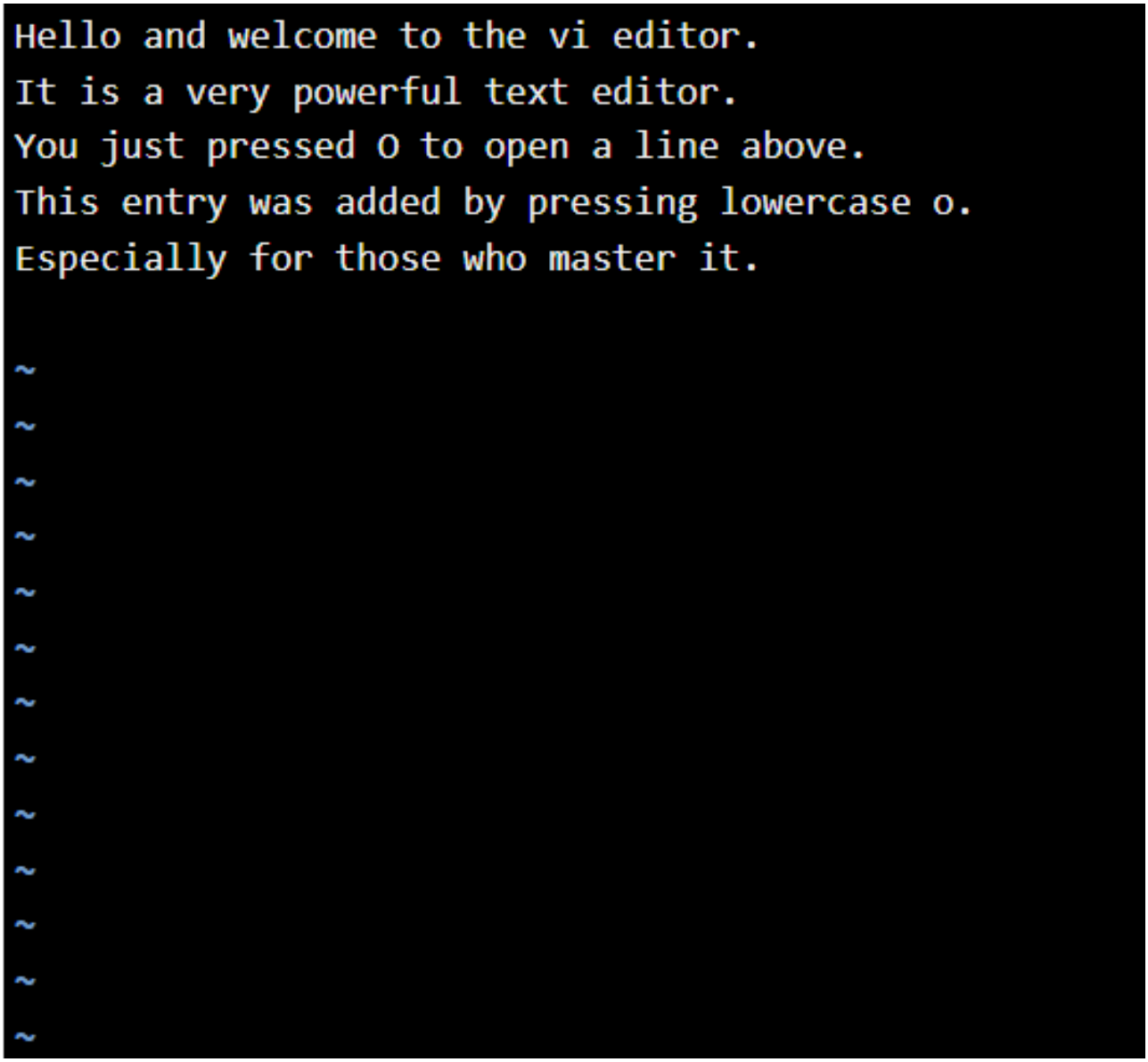
```
cw
entry
```

Your screen should look similar to the following:



# 11.2.56 Step 56

Press **Esc** key to exit insert mode.

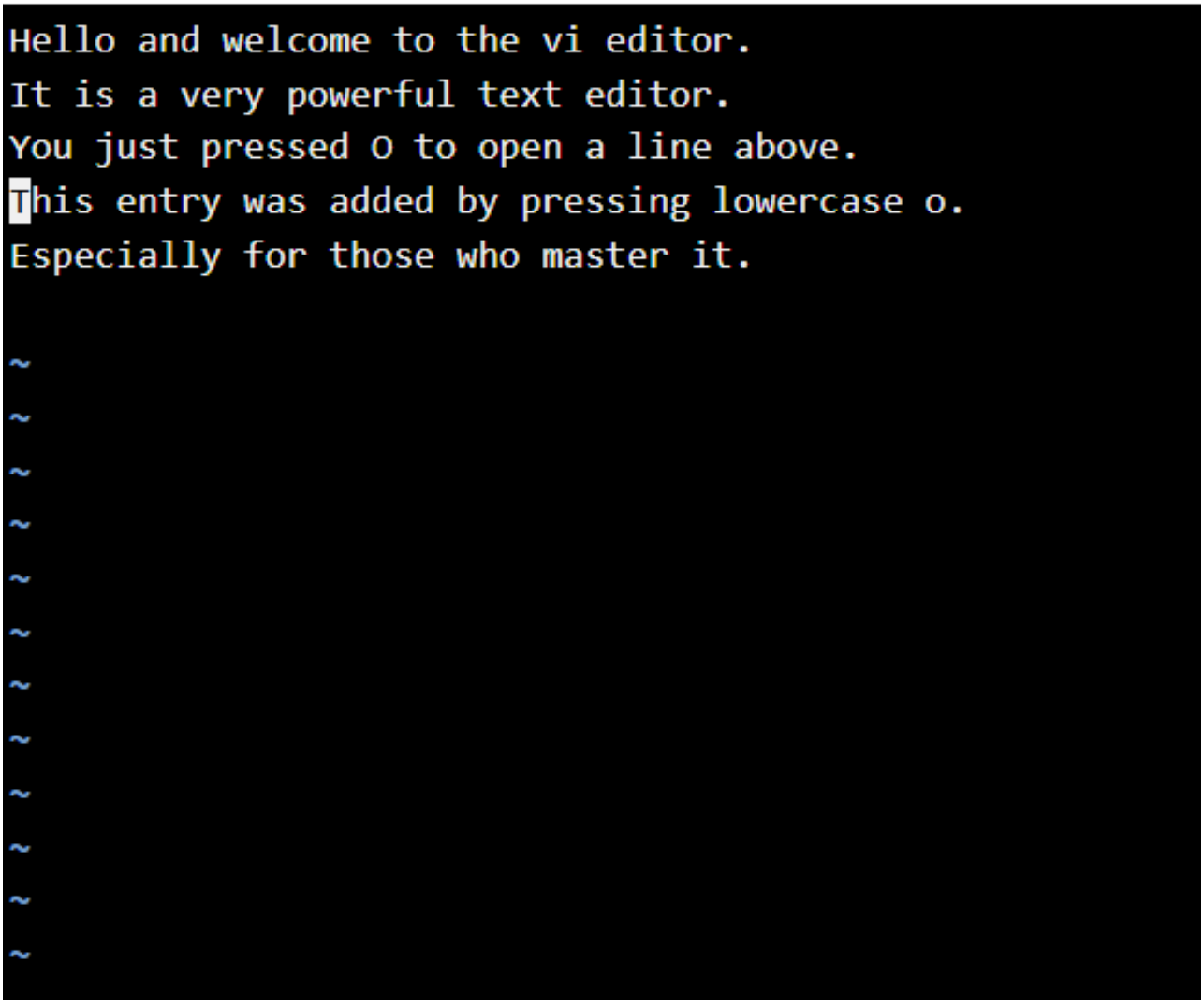


# 11.2.57 Step 57

Add text at the beginning of a line. Enter insert mode again and add a line by pressing upper case `i`:

```
I
```

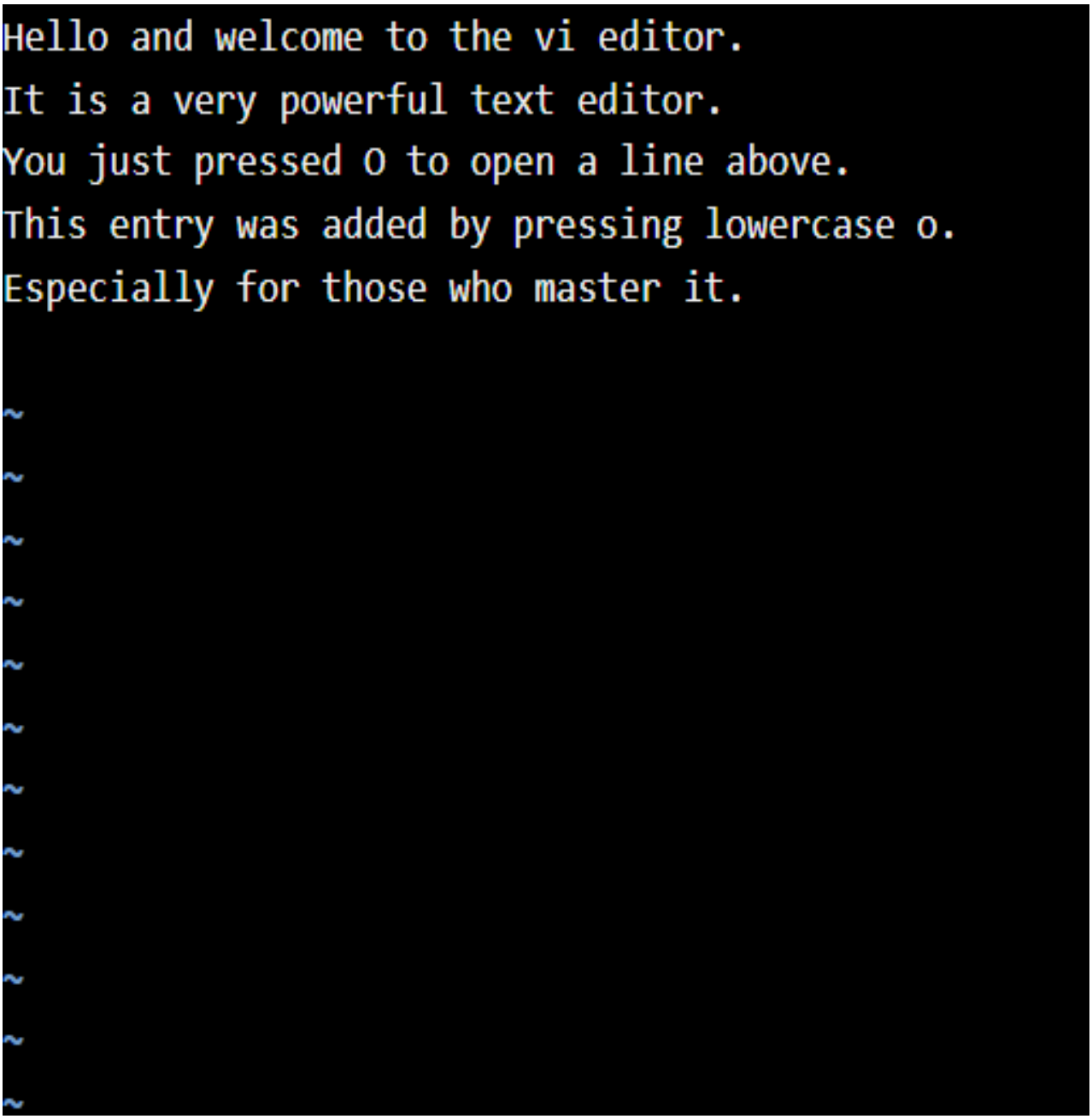
Your screen should look similar to the following:




Insert modes include: `i`, `I`, `a`, `A`, `o`, and `O`

# 11.2.58 Step 58

Press the **Esc** key to return to command mode.



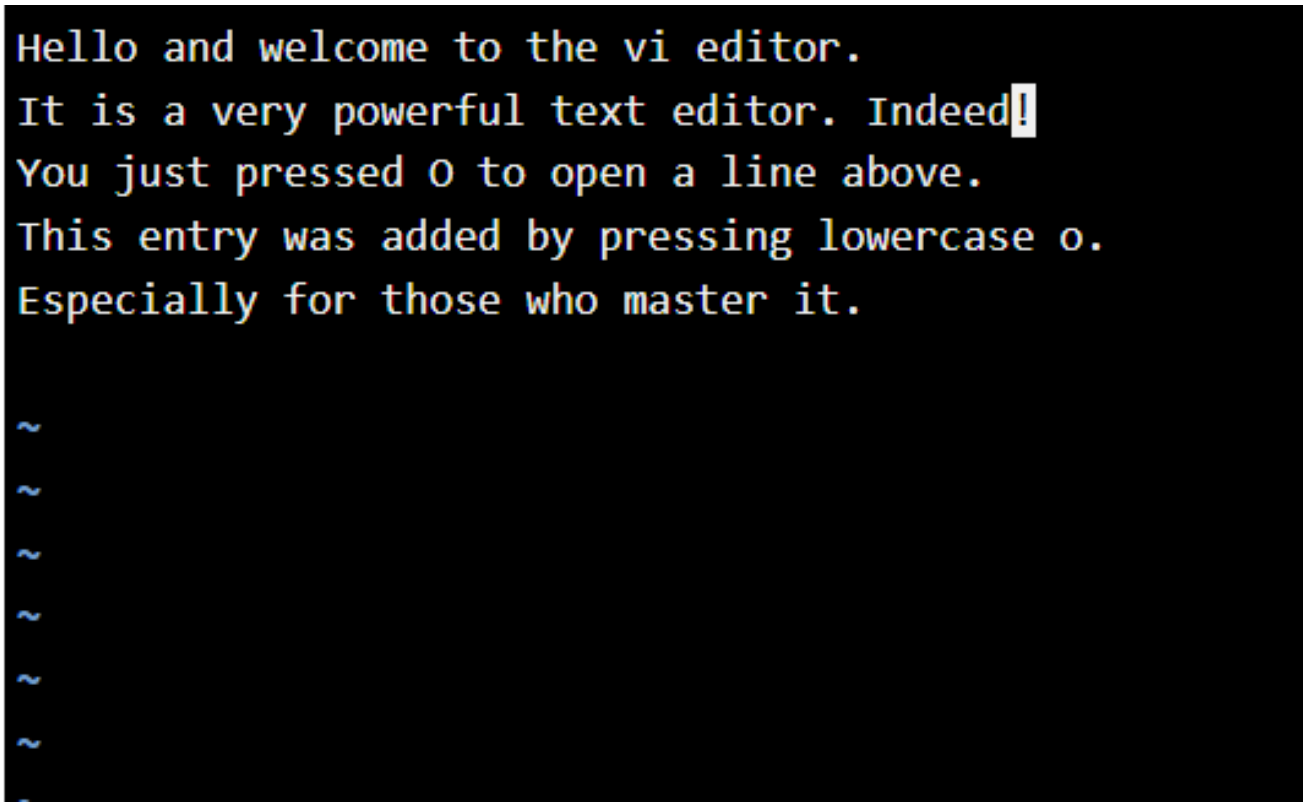
# 11.2.59 Step 59

Add text at the end of a line (uppercase `A`). First move to the second line and add the phrase `Indeed!`:

```
2G
A
[Space]Indeed!
```

Press the **Esc** key to return to command mode.

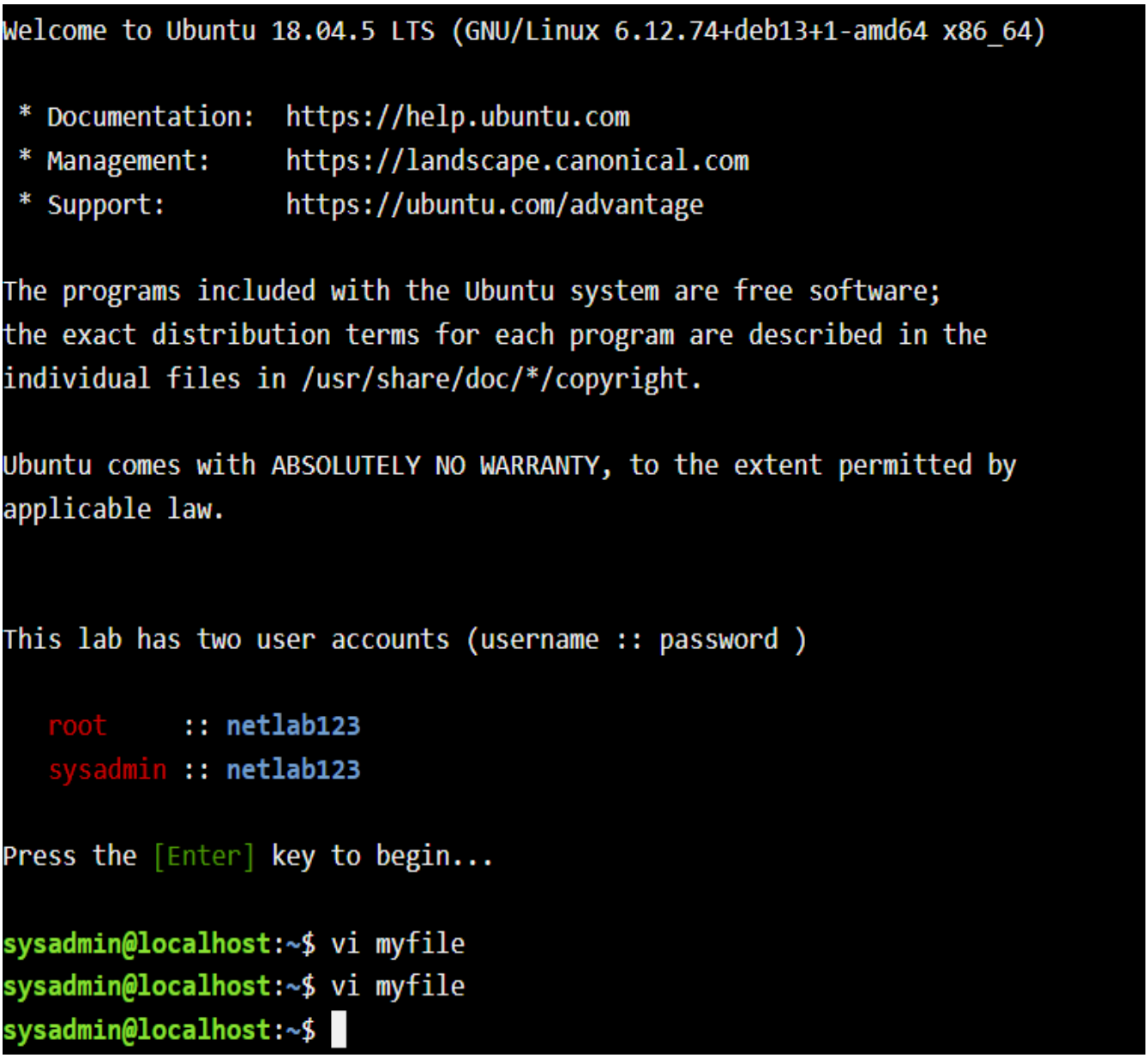
Your screen should look similar to the following:



# 11.2.60 Step 60

Save your changes and exit `vi`:

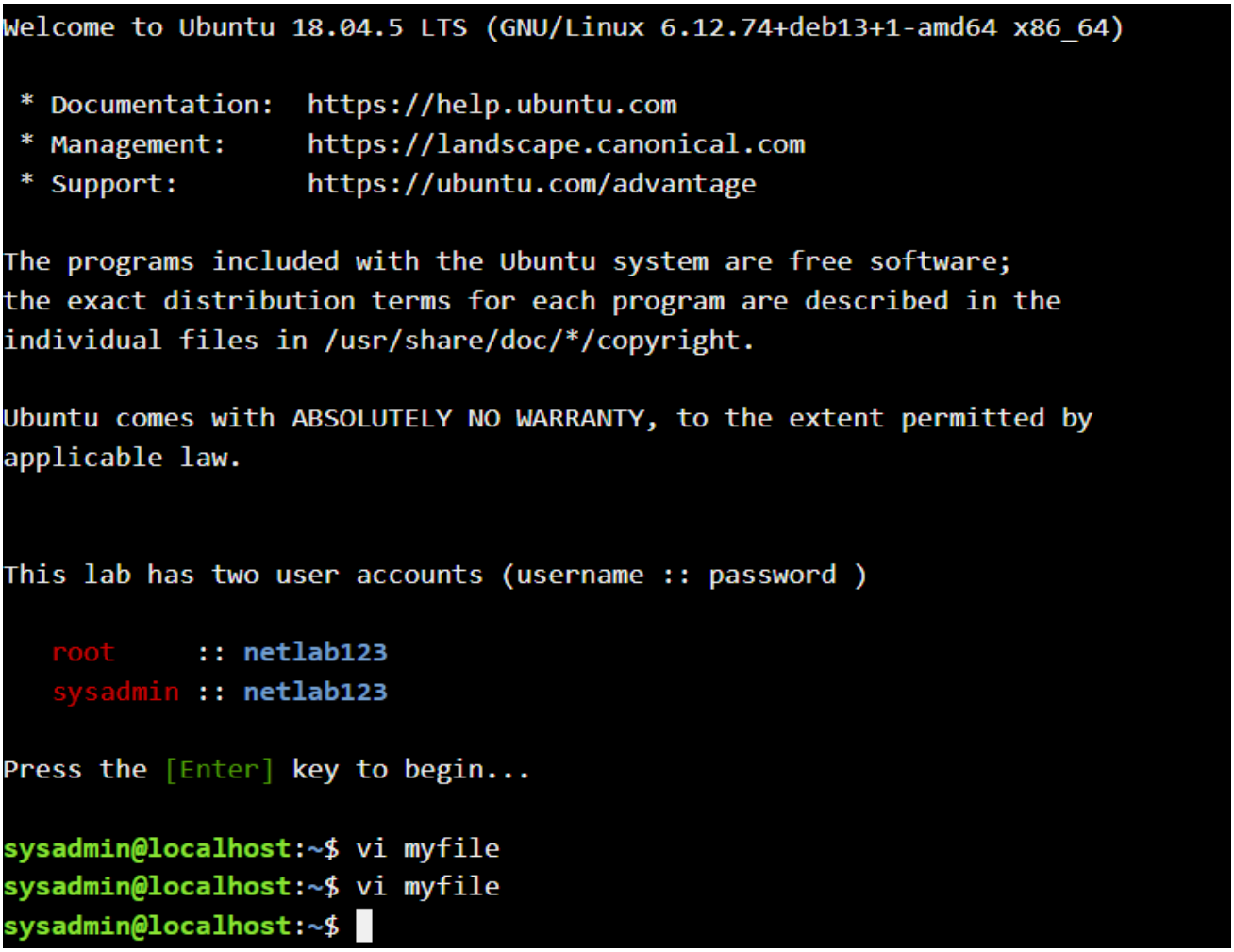
```
:x
```



# 11.3 Basic Shell Scripting

Shell scripting allows you to take a complex sequence of commands, place them into a file and then run the file as a program. This saves you the time of having to repeatedly type a long sequence of commands that you routinely use.

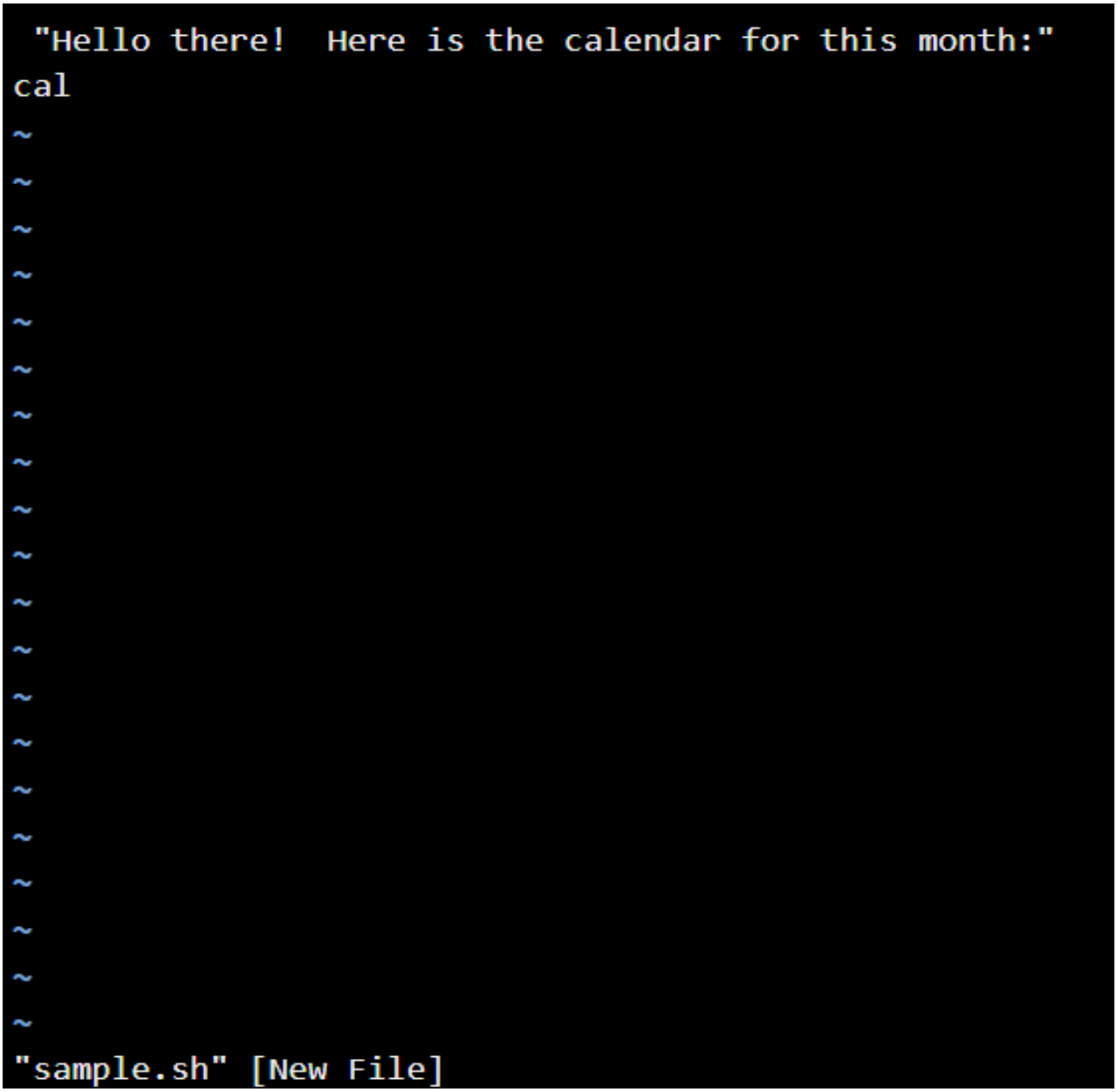
This lab will focus on how to create simple shell scripts. For the purpose of this lab, it is assumed that you know how to use a text editor. Feel free to use the editor of your choice: `vi`, `nano`, `gedit` or any other editor that you like.



# 11.3.1 Step 1

To create a simple shell script, you just need to create a text file and add commands. Create a file called `sample.sh` and add the following lines:

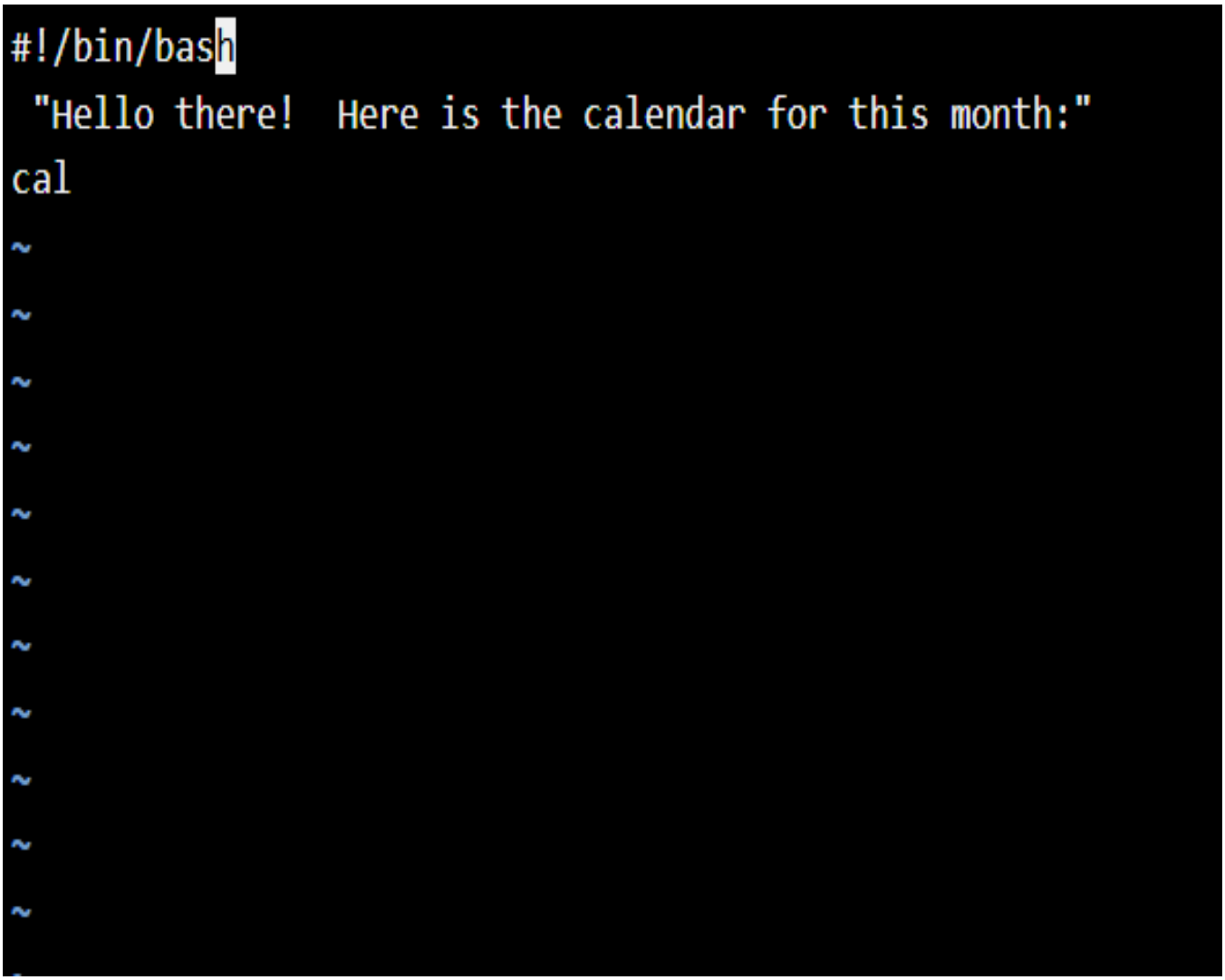
```
echo "Hello there!  Here is the calendar for this month:"
cal
```



# 11.3.2 Step 2

To make it clear that this is a BASH shell script, you need to include a special line at the top of the file called a "shbang" (or "shebang"). This line starts with `#!` and then contains the path to the BASH shell executable. Add the following line at the top of the `sample.sh` file:

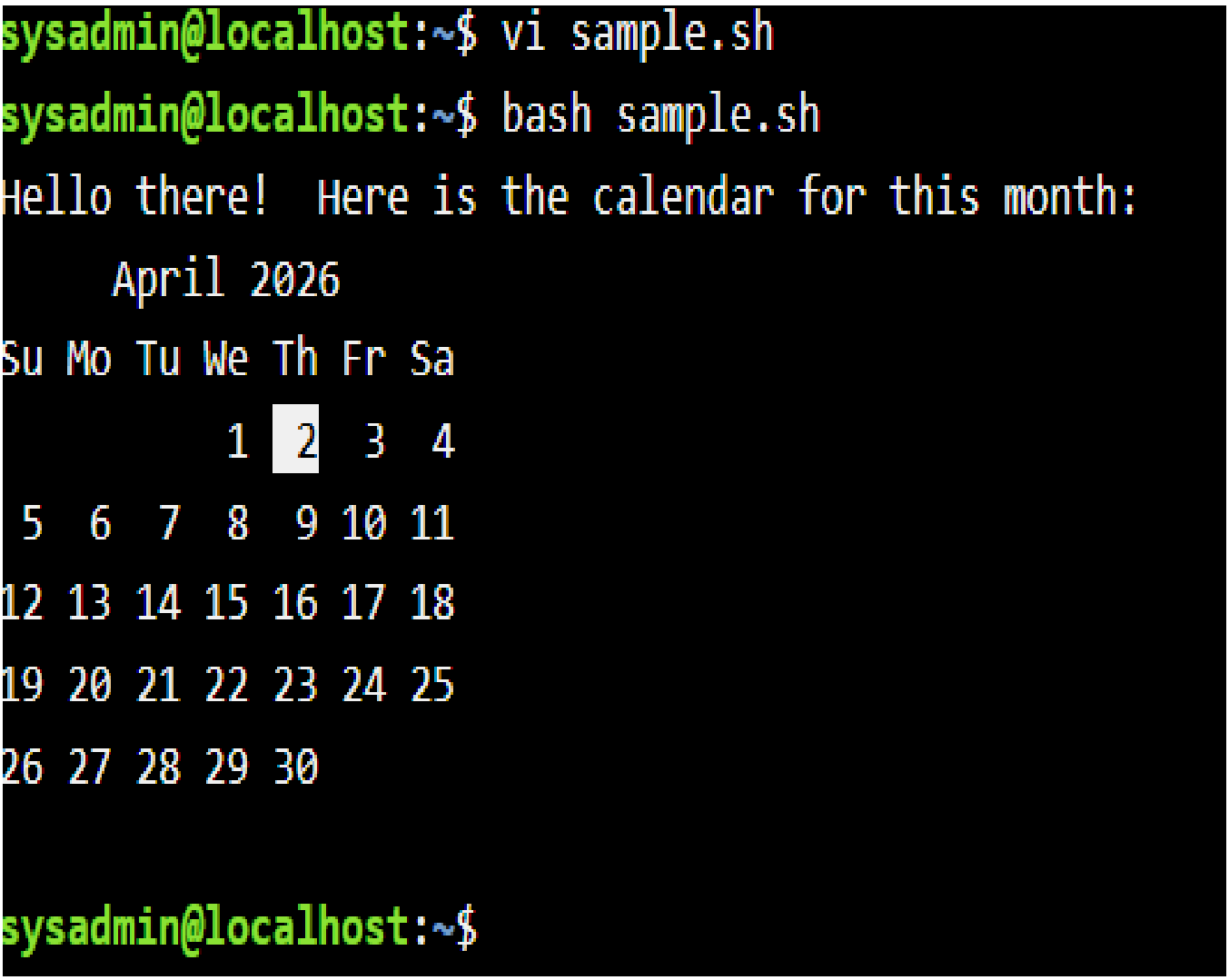
```
#!/bin/bash
```



# 11.3.3 Step 3

One way that you can run this program is by typing `bash` before the filename. Execute the following:

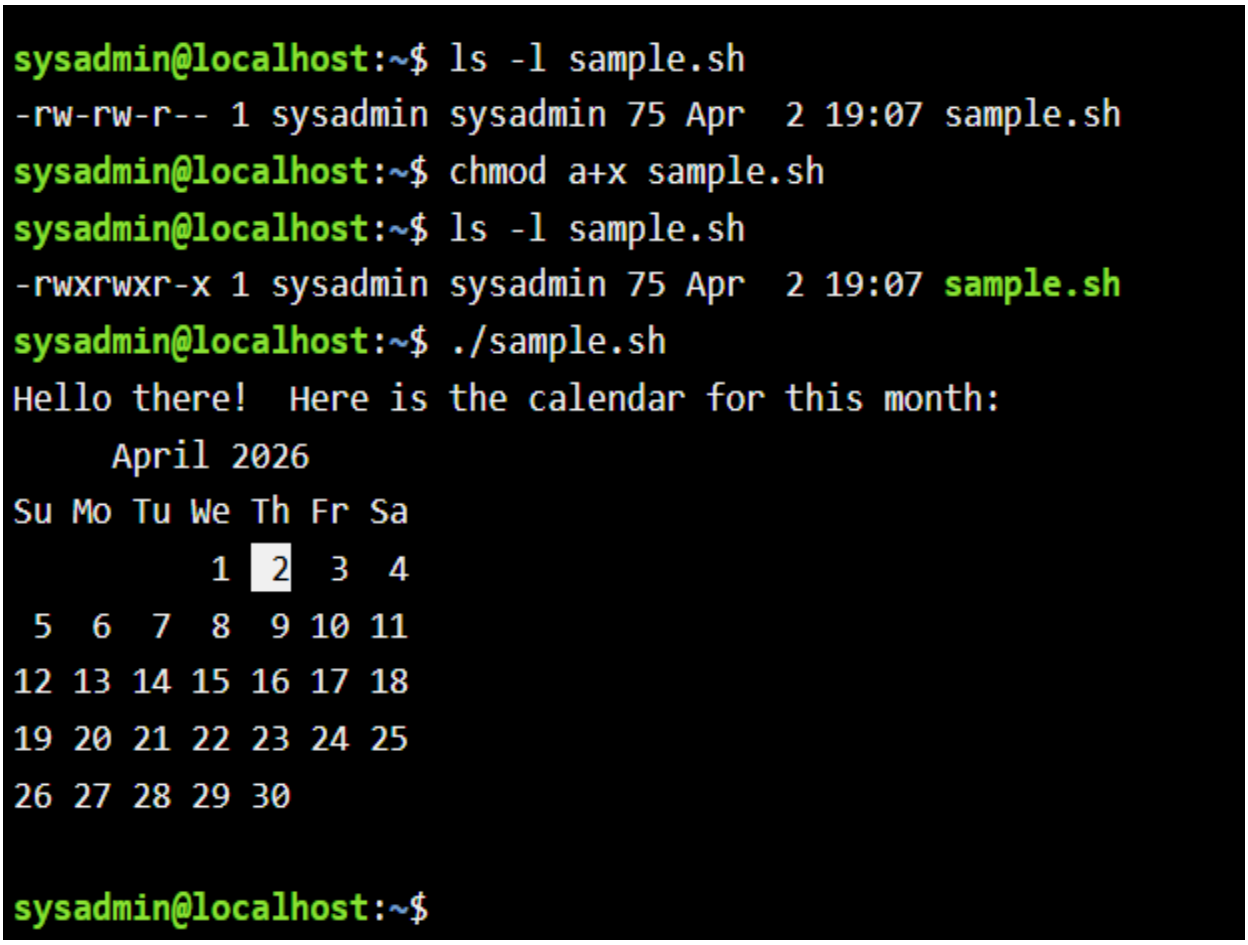
```
bash sample.sh
```



# 11.3.4 Step 4

You can avoid having to type bash in front of the filename by making the file "executable" for all users. Run the following commands:

```
ls -l sample.sh
chmod a+x sample.sh
ls -l sample.sh
./sample.sh
```



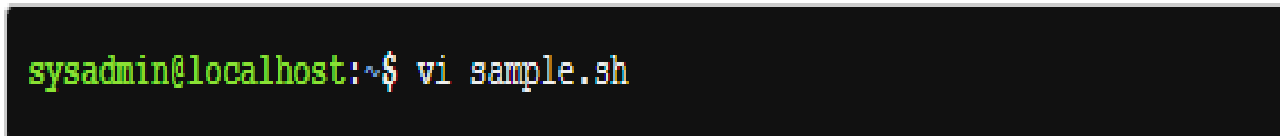

The `chmod` command is used to change permissions on the file so that the file can be executed.

# 11.3.5 Step 5

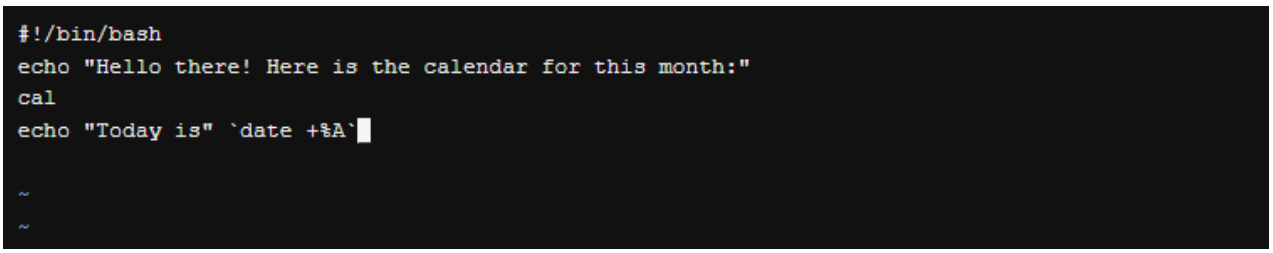
A common feature used in scripting is "backquoting". With this technique, you can run a shell command "within" another shell command. The outcome of the internal command will be returned as an argument to the external command.

Add the following to the bottom of the `sample.sh` file:

```
vi sample.sh
```

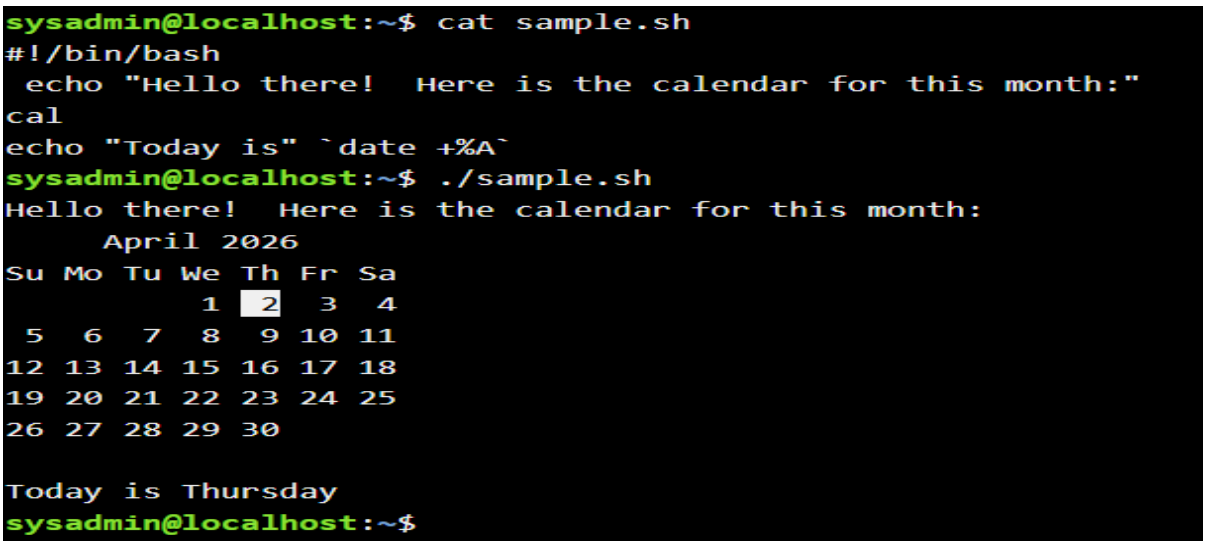



```
echo "Today is" `date +%A`
```




Exit insert mode by pressing **Esc**, then type `:wq!` and hit **Enter** to save and exit the file.

```
cat sample.sh
./sample.sh
```

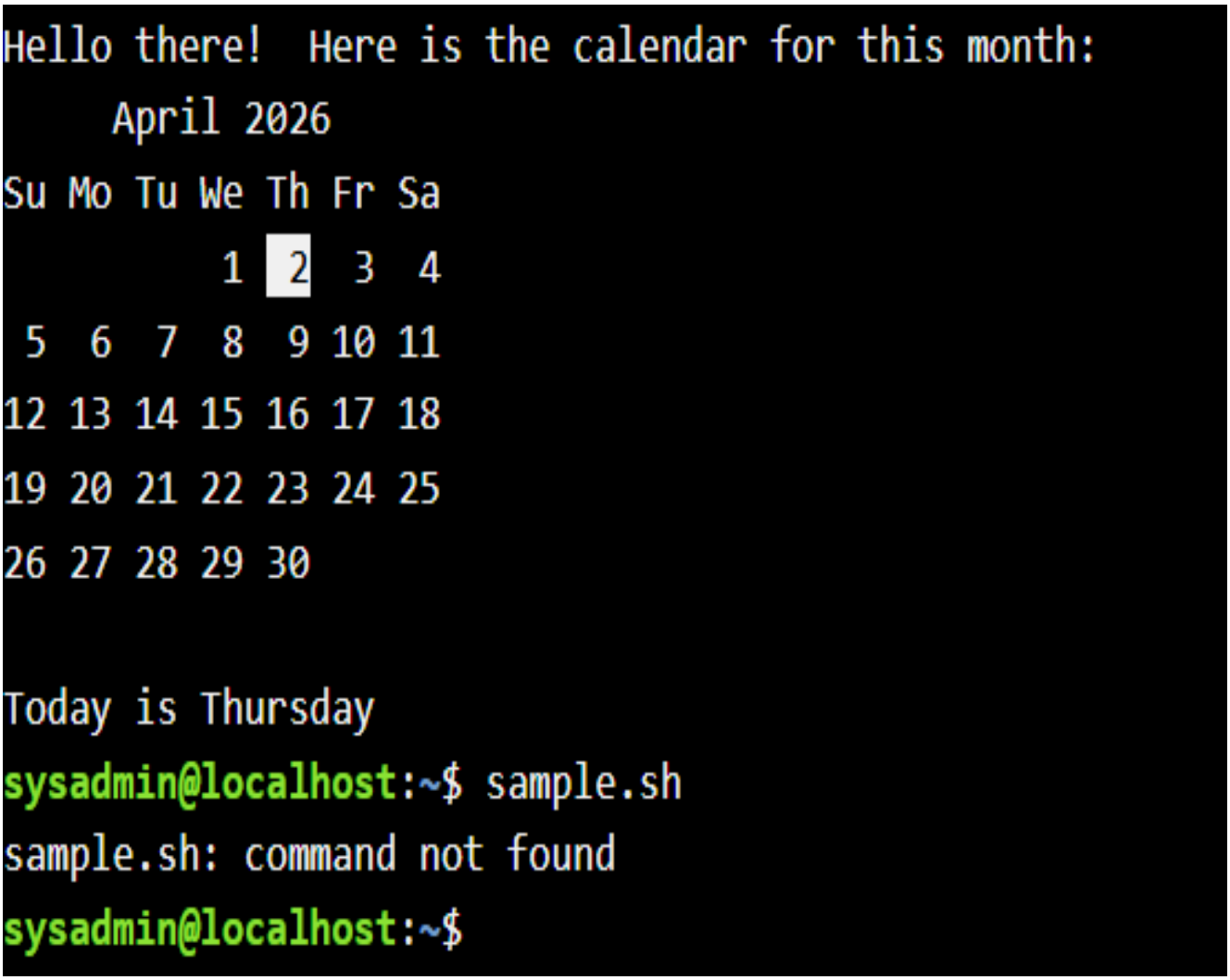


# 11.3.6 Step 6

You have been using `./` in front of the `sample.sh` filename to indicate that the file is in the current directory. Execute the following to see how the shell would fail to find the file if you don't use the `./`:

```
sample.sh
```

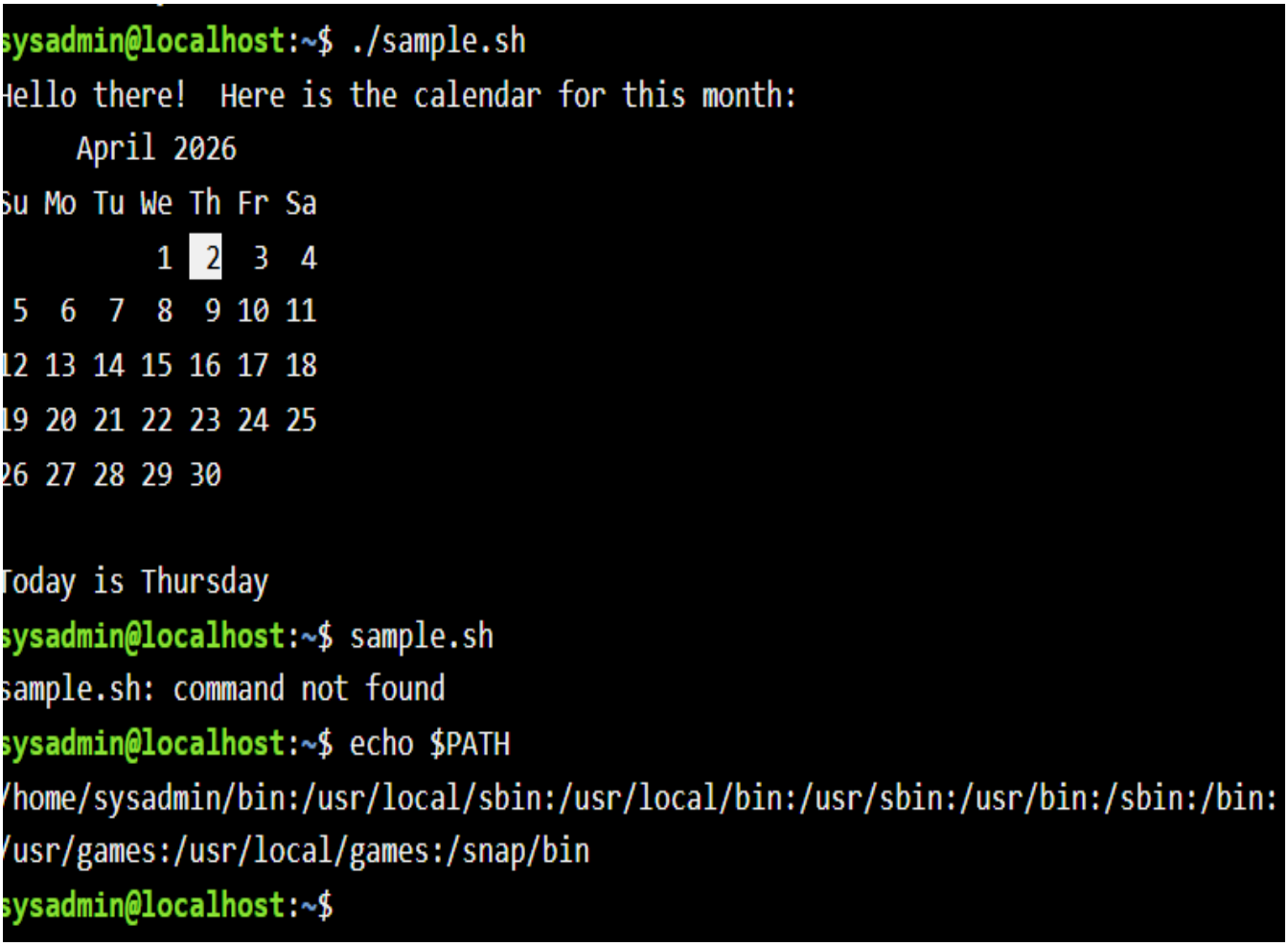
Your screen should look like the following:



# 11.3.7 Step 7

Recall that the `$PATH` variable is used to search for commands that you type. Execute the following to see the `$PATH` variable for the sysadmin account:

```
echo $PATH
```



# 11.3.8 Step 8

Note that `/home/sysadmin/bin` is one of the directories in the `$PATH` variable. This is a great place to put your shell scripts:

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/snap/bin
sysadmin@localhost:~$ mkdir bin
sysadmin@localhost:~$ mv sample.sh bin
sysadmin@localhost:~$ sample.sh
Hello there!  Here is the calendar for this month:
     April 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

Today is Thursday
sysadmin@localhost:~$
```

# 11.4 Conditional and Repetitive Execution

Note that in this section examples that are more complex will be demonstrated. When doing so, you will be using a technique to describe what is happening in the program. The technique will look like the following:

| Enter this column into drive.sh | This column describes the code (don't enter into the file) |
|---|---|
| `echo "Please enter your age"` | # print a prompt |
| `read age` | # read user input and place in $age variable |

When following the instructions provided, you are to enter the text from the left column into the specified file (`drive.sh` in the example above). The right column is used to describe specific lines in the program. The pound (hash) sign `#` character is used because in a shell script you can place comments within your program by using a `#` character.

# 11.4.1 Step 1

Scripts that are more complex may make use of conditional execution. A conditional expression, like the `if` statement, can make use of the outcome of a command called `test`. The `test` statement compares two numbers (or two strings) for things like "equal to", "less than", etc.

Create the following `drive.sh` file and make it executable to see how the `if` and `test` statements work.

```
vi drive.sh
```

Begin by placing the following in `drive.sh`:

| Enter this column into drive.sh | This column describes the code (do not enter into the file) |
|---|---|
| `#!/bin/bash` | |
| `echo "Please enter your age"` | # print a prompt |
| `read age` | # read user input and place in the `$age` variable |
| `if test $age -lt 16` | # `test $age -lt 16` returns "true" if `$age` is numerically less than 16 |
| `then` | |
| `  echo "You are not old enough to drive."` | # executes when `test` is true |
| `else` | |
| `  echo "You can drive!"` | # executes when `test` is false |
| `fi` | # This ends the `if` statement |

Then make the file executable and run it:

```
cat drive.sh
chmod a+x drive.sh
./drive.sh
```

Your screen should look like the following:

```
sysadmin@localhost:~$ cat drive.sh
#!/bin/bash
echo "Please enter your age"
read age
if test $age -lt 16
then
    echo "You are not old enough to drive."
else
    echo "You can drive!"
fi
sysadmin@localhost:~$ chmod a+x drive.sh
sysadmin@localhost:~$ ./drive.sh
Please enter your age
14
You are not old enough to drive.
sysadmin@localhost:~$
```

Verbally, you could read the `if` statement as *If `$age` is less than 16, then `echo` 'You are not old enough to drive', else `echo` 'You can drive!'*. The `fi` ends the `if` statement.

# 11.4.2 Step 2

The test statement is automatically called when you place its arguments within square brackets `[ ]` surrounded by spaces. Modify the `if` line of `drive.sh` so it looks like the following:

```
if [ $age -lt 16 ]
```

Then run the program again:

```
cat drive.sh
./drive.sh
```

Your screen should look like the following:

```
sysadmin@localhost:~$ cat drive.sh
#!/bin/bash
echo "Please enter your age"
read age
if [ $age -lt 16 ]
then
   echo "You are not old enough to drive."
else
   echo "You can drive!"
fi
sysadmin@localhost:~$ ./drive.sh
Please enter your age
21
You can drive!
sysadmin@localhost:~$
```

To see a full list of test conditions, run the command `man test`.

**Important**

There must be spaces around the square brackets. `[$age -lt 16]` would fail, but `[ $age -lt 16 ]` would work.

# 11.4.3 Step 3

You can also use the outcome of other shell commands as they all return "success" or "failure". For example, create and run the following program, which can be used to determine if a user account is on this system.

create a file called `check.sh`:

```
vi check.sh
```

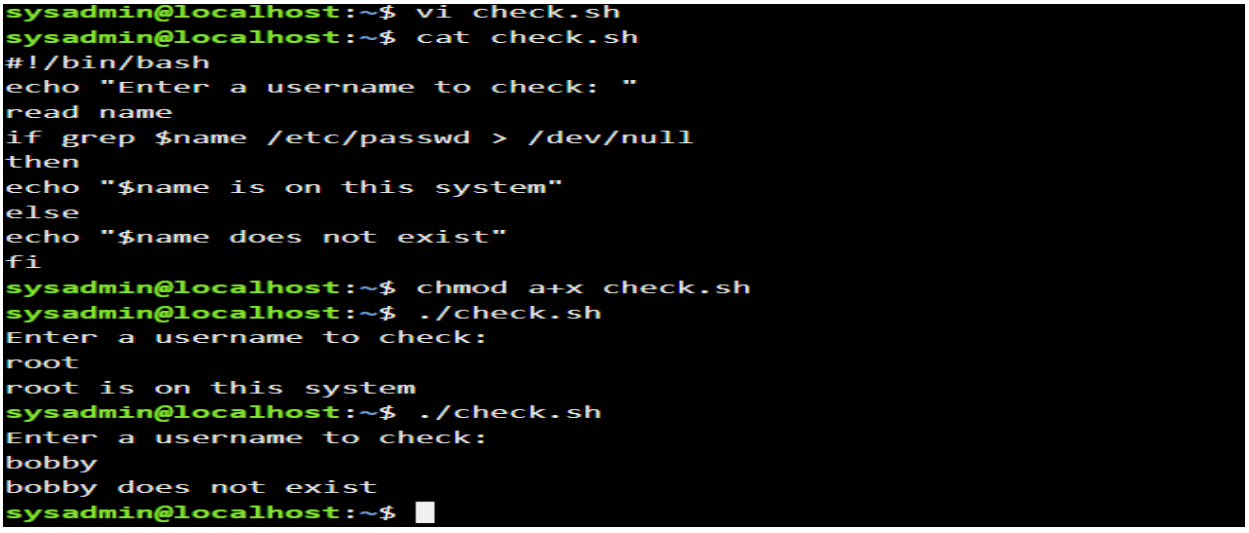

Add the following to `check.sh`:

```
#!/bin/bash
echo "Enter a username to check: "
read name
if grep $name /etc/passwd > /dev/null
then
    echo "$name is on this system"
else
    echo "$name does not exist"
fi
```

Then run the following commands:

```
cat check.sh
chmod a+x check.sh
./check.sh
```

When prompted for a username, give the value of "`root`". Execute the command again (`./check.sh`) and provide the value of "`bobby`". Your screen should look like the following:



# 11.4.4 Step 4

Another common conditional statement is called the `while` loop. This statement is used to execute code repeatedly as long as a conditional check returns "true". Begin by creating a file called `num.sh`:

```
vi num.sh
```

| Enter this into the *num.sh* file | This describes the code (don't enter into the file) |
|---|---|
| `#!/bin/bash` | |
| `echo "Please enter a number greater than 100"` | |
| `read num` | |
| `while [ $num -le 100 ]` | |
| `do` | # Execute code from `do` to `done` if test condition is "true" |
| `  echo "$num is NOT greater than 100."` | |
| `  echo "Please enter a number greater than 100"` | |
| `  read num` | |
| `done` | # This ends the `while` statement |
| `echo "Finally, $num is greater than 100"` | |

Then make the file executable and run it:

```
cat num.sh
chmod a+x num.sh
./num.sh
```

When prompted for a number, enter 25. When prompted again, enter 99. Finally, enter 101 when prompted for a number the third time. Your screen should look like the following:

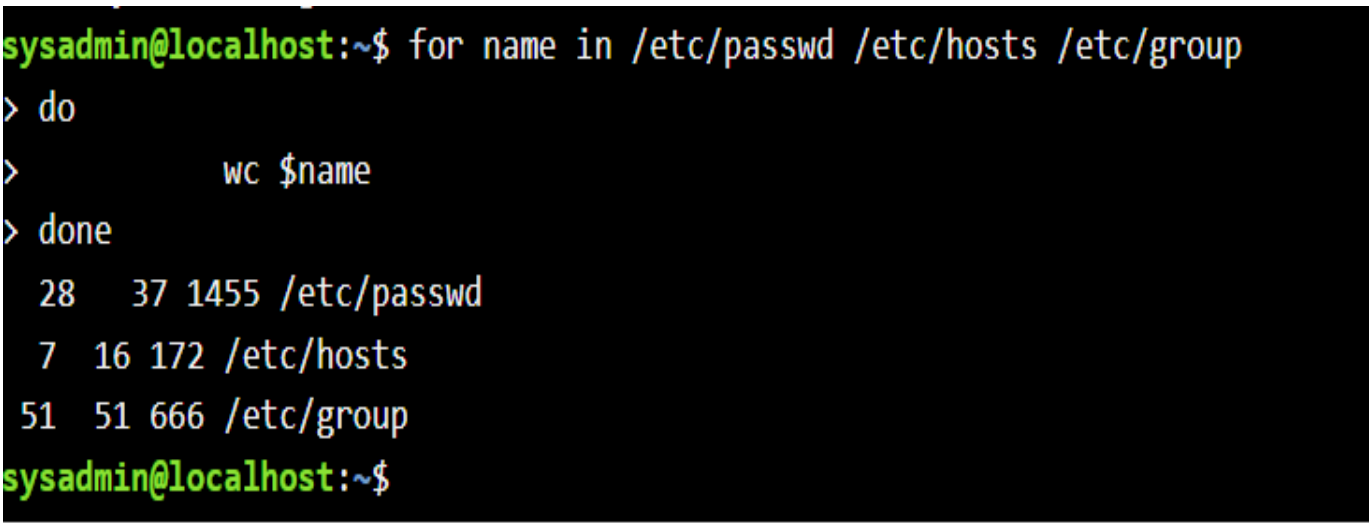
```
sysadmin@localhost:~$ cat num.sh
#!/bin/bash
echo "Please enter a number greater than 100"
read num
while [ $num -le 100 ]
do
echo "$num is NOT greater than 100."
echo "Please enter a number greater than 100"
 read num
done
echo "Finally, $num is greater than 100"
sysadmin@localhost:~$ chmod a+x num.sh
sysadmin@localhost:~$ ./num.sh
Please enter a number greater than 100
25
25 is NOT greater than 100.
Please enter a number greater than 100
99
99 is NOT greater than 100.
Please enter a number greater than 100
101
Finally, 101 is greater than 100
sysadmin@localhost:~$
```

# 11.4.5 Step 5

Scripting code is part of the BASH shell, which means you can use these statements on the command line just like you use them in a shell script. This can be useful for a statement like the `for` statement, a statement that will assign a list of values one at a time to a variable. This allows you to perform a set of operations on each value. For example, run the following on the command line:

```
for name in /etc/passwd /etc/hosts /etc/group
do
          wc $name
done
```

Your screen should look like the following:




Note that the `wc` command was run three times: once for `/etc/passwd`, once for `/etc/hosts` and once for `/etc/group`.

# 11.4.6 Step 6

Often the `seq` command is used in conjunction with the `for` statement. The `seq` command can generate a list of integer values, for instance from 1 to 10. For example, run the following on the command line to create 12 files named `test1`, `test2`, `test3`, etc. (up to `test12`):

```
ls
for num in `seq 1 12`
do
          touch test$num
done
ls
```